

Formalización en Isabelle/HOL de aplicaciones combinatoriales del Teorema de Compacidad para la lógica proposicional

Fabián Fernando Serrano Suárez

Facultad de Ciencias Exactas y Naturales
Departamento de Matemáticas y Estadística
Universidad Nacional de Colombia, Manizales

Índice

1	Introducción	2
2	Sintaxis y Semántica Proposicional	4
3	Teorema de Compacidad	7
4	Coloración de grafos	9
5	Lema de König	15
6	Conclusiones y trabajos futuros	25
7	Anexo 1. Teoría Compacidad	27
8	Anexo 2. Teoría Coloración	36
9	Anexo 3. Teoría König	56

1 Introducción

Por formalizar demostraciones matemáticas entendemos, de manera general, el escribir definiciones y pruebas en un lenguaje de ordenador de tal forma que estas puedan ser verificadas formalmente por una “máquina”. Este estudio mecánico de las matemáticas tiene su inicio con la prueba automática de teoremas; en particular, con los trabajos de Gödel, Skolem, Church, Kleene, Turing, Herbrand y Löwenheim. En 1930 J. Herbrand, en su artículo “Investigations in proof theory”, propone un importante método mecánico para probar teoremas. A partir de este, se derivaron una serie de procedimientos modernos de prueba de teoremas, que han dado origen a toda una teoría de prueba automática de teoremas [4].

Actualmente se ha mostrado que la prueba interactiva de teoremas es una técnica apropiada para el desarrollo de teorías formales. Por ejemplo, en la verificación de la demostración, es decir, en la certificación de la validez de una demostración existente de un teorema. Específicamente los demostradores interactivos de teoremas requieren de un usuario humano para orientar al sistema. Dependiendo del grado de automatización, el demostrador puede ser reducido a un verificador de la demostración, en el que el usuario propor-

ción la demostración de una manera formal, o puede ser usado en tareas importantes de la demostración que se pueden realizar automáticamente. Los sistemas de pruebas automáticas han logrado demostrar una cantidad de teoremas interesantes y difíciles, por ejemplo, el Teorema de los Cuatro Colores. En [13] se hace una descripción de 100 teoremas importantes junto con los respectivos sistemas que han formalizado su demostración.

Entre los sistemas de pruebas automáticas que más han sido utilizados exitosamente por los usuarios [14], se encuentran: HOL, Coq, Isabelle, PVS, y Mizar. El aspecto más notable del sistema Mizar, a diferencia de los otros sistemas, es su lenguaje de prueba estructurada, el cual ha sido diseñado para representar de manera formal patrones comunes de pruebas matemáticas. Esta característica ha motivado posteriores investigaciones en el diseño de sistemas de pruebas formales entendibles por las personas. En este sentido, el ambiente Isabelle/Isar [12], ha sido implementado con el fin de ser una herramienta para la generación automática de documentos de pruebas formales legibles por las personas, que son compuestos por el usuario y verificados por la máquina.

Isabelle es un asistente de prueba genérico; Isabelle/Hol es una especialización de Isabelle con lógica de orden superior (HOL) [9], e Isabelle/Isar es una extensión de Isabelle con pruebas estructuradas en el lenguaje Isar ("Intelligible semi-automated reasoning"), similares a las pruebas usuales en matemáticas [7].

En la página web [Archive of Formal Proofs](#) se encuentra una colección de librerías de resultados verificados mecánicamente en Isabelle.

En este trabajo se ilustra cómo se elaboran documentos de pruebas formales y estructuradas en el lenguaje Isabelle/Isar.

Entre los teoremas más importantes de la Lógica Matemática se destaca el Teorema de Compacidad:

Teorema 1.1 (Teorema de Compacidad). *Sea Γ un conjunto infinito de fórmulas tal que todo subconjunto finito de Γ es satisfacible, entonces Γ es satisfacible.*

En un trabajo anterior hemos formalizado en Isabelle/Isar este teorema, para conjuntos enumerables de fórmulas en la lógica proposicional, orientados por la prueba que aparece en el texto de lógica de Fitting [1].

El objetivo específico de este trabajo es el desarrollo de la automatización de aplicaciones del Teorema de Compacidad para la Lógica de Proposiciones. Se han seleccionado dos casos de estudio que corresponden a conceptos fundamentales en la solución de problemas combinatoriales en el área de

Matemáticas Discretas, el teorema de k -coloramiento de grafos enumerables y el Lema de König.

Teorema 1.2 (Erdős). *Un grafo G infinito enumerable es k -colorable si todo subgrafo finito de G es k -colorable.*

Teorema 1.3 (König). *Todo árbol infinito enumerable de ramificación finita tiene una rama infinita.*

El desarrollo de las formalizaciones de estos resultados está guiado por las pruebas clásicas que se presentan en los textos de matemáticas.

En la bibliografía consultada no se encontró un trabajo similar al que aquí se presenta. En este sentido, este trabajo es un aporte original al estudio de la automatización de importantes teoremas de las matemáticas con el fin de contribuir a nuevas formalizaciones y en particular a la mecanización de resultados que se derivan de estos teoremas.

2 Sintaxis y Semántica Proposicional

En la lógica objeto proposicional que consideramos, las fórmulas se construyen a partir de las conectivas $\perp$, $\top$, $\neg$, $\wedge$, $\vee$, $\rightarrow$. La sintaxis y semántica de la lógica proposicional y su formalización es la siguiente.

Definición 2.1. *El **alfabeto** de un lenguaje proposicional se compone de los siguientes símbolos.*

1. Símbolos lógicos:

conectivas: $\perp$, $\top$, $\neg$, $\wedge$, $\vee$, $\rightarrow$

símbolos de puntuación: $'(,)'$, $' , '$

2. Símbolos no lógicos:

símbolos proposicionales $P_0, P_1, \dots$

Las fórmulas se definen de la siguiente manera.

Definición 2.2. *El conjunto de **fórmulas** es el conjunto más pequeño que satisface las siguientes condiciones:*

1. Los símbolos $\perp$ y $\top$ son fórmulas.
2. Cualquier símbolo proposicional es una fórmula.

3. Si F es una fórmula, entonces $\neg F$ es una fórmula.
4. Si F y G son fórmulas, entonces $(F \wedge G), (F \vee G), (F \rightarrow G)$ son fórmulas.

El siguiente tipo de dato define en Isabelle las fórmulas de la lógica proposicional.

```
datatype 'b formula =
  FF
| TT
| Atomo 'b
| Negacion 'b formula                ( $\neg$ .(-) [110] 110)
| Conjuncion 'b formula 'b formula  (infixl  $\wedge$ . 109)
| Disyuncion 'b formula 'b formula  (infixl  $\vee$ . 108)
| Implicacion 'b formula 'b formula (infixl  $\rightarrow$ . 100)
```

En lo que sigue presentamos la formalización de los conceptos semánticos de la lógica proposicional. Para formalizar la noción de valor de una fórmula en una interpretación (2.5), introducimos los conceptos de valores de verdad (2.3) e interpretación (2.4).

Definición 2.3. Los **valores de verdad** de la lógica proposicional clásica son V (que se interpreta como **verdadero**) y F (que se interpreta como **falso**).

En la formalización se identifica el conjunto de los valores de verdad con el tipo *v-verdad*, el valor V con la constante *Verdad* y el valor F con la constante *Falso* respectivamente.

```
datatype v-verdad = Verdad | Falso
```

Definición 2.4. Una **interpretación** es una función del conjunto de los símbolos proposicionales en el conjunto de los valores de verdad.

La representación en Isabelle del concepto de interpretación es una función de la forma $I :: 'b \Rightarrow v\text{-verdad}$.

Ahora definimos por recursión el valor de una fórmula respecto de una interpretación.

Definición 2.5. El **valor** de una fórmula proposicional F en una interpretación I (representado por $I'(F)$) se define por recursión como sigue:

$$I'(P) = I(P) \text{ si } P \text{ es un símbolo proposicional}$$

$$I'(\neg F) = \begin{cases} \text{V}, & \text{si } I'(F) = \text{F} \\ \text{F}, & \text{si } I'(F) = \text{V} \end{cases}$$

$$I'(F \wedge G) = \begin{cases} \text{V}, & \text{si } I'(F) = \text{V} \text{ y } I'(G) = \text{V} \\ \text{F}, & \text{en caso contrario.} \end{cases}$$

$$I'(F \vee G) = \begin{cases} \text{F}, & \text{si } I'(F) = \text{F} \text{ y } I'(G) = \text{F} \\ \text{V}, & \text{en caso contrario.} \end{cases}$$

$$I'(F \rightarrow G) = \begin{cases} \text{F}, & \text{si } I'(F) = \text{V} \text{ y } I'(G) = \text{F} \\ \text{V}, & \text{en caso contrario.} \end{cases}$$

Para formalizarlo, definimos las funciones de verdad de las conectivas.

definition *v-negacion* :: *v-verdad* $\Rightarrow$ *v-verdad* **where**
v-negacion *x* $\equiv$ (if *x* = *Verdad* then *Falso* else *Verdad*)

definition *v-conjuncion* :: *v-verdad* $\Rightarrow$ *v-verdad* $\Rightarrow$ *v-verdad* **where**
v-conjuncion *x y* $\equiv$ (if *x* = *Falso* then *Falso* else *y*)

definition *v-disyuncion* :: *v-verdad* $\Rightarrow$ *v-verdad* $\Rightarrow$ *v-verdad* **where**
v-disyuncion *x y* $\equiv$ (if *x* = *Verdad* then *Verdad* else *y*)

definition *v-implicacion* :: *v-verdad* $\Rightarrow$ *v-verdad* $\Rightarrow$ *v-verdad* **where**
v-implicacion *x y* $\equiv$ (if *x* = *Falso* then *Verdad* else *y*)

La formalización del valor de una fórmula es:

primrec *valor* :: (*'b* $\Rightarrow$ *v-verdad*) $\Rightarrow$ *'b formula* $\Rightarrow$ *v-verdad*
where

valor *I* *FF* = *Falso*
| *valor* *I* *TT* = *Verdad*
| *valor* *I* (*Atomo* *P*) = *I* *P*
| *valor* *I* ($\neg$. *F*) = (*v-negacion* (*valor* *I* *F*))
| *valor* *I* (*F* $\wedge$. *G*) = (*v-conjuncion* (*valor* *I* *F*) (*valor* *I* *G*))
| *valor* *I* (*F* $\vee$. *G*) = (*v-disyuncion* (*valor* *I* *F*) (*valor* *I* *G*))
| *valor* *I* (*F* $\rightarrow$. *G*) = (*v-implicacion* (*valor* *I* *F*) (*valor* *I* *G*))

Formalizamos las noción de satisfacibilidad en términos del concepto de *modelo*.

Definición 2.6. Una interpretación *I* es **modelo** de un conjunto de fórmulas *S* si, para toda fórmula *F* de *S*, $I'(F) = \text{V}$.

Su formalización es:

definition

$modelo :: ('b \Rightarrow v-verdad) \Rightarrow 'b \text{ formula set} \Rightarrow bool$ ($- \text{ modelo} - [80,80] \ 80$)

where

$I \text{ modelo } S \equiv (\forall F \in S. \text{ valor } I \ F = \text{ Verdad})$

Definición 2.7. *Un conjunto de fórmulas es satisfacible si tiene algún modelo. En caso contrario se dice que es insatisfacible.*

Su formalización es:

definition $satisfacible :: 'b \text{ formula set} \Rightarrow bool$ **where**

$satisfacible \ S \equiv (\exists v. v \text{ modelo } S)$

3 Teorema de Compacidad

En esta sección presentamos las ideas principales de la manera como hemos formalizado el Teorema de Compacidad. En el anexo 1 se muestra el desarrollo completo de la respectiva teoría.

Para enunciar el Teorema de Compacidad necesitamos los conceptos de conjunto finito y conjunto enumerable.

Definición 3.1. *Sea n un número natural. El segmento inicial determinado por n es el conjunto $I_n = \{m \in \mathbb{N} \mid m < n\}$.*

Un conjunto A es finito si existe una función sobreyectiva $f: I_n \rightarrow A$ de algún segmento inicial I_n en A . En caso contrario se dice que es infinito.

En Isabelle se define de manera inductiva el concepto de conjunto finito [8]:

inductive $finite :: 'a \text{ set} \Rightarrow bool$

where

$emptyI: \quad finite \ \{\}$
 $| \ insertI: \quad finite \ A \Longrightarrow finite \ (insert \ a \ A)$

El siguiente lema afirma que las anteriores definiciones son equivalentes.

lemma $finite \ A \longleftrightarrow (\exists n \ f. \ A = f \ ` \ \{i::nat. \ i < n\})$

Definición 3.2. *Un conjunto A es enumerable si es vacío o existe una función sobreyectiva $f: \mathbb{N} \rightarrow A$ del conjunto de los números naturales en el conjunto A . En este caso, se dice que f es una enumeración de A , y sus elementos se puede listar como una sucesión infinita: $f(0), f(1), f(2), \dots$, en la cual aparecen todos los elementos de A , con posibles repeticiones, y solo elementos de A . En particular, cualquier conjunto finito es enumerable.*

definition *enumeracion* :: (nat $\Rightarrow$ 'b) $\Rightarrow$ bool **where**
enumeracion f = ($\forall$ y. $\exists$ n. y = (f n))

Teorema 3.3 (Teorema de Compacidad (3.6.3 en [1])). *Sean L un lenguaje proposicional enumerable y S un conjunto de fórmulas proposicionales. Si todo subconjunto finito de S es satisfacible, entonces S es satisfacible.*

La demostración del Teorema de Compacidad que se presenta en los textos clásicos de introducción a la lógica utiliza el *Lema de König*. El Teorema de Existencia de Modelos permite otra forma de demostrar este resultado.

Teorema 3.4 (Teorema de Existencia de Modelos (3.6.2 en [1])). *Sea L un lenguaje proposicional enumerable. Si C es una propiedad de consistencia proposicional y $S \in C$, entonces S es satisfacible.*

La formalización en Isabelle es la siguiente.

theorem *TeoremaExistenciaModelos*:
assumes h1: $\exists$ g. *enumeracion* (g :: nat $\Rightarrow$ 'b formula)
and h2: *consistenciaP* C
and h3: (S :: 'b formula set) $\in$ C
shows *satisfacible* S

Para aplicar el teorema anterior en la formalización del Teorema de Compacidad utilizamos el siguiente lema.

Lema 3.5. *La colección de conjuntos de fórmulas proposicionales,*

$$C = \{W \mid \forall A (A \subseteq W \wedge A \text{ es finito} \rightarrow A \text{ es satisfacible})\}$$

es una propiedad de consistencia.

La formalización del lema anterior es la siguiente.

lemma *ConsistenciaCompacidad*:
shows *consistenciaP* {W :: 'b formula set. $\forall A. (A \subseteq W \wedge \text{finite } A) \longrightarrow \text{satisfacible } A$ }

La formalización del Teorema de Compacidad es la siguiente:

theorem *TeoremaCompacidad1*:
assumes $\exists$ g. *enumeracion* (g :: nat $\Rightarrow$ 'b formula)
and $\forall A. (A \subseteq (S :: 'b formula set) \wedge \text{finite } A) \longrightarrow \text{satisfacible } A$
shows *satisfacible* S

4 Coloración de grafos

El procedimiento de coloración de grafos se presenta en aplicaciones de problemas combinatoriales, por ejemplo en temas de programación lineal (asignación de tareas) y optimización.

Un grafo es un conjunto de puntos (vértices) y segmentos (aristas) que los conectan. La coloración de un grafo consiste en asignar colores a los vértices del grafo de tal forma que dos vértices que compartan la misma arista (son adyacentes) tengan colores diferentes. Una coloración que usa a lo más k colores se llama una k -coloración del grafo.

Un grafo *planar*, es un grafo que se puede dibujar en el plano sin que se intersecten sus aristas (bordes). El Teorema de los Cuatro Colores afirma que todo grafo finito *planar* es 4-colorable. Este teorema fue demostrado por Kenneth Appel y Wolfgang Haken en 1976. La demostración por reducción al absurdo, se basó en la clasificación de todos los posibles grafos planares 4-colorables, 1936 en total; y con la ayuda de un programa de ordenador verificaron que efectivamente cada uno de estos grafos tenían la propiedad 4-colorable.

En el año 2015, Georges Gonthier utilizando el sistema Coq, realizó una prueba automatizada del Teorema de los Cuatro Colores [2], [3]. En esta sección formalizamos el teorema de k -coloramiento para grafos enumerables como una consecuencia del Teorema de Compacidad.

En lo que sigue cada concepto o propiedad referente a grafos estará asociado respectivamente a la correspondiente definición o lema en Isabelle.

Definición 4.1. *Un grafo es un pareja ordenada $G = (V, E)$ donde V es un conjunto (de vértices) y $E \subseteq V \times V$ es una relación binaria en V irreflexiva. Si $(u, v) \in E$ o $(v, u) \in E$ decimos que u y v son adyacentes.*

type-synonym $'v \text{ digrafo} = ('v \text{ set}) \times (('v \times 'v) \text{ set})$

abbreviation $\text{vert} :: 'v \text{ digrafo} \Rightarrow 'v \text{ set} \quad (V[-] [80] 80) \text{ where}$
 $V[G] \equiv \text{fst } G$

abbreviation $\text{arc} :: 'v \text{ digrafo} \Rightarrow ('v \times 'v) \text{ set} \quad (E[-] [80] 80) \text{ where}$
 $E[G] \equiv \text{snd } G$

definition $\text{es-grafo} :: 'v \text{ digrafo} \Rightarrow \text{bool} \text{ where}$
 $\text{es-grafo } G \equiv \forall u \ v. (u, v) \in E[G] \longrightarrow u \in V[G] \wedge v \in V[G] \wedge u \neq v$

Definición 4.2. Sea $G = (V, E)$ un grafo. Dado un subconjunto $S \subseteq V$, el grafo $G_S = (S, E \cap (S \times S))$ se llama el subgrafo de $G = (V, E)$ inducido por S .

definition *es-subgrafo-inducido* :: 'v digrafo $\Rightarrow$ 'v digrafo $\Rightarrow$ bool **where**
es-subgrafo-inducido $H \ G \equiv$
 $(V[H] \subseteq V[G]) \wedge E[H] = E[G] \cap ((V[H]) \times (V[H]))$

lemma

assumes *es-grafo* G **and** *es-subgrafo-inducido* $H \ G$
shows *es-grafo* H

Definición 4.3. Un grafo $G = (V, E)$ es enumerable si el conjunto V es enumerable.

definition *grafo-enumerable* :: 'v digrafo $\Rightarrow$ bool **where**
grafo-enumerable $G \equiv \exists g. \text{enumeracion } (g :: \text{nat} \Rightarrow 'v)$

Definición 4.4. Sea k un entero positivo. Un k -coloramiento de un grafo $G = (V, E)$ es una función $c : V \rightarrow [k] = \{1, \dots, k\}$ tal que $c(u) \neq c(v)$ para $(u, v) \in E$ (vértices adyacentes tienen diferentes “colores”).

Un grafo G es k -colorable si existe un k -coloramiento de G .

definition *coloramiento* :: ('v $\Rightarrow$ nat) $\Rightarrow$ nat $\Rightarrow$ 'v digrafo $\Rightarrow$ bool **where**
coloramiento $c \ k \ G \equiv$
 $(\forall u. u \in V[G] \longrightarrow c(u) \leq k) \wedge (\forall u \ v. (u, v) \in E[G] \longrightarrow c(u) \neq c(v))$

definition *colorable* :: 'v digrafo $\Rightarrow$ nat $\Rightarrow$ bool **where**
colorable $G \ k \equiv \exists c. \text{coloramiento } c \ k \ G$

Teorema 4.5 (Erdős). Sean $G = (V, E)$ un grafo enumerable y k un entero positivo. Si para todo subconjunto finito $S \subseteq V$ se tiene que G_S es k -colorable, entonces G es k -colorable.

Demostración: Fijamos un conjunto de símbolos proposicionales,

$$\mathcal{P} = \{C_{u,i} \mid u \in V, 1 \leq i \leq k\}$$

donde $C_{u,i}$ es interpretado como “el vértice u tiene el color i ”. Definimos tres conjuntos de proposiciones:

1.

$$\mathcal{F} = \{C_{u,1} \vee C_{u,2} \vee \dots \vee C_{u,k} \mid u \in V\},$$

2.

$$\mathcal{G} = \{\neg(C_{u,i} \wedge C_{u,j}) \mid u \in V, 1 \leq i, j \leq k, i \neq j\},$$

3.

$$\mathcal{H} = \{\neg(C_{u,i} \wedge C_{v,i}) \mid u, v \in V, (u, v) \in E, 1 \leq i \leq k\}.$$

Los anteriores conjuntos describen las siguientes propiedades respecto a los vértices de G y a “los k colores dados”: a cada vértice le corresponde por lo menos un color, ningún vértice tiene asociado más de un color y a vértices adyacentes les corresponden colores distintos.

Sea $\mathcal{T} = \mathcal{F} \cup \mathcal{G} \cup \mathcal{H}$. Aplicamos el Teorema de Compacidad para demostrar que $\mathcal{T}$ es satisfacible.

Sea S un subconjunto finito de $\mathcal{T}$ y $V_0 = \{u_1, \dots, u_n\}$ el conjunto de todos los vértices u tal que $C_{u,i}$ para algún i , ocurre en alguna fórmula de S . Sea $G_{V_0} = (V_0, E_0)$ el subgrafo de G inducido por V_0 . Sea $c : V_0 \rightarrow [k]$ un k -coloramiento de G_{V_0} (el cual existe por hipótesis). Definimos la interpretación $v : \mathcal{P} \rightarrow \{\mathbf{V}, \mathbf{F}\}$ por,

$$v(C_{u,i}) = \begin{cases} \mathbf{V} & \text{si } u \in V_0 \text{ y } c(u) = i, \\ \mathbf{F} & \text{en otro caso.} \end{cases}$$

Se tiene que $v(F) = \mathbf{V}$ para toda $F \in S$, por ser c un k -coloramiento y $F \in \mathcal{F} \cup \mathcal{G} \cup \mathcal{H}$. De esta forma, $\mathcal{T}$ es finitamente satisfacible y por lo tanto, por el Teorema de Compacidad, es satisfacible.

Sea $I : \mathcal{P} \rightarrow \{\mathbf{V}, \mathbf{F}\}$ una interpretación que satisface a $\mathcal{T}$. Establecemos una correspondencia $c : V \rightarrow [k]$ definida por, $c(u) = i$ si y solamente si $I(C_{u,i}) = \mathbf{V}$. De esta forma, por la definición de $\mathcal{T}$ y puesto que $I(F) = \mathbf{V}$ para toda $F \in \mathcal{T}$ se tiene que c es un k -coloramiento de $G = (V, E)$: puesto que $\mathcal{F}$ y $\mathcal{G}$ son satisfacibles, a cada vértice $v \in V$ le corresponde exactamente un color en $[k]$ (es decir, c es una función). Además, como $\mathcal{H}$ es satisfacible, los vértices que son adyacentes no tienen el mismo color. □

Las siguientes definiciones en Isabelle corresponden a la formalización de los conjuntos $\mathcal{F}, \mathcal{G}, \mathcal{H}$ y $\mathcal{T}$ respectivamente.

primrec *disyuncion-atomicas* :: ' $v \Rightarrow \text{nat} \Rightarrow ('v \times \text{nat})\text{formula}$ ' **where**
disyuncion-atomicas v 0 = *Atomo* (v , 0)
| *disyuncion-atomicas* v (*Suc* k) =
(*Atomo* (v , *Suc* k)) $\vee$. (*disyuncion-atomicas* v k)

definition $\mathcal{F}$:: ' $v \text{ digrafo} \Rightarrow \text{nat} \Rightarrow (('v \times \text{nat})\text{formula}) \text{ set}$ ' **where**
 $\mathcal{F}$ G $k \equiv (\bigcup_{v \in V[G]}. \{ \text{disyuncion-atomicas } v \ k \})$

definition $\mathcal{G} :: 'v \text{ digrafo} \Rightarrow \text{nat} \Rightarrow ('v \times \text{nat})\text{formula set}$ **where**

$$\mathcal{G} \ G \ k \equiv \{ \neg.(\text{Atomo } (v, i) \wedge. \text{Atomo}(v, j)) \\ | \ v \ i \ j. (v \in V[G]) \wedge (0 \leq i \wedge 0 \leq j \wedge i \leq k \wedge j \leq k \wedge i \neq j) \}$$

definition $\mathcal{H} :: 'v \text{ digrafo} \Rightarrow \text{nat} \Rightarrow ('v \times \text{nat})\text{formula set}$ **where**

$$\mathcal{H} \ G \ k \equiv \{ \neg.(\text{Atomo } (u, i) \wedge. \text{Atomo}(v, i)) \\ | \ u \ v \ i. (u \in V[G] \wedge v \in V[G] \wedge (u, v) \in E[G]) \wedge (0 \leq i \wedge i \leq k) \}$$

definition $\mathcal{T} :: 'v \text{ digrafo} \Rightarrow \text{nat} \Rightarrow ('v \times \text{nat})\text{formula set}$ **where**

$$\mathcal{T} \ G \ k \equiv (\mathcal{F} \ G \ k) \cup (\mathcal{G} \ G \ k) \cup (\mathcal{H} \ G \ k)$$

Definimos en Isabelle el conjunto de vértices que ocurren en un conjunto de fórmulas de la siguiente manera.

primrec $\text{vertices-formula} :: ('v \times \text{nat})\text{formula} \Rightarrow 'v \text{ set}$ **where**

$$\begin{aligned} & \text{vertices-formula } FF = \{\} \\ & | \text{vertices-formula } TT = \{\} \\ & | \text{vertices-formula } (\text{Atomo } P) = \{\text{fst } P\} \\ & | \text{vertices-formula } (\neg. F) = \text{vertices-formula } F \\ & | \text{vertices-formula } (F \wedge. G) = \text{vertices-formula } F \cup \text{vertices-formula } G \\ & | \text{vertices-formula } (F \vee. G) = \text{vertices-formula } F \cup \text{vertices-formula } G \\ & | \text{vertices-formula } (F \rightarrow. G) = \text{vertices-formula } F \cup \text{vertices-formula } G \end{aligned}$$

definition $\text{vertices-conjunto-formulas} :: ('v \times \text{nat})\text{formula set} \Rightarrow 'v \text{ set}$ **where**

$$\text{vertices-conjunto-formulas } S = (\bigcup F \in S. \text{vertices-formula } F)$$

El conjunto de vértices de cualquier subconjunto S de fórmulas de $\mathcal{T}$ está contenido en el conjunto de vértices de G .

lemma $\text{vertices-subconjunto-formulas}$:

$$\text{assumes } S \subseteq (\mathcal{T} \ G \ k)$$

$$\text{shows } \text{vertices-conjunto-formulas } S \subseteq V[G]$$

Sea S un subconjunto finito de $\mathcal{T}$ y $V_0 = \{u_1, \dots, u_n\}$ el conjunto de todos los vértices u tal que, $C_{u,i}$ para algún i ocurre en alguna fórmula de S . El subgrafo de G inducido por V_0 , $G_{V_0} = (V_0, E_0)$, es un grafo finito.

definition $\text{subgrafo-aux} :: 'v \text{ digrafo} \Rightarrow 'v \text{ set} \Rightarrow 'v \text{ digrafo}$ **where**

$$\text{subgrafo-aux } G \ V \equiv (V, E[G] \cap (V \times V))$$

lemma subgrafo-inducido :

$$\text{assumes } \text{es-grafo } G \text{ and } S \subseteq (\mathcal{T} \ G \ k)$$

$$\text{shows } \text{es-subgrafo-inducido } (\text{subgrafo-aux } G \ (\text{vertices-conjunto-formulas } S)) \ G$$

lemma *finito-subgrafo*:

assumes *es-grafo* G **and** $S \subseteq (\mathcal{T} \ G \ k)$ **and** *finite* S
shows *grafo-finito* (*subgrafo-aux* G (*vertices-conjunto-formulas* S))

Un coloramiento de G_{V_0} permite construir un modelo de S .

fun *interpretacion-grafo* :: $'v \text{ digrafo} \Rightarrow ('v \Rightarrow \text{nat}) \Rightarrow (('v \times \text{nat}) \Rightarrow v\text{-verdad})$
where
interpretacion-grafo $G \ f = (\lambda(v,i).(if \ v \in V[G] \wedge f(v) = i \ \text{then} \ \text{Verdad} \ \text{else} \ \text{Falso}))$

theorem *coloramiento-satisfactible*:

assumes *es-grafo* G **and** $S \subseteq (\mathcal{T} \ G \ k)$ **and**
coloramiento $f \ k$ (*subgrafo-aux* G (*vertices-conjunto-formulas* S))
shows *satisfacible* S

proof–

let $?V = \text{vertices-conjunto-formulas } S$
let $?H = \text{subgrafo-aux } G \ ?V$
have (*interpretacion-grafo* $?H \ f$) *modelo* S
thus *satisfacible* S

Una interpretación $I : \mathcal{P} \rightarrow \{\mathbf{V}, \mathbf{F}\}$ que satisface a $\mathcal{T}$ establece un coloramiento $c : V \rightarrow [k]$ definido por, $c(u) = i$ si y solamente si $I(C_{u,i}) = \mathbf{V}$.

fun *coloramiento-grafo* :: $(('v \times \text{nat}) \Rightarrow v\text{-verdad}) \Rightarrow \text{nat} \Rightarrow ('v \Rightarrow \text{nat})$
where
coloramiento-grafo $I \ k = (\lambda v.(THE \ i. (\text{valor } I \ (\text{Atomo } (v,i)) = \text{Verdad}) \wedge 0 \leq i \wedge i \leq k))$

lemma *funcion-coloramiento*:

assumes $u \in V[G]$ **and** I *modelo* $(\mathcal{T} \ G \ k)$
shows $\exists! i. (\text{valor } I \ (\text{Atomo } (u,i)) = \text{Verdad} \wedge 0 \leq i \wedge i \leq k) \wedge \text{coloramiento-grafo } I \ k \ u = i$

lemma *colores-distintos*:

assumes *es-grafo* G **and** $(u,v) \in E[G]$ **and** I : I *modelo* $(\mathcal{T} \ G \ k)$
shows *coloramiento-grafo* $I \ k \ u \neq \text{coloramiento-grafo } I \ k \ v$

theorem *satisfactible-coloramiento*:

assumes *es-grafo* G **and** *satisfacible* $(\mathcal{T} \ G \ k)$
shows *colorable* $G \ k$

Por último, la formalización del teorema 4.5 es la siguiente.

theorem *coloracion*:

assumes *es-grafo* ($G :: ('vertices\ set) \times (('vertices \times 'vertices)\ set)$)
and $\exists g. \textit{enumeracion}\ (g :: nat \Rightarrow 'vertices)$
and $\forall H. (\textit{es-subgrafo-inducido}\ H\ G \wedge \textit{grafo-finito}\ H \longrightarrow \textit{colorable}\ H\ k)$
shows *colorable* $G\ k$

proof–

have $\forall S. S \subseteq (\mathcal{T}\ G\ k) \wedge (\textit{finite}\ S) \longrightarrow \textit{satisfacible}\ S$

proof(*rule allI, rule impI*)

fix S **assume** $S \subseteq (\mathcal{T}\ G\ k) \wedge (\textit{finite}\ S)$

hence *hip1*: $S \subseteq (\mathcal{T}\ G\ k)$ **and** *hip2*: *finite* S **by** *auto*

show *satisfacible* S

proof –

let $?V = \textit{vertices-conjunto-formulas}\ S$

let $?H = (?V, E[G] \cap (?V \times ?V))$

have *es-subgrafo-inducido* $?H\ G$

using *assms*(1) *hip1* *subgrafo-inducido*[*of* $G\ S\ k$]

by(*unfold* *subgrafo-aux-def*, *auto*)

moreover

have *grafo-finito* $?H$

using *assms*(1) *hip1* *hip2* *finito-subgrafo*[*of* $G\ S\ k$]

by(*unfold* *subgrafo-aux-def*, *auto*)

ultimately

have *colorable* $?H\ k$ **using** *assms*(3) **by** *auto*

hence $\exists f. \textit{coloramiento}\ f\ k\ ?H$ **by**(*unfold* *colorable-def*, *auto*)

then obtain f **where** *coloramiento* $f\ k\ ?H$ **by** *auto*

thus *satisfacible* S **using** *coloramiento-satisfactible*[*OF* *assms*(1) *hip1*]

by(*unfold* *subgrafo-aux-def*, *auto*)

qed

qed

hence *satisfacible* $(\mathcal{T}\ G\ k)$ **using**

TeoremaCompacidad1 [*OF* *EnumFormulasVertices* [*OF*

$\langle \exists g. \textit{enumeracion}\ (g :: nat \Rightarrow 'v) \rangle$], *of* $(\mathcal{T}\ G\ k)$]

by *auto*

thus *?thesis* **using** *assms*(1) *satisfactible-coloramiento* **by** *blast*

qed

5 Lema de König

El lema de König es un teorema en teoría de grafos equivalente al Teorema de Compacidad, que permite estudiar propiedades combinatoriales de problemas que se modelan a través de la estructura de árbol. El lema de König afirma que todo árbol infinito de ramificación finita tiene una rama infinita. En esta sección, como una consecuencia del Teorema de Compacidad de la lógica proposicional, verificamos mecánicamente este teorema, para árboles enumerables.

En lo que sigue cada concepto o propiedad referente a árboles estará asociado respectivamente a la correspondiente definición o lema en Isabelle.

Definición 5.1. Sea R una relación binaria en un conjunto A , es decir, $R \subseteq A \times A$.

1. R es irreflexiva si,

$$\forall x \in A, (x, x) \notin R.$$

2. R es transitiva si,

$$\forall x \in A \forall y \in A \forall z \in A ((x, y) \in R \wedge (y, z) \in R \longrightarrow (x, z) \in R).$$

3. R es total si,

$$\forall x \in A \forall y \in A (x \neq y \longrightarrow (x, y) \in R \vee (y, x) \in R).$$

4. El elemento $a \in A$ es un elemento mínimo de A si,

$$\forall x \in A (x \neq a \longrightarrow (a, x) \in R).$$

5. Sea $a \in A$. El conjunto de predecesores de a está definido por,

$$Pr(a) = \{x \in A \mid (x, a) \in R\}.$$

type-synonym $'a \text{ rel} = ('a \times 'a) \text{ set}$

definition $irrefl\text{-}en :: 'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow bool$

where $irrefl\text{-}en \ A \ r \equiv (\forall x \in A. (x, x) \notin r)$

definition $trans\text{-}en :: 'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow bool$

where $trans\text{-}en \ A \ r \equiv$

$(\forall x \in A. \forall y \in A. \forall z \in A. (x, y) \in r \wedge (y, z) \in r \longrightarrow (x, z) \in r)$

definition *total-en* :: 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool

where *total-en* A r $\equiv (\forall x \in A. \forall y \in A. x \neq y \longrightarrow (x, y) \in r \vee (y, x) \in r)$

definition *minimo* :: 'a set $\Rightarrow$ 'a $\Rightarrow$ 'a rel $\Rightarrow$ bool

where *minimo* A a r $\equiv (a \in A \wedge (\forall x \in A. x \neq a \longrightarrow (a, x) \in r))$

definition *predecesores-de* :: 'a set $\Rightarrow$ 'a $\Rightarrow$ 'a rel $\Rightarrow$ 'a set

where *predecesores-de* A a r $\equiv \{x \in A. (x, a) \in r\}$

Definición 5.2. Sea R una relación binaria en A tal que para todo $a \in A$ el conjunto $Pr(a)$ es finito.

1. Para cada $a \in A$, se define la altura de a igual al número de sus predecesores,

$$Alt(a) = |Pr(a)|.$$

2. Para cada entero $n \geq 0$, el n -ésimo nivel de R es el conjunto de elementos de A de altura n ,

$$Niv(n) = \{a \in A \mid Alt(a) = n\}.$$

3. Para cada $a \in A$, el conjunto de sucesores inmediatos de a está definido por,

$$Suc(a) = \{y \in A \mid (a, y) \in R \wedge Alt(y) = Alt(a) + 1\}.$$

definition *altura* :: 'a set $\Rightarrow$ 'a $\Rightarrow$ 'a rel $\Rightarrow$ nat

where *altura* A a r $\equiv \text{card } (\text{predecesores-de } A \ a \ r)$

definition *nivel* :: 'a set $\Rightarrow$ 'a rel $\Rightarrow$ nat $\Rightarrow$ 'a set

where *nivel* A r n $\equiv \{x \in A. \text{altura } A \ x \ r = n\}$

definition *sucesores-inmed-de* :: 'a set $\Rightarrow$ 'a $\Rightarrow$ 'a rel $\Rightarrow$ 'a set

where *sucesores-inmed-de* A a r $\equiv \{x \in A. (a, x) \in r \wedge \text{altura } A \ x \ r = (\text{altura } A \ a \ r) + 1\}$

Definición 5.3. Sea R una relación binaria en A .

1. (A, R) es un orden parcial estricto si R es irreflexiva y transitiva.
2. (A, R) es un orden lineal si es un orden parcial estricto y R es total.

definition *orden-parcial-estricto-en* :: 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *orden-parcial-estricto-en* A r $\equiv$ *irrefl-en* A r $\wedge$ *trans-en* A r

definition *orden-lineal-en* :: 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *orden-lineal-en* A r $\equiv$ *orden-parcial-estricto-en* A r $\wedge$ *total-en* A r

Lema 5.4. Si un orden parcial estricto tiene un elemento mínimo es único.

lemma *unico-minimo*:
assumes *orden-parcial-estricto-en* A r **and** *minimo* A a r **and** *minimo* A b r
shows a=b

Definición 5.5. Sea $R \neq \emptyset$ una relación binaria en un conjunto A. La pareja $T = (A, R)$ es un árbol si:

1. R tiene un elemento mínimo, la raíz de T.
2. (A, R) es un orden parcial estricto.
3. Para todo $a \in A$, el conjunto $Pr(a)$ es finito y la restricción de R a $Pr(a)$ es total.

Los elementos de A son llamados nodos.

definition *arbol* :: 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *arbol* A r $\equiv$
 $r \subseteq A \times A \wedge r \neq \{\}$ $\wedge$ (*orden-parcial-estricto-en* A r) $\wedge$ ($\exists a. \text{minimo } A \ a \ r$) $\wedge$
 $(\forall a \in A. \text{finite } (\text{predecesores-de } A \ a \ r) \wedge (\text{total-en } (\text{predecesores-de } A \ a \ r) \ r))$

Definición 5.6. Un árbol (A, R) es finito si el conjunto A de nodos es finito, en caso contrario es infinito.

definition *arbol-finito*:: 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where
arbol-finito A r $\equiv$ *arbol* A r $\wedge$ *finite* A

abbreviation *arbol-infinito*:: 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where
arbol-infinito A r $\equiv$ *arbol* A r $\wedge$ $\neg$ *finite* A

Definición 5.7. Un árbol $A = (A, R)$ es enumerable si el conjunto A es enumerable.

definition *arbol-enumerable* :: 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool **where**
arbol-enumerable A r $\equiv \exists g. \text{enumeracion } (g:: \text{nat} \Rightarrow 'a)$

Definición 5.8. Sea $T = (A, R)$ un árbol. T es de ramificación finita si para cada $a \in A$, el conjunto $\text{Suc}(a)$ es finito.

definition *ramificacion-finita* :: 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *ramificacion-finita* A r $\equiv (\forall x \in A. \text{finite } (\text{sucesores-inmed-de } A \ x \ r))$

Definición 5.9. Sea $T = (A, R)$ un árbol. Un conjunto de nodos $B \subseteq A$ es una rama de T si (B, R) es un orden lineal y es maximal (con respecto a la relación de contenencia). Si el conjunto B es finito, la rama es finita, de lo contrario es infinita.

definition *sub-orden-lineal-en* :: 'a set $\Rightarrow$ 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *sub-orden-lineal-en* B A r $\equiv B \subseteq A \wedge (\text{orden-parcial-estricto-en } A \ r) \wedge (\text{total-en } B \ r)$

definition *rama* :: 'a set $\Rightarrow$ 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *rama* B A r $\equiv$
 $(\text{sub-orden-lineal-en } B \ A \ r) \wedge$
 $(\forall C. B \subseteq C \wedge \text{sub-orden-lineal-en } C \ A \ r \longrightarrow B = C)$

definition *rama-finita* :: 'a set $\Rightarrow$ 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *rama-finita* B A r $\equiv \text{rama } B \ A \ r \wedge \text{finite } B$

definition *rama-infinita* :: 'a set $\Rightarrow$ 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *rama-infinita* B A r $\equiv \text{rama } B \ A \ r \wedge \neg \text{finite } B$

Los siguientes cuatro lemas son los resultados principales en los que se basa la demostración del Lema de König.

Lema 5.10. Sea $T = (A, R)$ un árbol. Las siguientes afirmaciones son equivalentes:

1. T es de ramificación finita.
2. Para todo $n \geq 0$, el conjunto $\text{Niv}(n)$ es finito.

lemma *finito-nivel*:

assumes *arbol* A r **and** *ramificacion-finita* A r
shows *finite* (nivel A r n)

lemma *finito-nivel-a*:

assumes *arbol* A r **and** $\forall n. \text{finite } (\text{nivel } A \ r \ n)$
shows *ramificacion-finita* A r

Lema 5.11. Sea $T = (A, R)$ un árbol. Si $n \geq 0$ y $x \in \text{Niv}(n+1)$ entonces, para todo k , $0 \leq k \leq n$ existe y tal que, $(y, x) \in R$ y $y \in \text{Niv}(k)$.

lemma camino1:

assumes arbol A r **and** $x \in (\text{nivel } A \text{ } r \text{ } (n+1))$

shows $\forall k. (0 \leq k \wedge k \leq n) \longrightarrow (\exists y. (y, x) \in r \wedge y \in (\text{nivel } A \text{ } r \text{ } k))$

Lema 5.12. Sea $T = (A, R)$ un árbol infinito de ramificación finita. Entonces, Para todo $n \geq 0$, $\text{Niv}(n) \neq \emptyset$.

lemma niveles-no-vacios:

assumes arbol-infinito A r **and** ramificacion-finita A r

shows $\forall n. \text{nivel } A \text{ } r \text{ } n \neq \{\}$

Lema 5.13. Sea $T = (A, R)$ un árbol. Supongamos que $(x, z) \in R$, $(y, z) \in R$ y $x \neq y$. Si $x \in \text{Niv}(n)$ y $y \in \text{Niv}(m)$ entonces, $\text{Niv}(n) \cap \text{Niv}(m) = \emptyset$.

lemma niveles-diferentes-arbol1:

assumes arbol A r **and** $(x, z) \in r$ **and** $(y, z) \in r$

and $x \neq y$ **and** $x \in (\text{nivel } A \text{ } r \text{ } n)$ **and** $y \in (\text{nivel } A \text{ } r \text{ } m)$

shows $\text{nivel } A \text{ } r \text{ } n \cap \text{nivel } A \text{ } r \text{ } m = \{\}$

Teorema 5.14 (König). Todo árbol infinito enumerable de ramificación finita tiene una rama infinita.

Demostración: Sea $T = (A, R)$ un árbol infinito enumerable de ramificación finita. Consideremos el siguiente conjunto de símbolos proposicionales indizados por los nodos de T :

$$\mathcal{P} = \{B_u \mid u \in A\}.$$

A partir de $\mathcal{P}$ se define un conjunto $\mathcal{T}$ de proposiciones de tal forma que si $\mathcal{T}$ es satisfacible entonces, para cualquier interpretación v que sea modelo de $\mathcal{T}$, el conjunto de nodos,

$$\mathcal{B} = \{u \in A \mid v(B_u) = \mathbf{V}\}$$

es una rama infinita de T .

$\mathcal{T}$ es la unión de los siguientes tres conjuntos de fórmulas proposicionales.

1.

$$\mathcal{F} = \left\{ \bigvee_{u \in \text{Niv}(n)} B_u \mid n \in \mathbb{N} \cup \{0\} \right\},$$

donde $\bigvee_{u \in \text{Niv}(n)} B_u$ es la disyunción de las fórmulas atómicas correspondientes a los elementos del nivel $\text{Niv}(n)$, el cual por el Lema 5.10 es finito.

2.

$$\mathcal{G} = \{B_u \longrightarrow B_v \mid u, v \in A, (v, u) \in R\},$$

3.

$$\mathcal{H} = \{\neg(B_u \wedge B_v) \mid u, v \in Niv(n), u \neq v, n \in \mathbb{N} \cup \{0\}\}.$$

Las definiciones de los anteriores conjuntos permiten caracterizar la noción de rama infinita o camino infinito de un árbol. Si un conjunto B de nodos de T “satisface” los anteriores conjuntos entonces: para todo n , hay por lo menos un nodo en T de altura n que pertenece a B , todo predecesor de cualquier elemento de B pertenece a B y B contiene a lo más un nodo de altura n .

Sea $\mathcal{T} = \mathcal{F} \cup \mathcal{G} \cup \mathcal{H}$; aplicamos el Teorema de Compacidad para demostrar que $\mathcal{T}$ es satisfacible.

Sea S un subconjunto finito de $\mathcal{T}$. Puesto que S es finito se tiene que el conjunto, $N = \{u \in A \mid B_u \text{ ocurre en alguna fórmula de } S\}$ es finito y por lo tanto existe $h \geq 0$, el máximo del conjunto de alturas de los nodos de N . También, se tiene que $Niv(h+1) \neq \emptyset$ puesto que T es infinito de ramificación finita (Lema 5.12).

Sea $t \in Niv(h+1)$. Definimos la interpretación $v : \mathcal{P} \rightarrow \{\mathbf{V}, \mathbf{F}\}$ por,

$$v(B_u) = \begin{cases} \mathbf{V} & \text{si } (u, t) \in R \\ \mathbf{F} & \text{en otro caso.} \end{cases}$$

Tenemos que $v(F) = \mathbf{V}$ para toda fórmula $F \in S$:

1. Si $F \in \mathcal{F}$ entonces, $F = \bigvee_{u \in Niv(n)} B_u$, la disyunción de las fórmulas atómicas correspondientes a los nodos del nivel n , para algún $n \leq h$. Puesto que los nodos que ocurren en F tienen altura $n < h+1$, existe $u \in Niv(n)$ tal que $(u, t) \in R$ (Lema 5.11). Por lo tanto, $v(B_u) = \mathbf{V}$. Así, $v(F) = \mathbf{V}$.
2. Si $F \in \mathcal{G}$ entonces, existen $u, w \in A$ tal que $F = B_u \longrightarrow B_w$ y $(w, u) \in R$. Por lo tanto, $v(F) = \mathbf{V}$ puesto que, si $v(F) = \mathbf{F}$ entonces $v(B_u) = \mathbf{V}$ y $v(B_w) = \mathbf{F}$ y por lo tanto, $(u, t) \in R$ y $(w, t) \notin R$ lo cual es imposible puesto que $(w, u) \in R$ y R es transitiva.
3. Si $F \in \mathcal{H}$ entonces, existen $u, w \in Niv(n)$ para algún $n \geq 0$ tal que $u \neq w$ y $F = \neg(B_u \wedge B_w)$. Puesto que u y w pertenecen al mismo nivel, se tiene que $(u, t) \notin R$ o $(w, t) \notin R$ (Lema 5.13). Luego, $v(B_u) = \mathbf{F}$ o $v(B_w) = \mathbf{F}$. Por lo tanto, $v(F) = \mathbf{V}$.

De esta forma, $\mathcal{T}$ es finitamente satisfacible y por lo tanto, por el Teorema de Compacidad, es satisfacible.

Sea $v : \mathcal{P} \rightarrow \{\mathbf{V}, \mathbf{F}\}$ un modelo de $\mathcal{T}$. Entonces,

$$\mathcal{B} = \{u \in A \mid v(B_u) = \mathbf{V}\}$$

es una rama infinita de T :

Puesto que v satisface a $\mathcal{F}$ y a $\mathcal{H}$, se tiene que $\mathcal{B}$ intersecta a cada nivel en un único nodo. Demostremos que $(\mathcal{B}, R)$ es total y maximal, y $\mathcal{B}$ infinito.

- (a) $(\mathcal{B}, R)$ es total: sean $u, w \in \mathcal{B}$ tal que $u \neq w$. Supongamos que $Alt(u) < Alt(w)$. Sean $n = Alt(u)$ y x el predecesor de w en el nivel n , entonces $B_w \longrightarrow B_x \in \mathcal{G}$ y por lo tanto $v(B_w \longrightarrow B_x) = \mathbf{V}$. Como $v(B_w) = \mathbf{V}$ entonces $v(B_x) = \mathbf{V}$. Por lo tanto, $x \in \mathcal{B}$ y como $u, x \in Niv(n)$ se concluye que $u = x$. Por lo tanto, $(u, w) \in R$. El caso $Alt(w) < Alt(u)$ es análogo al anterior. Luego $\mathcal{B}$ es total.
- (b) $(\mathcal{B}, R)$ es maximal: si $\mathcal{B} \subseteq C$ y (C, R) es total, demostremos que $C \subseteq \mathcal{B}$. Sean $x \in C$, $n = Alt(x)$ y u el nodo de intersección de $\mathcal{B}$ y el nivel $Niv(n)$. Entonces $u \in \mathcal{B}$ y por lo tanto $u \in C$. En el caso en que $u \neq x$, al ser C total, se tendría que $(u, x) \in R$ o $(x, u) \in R$, lo cual no es posible puesto que en un orden estricto elementos comparables, con un número finito de predecesores, pertenecen a niveles distintos. Luego $x = u$ y por lo tanto $x \in \mathcal{B}$ y así $C \subseteq \mathcal{B}$.
- (c) $\mathcal{B}$ es infinito: puesto que v satisface a $\mathcal{F}$ entonces, dado $n \geq 0$, si $Niv(n) \neq \emptyset$ existe u tal que $u \in \mathcal{B} \cap Niv(n)$. Por lo tanto, como T es infinito y de ramificación infinita se tiene que, para todo $n \geq 0$, existe u tal que $u \in \mathcal{B} \cap Niv(n)$. Luego, para todo $n \geq 0$, el conjunto $\mathcal{B} \cap Niv(n) \neq \emptyset$ y puesto que $Niv(n) \cap Niv(m) = \emptyset$ para $n \neq m$ entonces, $\bigcup_{n \in \mathbb{N} \cup \{0\}} \mathcal{B} \cap Niv(n)$ es infinito. Luego, $\mathcal{B}$ es infinito.

□

Las siguientes definiciones en Isabelle corresponden a la formalización de los conjuntos $\mathcal{F}, \mathcal{G}, \mathcal{H}$ y $\mathcal{T}$ respectivamente.

primrec *disyuncion-nodos* :: 'a list $\Rightarrow$ 'a formula **where**

disyuncion-nodos [] = FF

| *disyuncion-nodos* (v#D) = (Atomo v) $\vee$. (*disyuncion-nodos* D)

definition $\mathcal{F}'$:: 'a set $\Rightarrow$ 'a rel $\Rightarrow$ ('a formula) set **where**

$\mathcal{F}' A r \equiv (\bigcup n. \{disyuncion-nodos(conj-a-lista (nivel A r n))\})$

definition $\mathcal{G}' :: 'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow ('a \text{ formula}) \text{ set}$ **where**

$$\mathcal{G}' A r \equiv \{(Atomo\ u) \rightarrow. (Atomo\ v) \mid u\ v. u \in A \wedge v \in A \wedge (v, u) \in r\}$$

definition $\mathcal{H}n :: 'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow nat \Rightarrow ('a \text{ formula}) \text{ set}$ **where**

$$\mathcal{H}n A r n \equiv \{\neg.((Atomo\ u) \wedge. (Atomo\ v)) \mid u\ v. u \in (nivel\ A\ r\ n) \wedge v \in (nivel\ A\ r\ n) \wedge u \neq v\}$$

definition $\mathcal{H}' :: 'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow ('a \text{ formula}) \text{ set}$ **where**

$$\mathcal{H}' A r \equiv \bigcup n. \mathcal{H}n A r n$$

definition $\mathcal{T}' :: 'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow ('a \text{ formula}) \text{ set}$ **where**

$$\mathcal{T}' A r \equiv (\mathcal{F}' A r) \cup (\mathcal{G}' A r) \cup (\mathcal{H}' A r)$$

Definimos en Isabelle el máximo del conjunto de alturas de los nodos que ocurren en un conjunto (finito) de fórmulas.

primrec *nodos-formula* :: '*v* formula $\Rightarrow$ '*v* set **where**

$$\begin{aligned} & \text{nodos-formula } FF = \{\} \\ & | \text{nodos-formula } TT = \{\} \\ & | \text{nodos-formula } (Atomo\ P) = \{P\} \\ & | \text{nodos-formula } (\neg. F) = \text{nodos-formula } F \\ & | \text{nodos-formula } (F \wedge. G) = \text{nodos-formula } F \cup \text{nodos-formula } G \\ & | \text{nodos-formula } (F \vee. G) = \text{nodos-formula } F \cup \text{nodos-formula } G \\ & | \text{nodos-formula } (F \rightarrow. G) = \text{nodos-formula } F \cup \text{nodos-formula } G \end{aligned}$$

definition *nodos-conjunto-formulas* :: '*v* formula set $\Rightarrow$ '*v* set **where**

$$\text{nodos-conjunto-formulas } S = (\bigcup F \in S. \text{nodos-formula } F)$$

definition *maxima-altura* :: '*v* set $\Rightarrow$ '*v* rel $\Rightarrow$ '*v* formula set $\Rightarrow$ nat **where**

$$\text{maxima-altura } A r S = \text{Max } (\bigcup x \in \text{nodos-conjunto-formulas } S. \{altura\ A\ x\ r\})$$

Sea S un conjunto (finito) de fórmulas y h el máximo del conjunto de alturas de los nodos que ocurren en las fórmulas de S . La siguiente función retorna algún nodo del nivel $Niv(h + 1)$.

definition *nodo-sig-nivel-max* :: '*v* set $\Rightarrow$ '*v* rel $\Rightarrow$ '*v* formula set $\Rightarrow$ '*v*

where *nodo-sig-nivel-max* $A\ r\ S =$

$$(SOME\ u. u \in (nivel\ A\ r\ ((\text{maxima-altura } A\ r\ S) + 1)))$$

La siguiente función permite definir un modelo para $\mathcal{T}'$.

fun *interpretacion-camino* :: '*v* set $\Rightarrow$ '*v* rel $\Rightarrow$ '*v* $\Rightarrow$ ('*v* $\Rightarrow$ *v*-verdad) **where**

$$\text{interpretacion-camino } A\ r\ u = (\lambda v. (if\ (v, u) \in r\ \text{then } Verdad\ \text{else } Falso))$$

Cualquier subconjunto finito de $\mathcal{T}'$ es satisfacible:

lemma *camino-satisfacible*:

assumes *arbol-infinito* A r **and** *ramificacion-finita* A r **and** $S \subseteq (\mathcal{T}' A r)$
and *finite* S

shows *satisfacible* S

La siguiente es la definición del conjunto de nodos $\mathcal{B}$ que tiene la propiedad de ser una rama infinita. Antes de la demostración se enuncian algunas propiedades de $\mathcal{B}$ que se requieren en la prueba.

definition $\mathcal{B}$:: $'a \text{ set} \Rightarrow ('a \Rightarrow v\text{-verdad}) \Rightarrow 'a \text{ set}$ **where**
 $\mathcal{B} A I \equiv \{u \mid u. u \in A \wedge \text{valor } I \text{ } (Atomo \text{ } u) = Verdad\}$

$\mathcal{B}$ intersecta cada nivel en por lo menos un nodo:

lemma *interseccion-rama-nivel1*:

assumes *arbol-infinito* A r **and** *ramificacion-finita* A r
and $I: \forall F \in (\mathcal{F}' A r). \text{valor } I \text{ } F = Verdad$

shows $\forall n. \exists x. x \in \text{nivel } A \text{ } r \text{ } n \wedge x \in (\mathcal{B} A I)$ **using** *niveles-no-vacios*

$\mathcal{B}$ no intersecta en más de un nodo a cada nivel:

lemma *interseccion-rama-nivel2*:

assumes $I: \forall F \in (\mathcal{H}' A r). \text{valor } I \text{ } F = Verdad$
and $x \in (\mathcal{B} A I)$ **and** $y \in (\mathcal{B} A I)$ **and** $x \neq y$ **and** $n: x \in \text{nivel } A \text{ } r \text{ } n$
and $m: y \in \text{nivel } A \text{ } r \text{ } m$

shows $n \neq m$

$\mathcal{B}$ intersecta en un único nodo a cada nivel:

lemma *interseccion-rama-nivel*:

assumes *arbol-infinito* A r **and** *ramificacion-finita* A r
and $I: \forall F \in (\mathcal{F}' A r) \cup (\mathcal{H}' A r). \text{valor } I \text{ } F = Verdad$

shows $\forall n. \exists u. (\mathcal{B} A I) \cap \text{nivel } A \text{ } r \text{ } n = \{u\}$

Todo predecesor de cualquier elemento de $\mathcal{B}$ pertenece a $\mathcal{B}$:

lemma *predecesor-rama*:

assumes $I: \forall F \in (\mathcal{G}' A r). \text{valor } I \text{ } F = Verdad$
and $y \in (\mathcal{B} A I)$ **and** $(x, y) \in r$ **and** $x \in A$ **and** $y \in A$

shows $x \in (\mathcal{B} A I)$

$\mathcal{B}$ es una rama:

lemma *rama*:

assumes *arbol-infinito* A r **and** *ramificacion-finita* A r
and $I: \forall F \in (\mathcal{T}' A r). \text{valor } I \text{ } F = Verdad$

shows *rama* $(\mathcal{B} A I) A r$

$\mathcal{B}$ es un conjunto infinito:

lemma *rama-infinita*:

assumes *arbol-infinito* $A \ r$ **and** *ramificacion-finita* $A \ r$

and $I: \forall F \in (\mathcal{F}' A \ r). \text{valor } I \ F = \text{Verdad}$

shows *infinite* $(\mathcal{B} A \ I)$

Por último, tenemos la formalizacion del Lema de König [5.14](#).

theorem *Konig*:

assumes *arbol-infinito* $(A::'\text{nodos set}) \ r$

and *enumeracion* $(g:: \text{nat} \Rightarrow '\text{nodos})$

and *ramificacion-finita* $A \ r$

shows $\exists B. \text{rama-infinita } B \ A \ r$

proof–

have *satisfacible* $(\mathcal{T}' A \ r)$

proof–

have $\forall S. S \subseteq (\mathcal{T}' A \ r) \wedge (\text{finite } S) \longrightarrow \text{satisfacible } S$

using $\langle \text{arbol-infinito } A \ r \rangle \langle \text{ramificacion-finita } A \ r \rangle \text{camino-satisfacible}$

by *auto*

moreover

have $\exists h. \text{enumeracion } (h:: \text{nat} \Rightarrow '\text{nodos formula})$

using *EnumeracionFormulasP1*[*OF* $\langle \text{enumeracion } (g:: \text{nat} \Rightarrow '\text{nodos}) \rangle$]

by *auto*

ultimately

show *satisfacible* $(\mathcal{T}' A \ r)$

using *TeoremaCompacidad1*[*of* $(\mathcal{T}' A \ r)$] **by** *auto*

qed

hence $\exists I. (\forall F \in (\mathcal{T}' A \ r). \text{valor } I \ F = \text{Verdad})$

by(*unfold satisfacible-def*, *unfold modelo-def*, *auto*)

then obtain I **where** $I: \forall F \in (\mathcal{T}' A \ r). \text{valor } I \ F = \text{Verdad}$

by *auto*

hence $\forall F \in (\mathcal{F}' A \ r) \cup (\mathcal{G}' A \ r) \cup (\mathcal{H}' A \ r). \text{valor } I \ F = \text{Verdad}$

by(*unfold $\mathcal{T}'$ -def*)

hence $I1: \forall F \in (\mathcal{F}' A \ r). \text{valor } I \ F = \text{Verdad}$

and $I2: \forall F \in (\mathcal{G}' A \ r). \text{valor } I \ F = \text{Verdad}$

and $I3: \forall F \in (\mathcal{H}' A \ r). \text{valor } I \ F = \text{Verdad}$

by *auto*

let $?B = (\mathcal{B} A \ I)$

have *rama-infinita* $?B \ A \ r$

proof(*unfold rama-infinita-def*)

show *rama* $?B \ A \ r \wedge \text{infinite } ?B$

proof(*rule conjI*)

show *rama* $?B \ A \ r$

using $\langle \text{arbol-infinito } A \ r \rangle \langle \text{ramificacion-finita } A \ r \rangle I \text{rama}[\text{of } A \ r]$

```

    by auto
  show infinite ( $\mathcal{B}$   $A$   $I$ )
    using ⟨arbol-infinito  $A$   $r$ ⟩ ⟨ramificacion-finita  $A$   $r$ ⟩  $I1$  rama-infinita
  by auto
qed
qed
thus  $\exists B.$  rama-infinita  $B$   $A$   $r$  by auto
qed

```

6 Conclusiones y trabajos futuros

Las pruebas presentadas en este trabajo muestran las facilidades del lenguaje Isar para formalizar “textualmente” demostraciones clásicas de las matemáticas y de esta forma obtener pruebas automáticas bien estructuradas.

Desde el punto de vista pedagógico, el ejercicio de formalizar una demostración matemática, sirve para analizar los argumentos, propiedades y conceptos abstractos involucrados en la prueba. Este aspecto, en particular, es de gran utilidad en el proceso de automatizar demostraciones que no justifican explícitamente los resultados utilizados.

En general, en las demostraciones de teoremas, las justificaciones tales como “es obvio”, “así sucesivamente”, “por analogía” o “intuitivamente cierto”, que tradicionalmente no son probadas en los textos de matemáticas, deben ser verificados en la prueba formal, garantizando de esta forma la confiabilidad de la demostración usual. Sin embargo, este aspecto aumenta el tamaño de la prueba y su presentación se hace menos legible.

La teoría que permitió la formalización del teorema 4.5 sobre coloramiento de grafos contiene 15 definiciones y 35 lemas y la teoría para la formalización del Lema de König 5.14 contiene 31 definiciones y 66 lemas. En un documento anexo se incluye el desarrollo explícito de estas teorías.

El Teorema de los Cuatro Colores para grafos finitos planares se puede extender y formalizar para grafos planares enumerables utilizando el Teorema de Compacidad.

Los siguientes resultados básicos en combinatoria y teoría de grafos, que se proponen para su formalización, pueden ser verificados mecánicamente de manera análoga a los resultados que se presentan en este trabajo.

Definición 6.1. Sea $(A, \leq)$ un orden parcial, es decir, $\leq$ es una relación binaria en A , reflexiva, antisimétrica y transitiva.

Dos elementos $x, y \in A$ son comparables si $x \leq y$ o $y \leq x$ de lo contrario no son comparables.

El orden parcial $(A, \leq)$ es un orden total, si cualquier par de elementos de A son comparables.

Un conjunto $B \subseteq A$ es una cadena, si cualquier par de elementos de B son comparables.

Un conjunto $B \subseteq A$ es una anticadena si cualesquier par de elementos distintos de B no son comparables.

Definición 6.2. Sean $(A, \leq)$ un orden parcial y $(A, \leq')$ un orden total. $(A, \leq')$ extiende a $(A, \leq)$ si $x \leq y$ implica $x \leq' y$, para todo $x, y \in A$.

Teorema 6.3. Sea A un conjunto enumerable y $(A, \leq)$ un orden parcial. Entonces $(A, \leq)$ puede ser extendido a un orden total $(A, \leq')$.

Teorema 6.4 (Teorema de Dilworth). Sea A un conjunto enumerable y $(A, \leq)$ un orden parcial. Entonces A es la unión de (a lo más) k cadenas si y solamente si cada anticadena es de tamaño $\leq k$.

La versión finita de cada uno de los anteriores teoremas, es decir, para el caso en que A es finito, se demuestra por inducción sobre la cardinalidad de A y con base a estos resultados y el Teorema de Compacidad se demuestran respectivamente los teoremas para el caso en que A es infinito enumerable. Una formalización en Coq de la versión finita del Teorema de Dilworth aparece en [11].

Definición 6.5. Sean S un conjunto y $\{S_i\}_{i \in I}$ una colección de subconjuntos de S (no necesariamente distintos) con índices en el conjunto I .

Una sucesión $(x_i)_{i \in I}$ es un sistema de representantes distintos (SRD) para $\{S_i\}_{i \in I}$ si, para todo $i \in I$, $x_i \in S_i$ y, para todo $i, j \in I$, si $i \neq j$ entonces $x_i \neq x_j$.

Teorema 6.6 (Teorema de Hall, versión finita). Sean S un conjunto y n un entero positivo. Una colección $\{S_1, S_2, \dots, S_n\}$ de subconjuntos de S tiene un SRD si y solamente si:

(H) Para cualquier $1 \leq k \leq n$ y cualquier elección de k índices distintos $1 \leq i_1, \dots, i_k \leq n$, se tiene que $|S_{i_1} \cup \dots \cup S_{i_k}| \geq k$.

Teorema 6.7 (Teorema de Hall, versión infinita). *Sea S un conjunto. Una colección $\{S_i\}_{i \in \mathbb{N}^+}$ de subconjuntos **finitos** de S , tiene un SRD si y solamente si:*

(H) Para cualquier $k \in \mathbb{N}^+$ y cualquier elección de k índices distintos $i_1, \dots, i_k \in \mathbb{N}$, se tiene que $|S_{i_1} \cup \dots \cup S_{i_k}| \geq k$.*

La versión infinita del Teorema de Hall es una consecuencia de la versión finita y del Teorema de Compacidad. Una formalización en Isabelle/HOL de la versión finita aparece en [5] y [6].

Definición 6.8. *Sean A un conjunto no vacío y k un entero positivo.*

Un k -coloramiento de A es cualquier función $c : A \rightarrow \{1, \dots, k\}$. Un conjunto $B \subseteq A$ se dice que es monocromático, si existe $i \in \{1, \dots, k\}$ tal que para todo $x \in B$, $c(x) = i$ (todos los elementos de B tienen el mismo color).

En los siguientes dos teoremas, utilizamos la notación $[A]^k$ para denotar la colección de todos los subconjuntos de A de cardinalidad k , es decir, $[A]^k = \{X \subseteq A \mid |X| = k\}$.

Teorema 6.9 (Teorema de Ramsey, versión finita). *Sean k, m y d enteros positivos. Entonces, existe n tal que para todo k -coloramiento de $[\{1, \dots, n\}]^d$, existe un conjunto $A \subseteq \{1, \dots, n\}$ de cardinalidad m tal que $[A]^d$ es monocromático.*

En general, el teorema anterior es válido para cualquier otro conjunto de cardinalidad n diferente de $\{1, \dots, n\}$.

Teorema 6.10 (Teorema de Ramsey, versión infinita). *Sean k y d enteros positivos. Para todo k -coloramiento de $[\mathbb{N}]^d$, existe un conjunto infinito $A \subseteq \mathbb{N}$ tal que el conjunto $[A]^d$ es monocromático.*

En general, el teorema anterior es válido para cualquier otro conjunto infinito enumerable distinto a $\mathbb{N}$.

La prueba del Teorema de Ramsey para la versión *infinita* es por inducción sobre el número d ; una formalización en Isabelle/HOL aparece en [10]. La respectiva versión finita se demuestra a partir de la versión infinita utilizando el Teorema de Compacidad.

7 Anexo 1. Teoría Compacidad

Se formaliza en Isabelle/HOL, utilizando la formalización del Teorema de Existencia de Modelos, el Teorema de Compacidad para la Lógica Proposicional:

Sean L un lenguaje proposicional enumerable y S un conjunto de fórmulas proposicionales. Si todo subconjunto finito de S es satisfacible, entonces S es satisfacible.

lemma *NosatisfacibleAtom*:

shows $\neg(\text{satisfacible } \{F, \neg.F\})$

proof (*rule notI*)

assume *hip*: *satisfacible* $\{F, \neg.F\}$

show *False*

proof –

have $\exists I. I \text{ modelo } \{F, \neg.F\}$ **using** *hip* **by** (*unfold satisfacible-def, auto*)

then obtain *I* **where** *I*: *valor* *I* *F* = *Verdad*

and *valor* *I* $(\neg.F)$ = *Verdad*

by (*unfold modelo-def, auto*)

thus *False* **by** (*auto simp add: v-negacion-def*)

qed

qed

lemma *comp1*:

assumes $\forall (A::'b \text{ formula set}). (A \subseteq W \wedge \text{finite } A) \longrightarrow \text{satisfacible } A$

shows $(\forall P. \neg (\text{Atom } P \in W \wedge (\neg. \text{Atom } P) \in W))$

proof (*rule allI notI*) +

fix *P*

assume *h1*: *Atom* *P* $\in W \wedge (\neg. \text{Atom } P) \in W$

show *False*

proof –

have $\{\text{Atom } P, (\neg. \text{Atom } P)\} \subseteq W$ **using** *h1* **by** *simp*

moreover

have *finite* $\{\text{Atom } P, (\neg. \text{Atom } P)\}$ **by** *simp*

ultimately

have $\{\text{Atom } P, (\neg. \text{Atom } P)\} \subseteq W \wedge \text{finite } \{\text{Atom } P, (\neg. \text{Atom } P)\}$ **by** *simp*

moreover

have $(\{\text{Atom } P, (\neg. \text{Atom } P)\} \subseteq W \wedge \text{finite } \{\text{Atom } P, (\neg. \text{Atom } P)\}) \longrightarrow \text{satisfacible } \{\text{Atom } P, (\neg. \text{Atom } P)\}$

using *assms* **by** (*rule-tac x = \{\text{Atom } P, (\neg. \text{Atom } P)\}* **in** *allE, auto*)

ultimately

have *satisfacible* $\{\text{Atom } P, (\neg. \text{Atom } P)\}$ **by** *simp*

thus *False* **using** *NosatisfacibleAtom* **by** *auto*

qed

qed

lemma *NosatisfacibleFF*:

shows $\neg (\text{satisfacible } \{FF\})$

proof –

have $\forall I. \text{valor } I \text{ } FF = \text{Falso}$ **by** *simp*
hence $\forall I. \neg (I \text{ modelo } \{FF\})$ **by**(*unfold modelo-def, auto*)
thus *?thesis* **by**(*unfold satisfacible-def, auto*)
qed

lemma comp2:
assumes $\forall (A::'b \text{ formula set}). (A \subseteq W \wedge \text{finite } A) \longrightarrow \text{satisfacible } A$
shows $FF \notin W$
proof (*rule notI*)
assume *hip*: $FF \in W$
show *False*
proof –
have $\{FF\} \subseteq W$ **using** *hip* **by** *simp*
moreover
have *finite* $\{FF\}$ **by** *simp*
ultimately
have $\{FF\} \subseteq W \wedge \text{finite } \{FF\}$ **by** *simp*
moreover
have $(\{FF::'b \text{ formula}\} \subseteq W \wedge \text{finite } \{FF\}) \longrightarrow$
 $\text{satisfacible } \{FF::'b \text{ formula}\}$
using *assms* **by**(*rule-tac x = \{FF::'b formula\} in allE, auto*)
ultimately
have *satisfacible* $\{FF::'b \text{ formula}\}$ **by** *simp*
thus *False* **using** *NosatisfacibleFF* **by** *auto*
qed
qed

lemma NosatisfacibleFFa:
shows $\neg (\text{satisfacible } \{\neg.TT\})$
proof –
have $\forall I. \text{valor } I \text{ } TT = \text{Verdad}$ **by** *simp*
have $\forall I. \text{valor } I (\neg.TT) = \text{Falso}$ **by**(*auto simp add: v-negacion-def*)
hence $\forall I. \neg (I \text{ modelo } \{\neg.TT\})$ **by**(*unfold modelo-def, auto*)
thus *?thesis* **by**(*unfold satisfacible-def, auto*)
qed

lemma comp3:
assumes $\forall (A::'b \text{ formula set}). (A \subseteq W \wedge \text{finite } A) \longrightarrow \text{satisfacible } A$
shows $\neg.TT \notin W$
proof (*rule notI*)
assume *hip*: $\neg.TT \in W$
show *False*

proof –
 have $\{\neg.TT\} \subseteq W$ **using** *hip* **by** *simp*
 moreover
 have *finite* $\{\neg.TT\}$ **by** *simp*
 ultimately
 have $\{\neg.TT\} \subseteq W \wedge \text{finite } \{\neg.TT\}$ **by** *simp*
 moreover
 have $(\{\neg.TT::'b \text{ formula}\} \subseteq W \wedge \text{finite } \{\neg.TT\}) \longrightarrow$
 satisfacible $\{\neg.TT::'b \text{ formula}\}$
 using *assms* **by**(*rule-tac* $x = \{\neg.TT::'b \text{ formula}\}$ **in** *allE*, *auto*)
 ultimately
 have *satisfacible* $\{\neg.TT::'b \text{ formula}\}$ **by** *simp*
 thus *False* **using** *NosatisfacibleFFa* **by** *auto*
qed
qed

lemma *SubSatis*:

assumes *hip1*: *satisfacible* S **and** *hip2*: $S' \subseteq S$
 shows *satisfacible* S'

proof –
 have $\exists I. \forall F \in S. \text{valor } I F = \text{Verdad}$ **using** *hip1*
 by (*unfold satisfacible-def*, *unfold modelo-def*, *auto*)
 hence $\exists I. \forall F \in S'. \text{valor } I F = \text{Verdad}$ **using** *hip2* **by** *auto*
 thus *?thesis* **by**(*unfold satisfacible-def*, *unfold modelo-def*, *auto*)
qed

lemma *satisfacibleUnion1*:

assumes *satisfacible* $(A \cup \{\neg.\neg.F\})$
 shows *satisfacible* $(A \cup \{F\})$

proof –
 have $\exists I. \forall G \in (A \cup \{\neg.\neg.F\}). \text{valor } I G = \text{Verdad}$
 using *assms* **by**(*unfold satisfacible-def*, *unfold modelo-def*, *auto*)
 then obtain I **where** $I: \forall G \in (A \cup \{\neg.\neg.F\}). \text{valor } I G = \text{Verdad}$
 by *auto*
 hence $1: \forall G \in A. \text{valor } I G = \text{Verdad}$
 and $2: \text{valor } I (\neg.\neg.F) = \text{Verdad}$
 by *auto*
 have *tipoFormula* $(\neg.\neg.F) = \text{NoNo}$ **by** *auto*
 hence $\text{valor } I F = \text{Verdad}$ **using** *EquivNoNoComp*[*of* $\neg.\neg.F$] 2
 by (*unfold equivalentes-def*, *unfold Comp1-def*, *auto*)
 hence $\forall G \in A \cup \{F\}. \text{valor } I G = \text{Verdad}$ **using** 1 **by** *auto*
 thus *satisfacible* $(A \cup \{F\})$
 by(*unfold satisfacible-def*, *unfold modelo-def*, *auto*)
qed

lemma comp4:
assumes *hip1*: $\forall (A::'b \text{ formula set}). (A \subseteq W \wedge \text{finite } A) \longrightarrow \text{satisfacible } A$
and *hip2*: $\neg.\neg.F \in W$
shows $\forall (A::'b \text{ formula set}). (A \subseteq W \cup \{F\} \wedge \text{finite } A) \longrightarrow \text{satisfacible } A$
proof (*rule allI, rule impI*)+
fix *A*
assume *hip*: $A \subseteq W \cup \{F\} \wedge \text{finite } A$
show *satisfacible A*
proof –
have $A - \{F\} \subseteq W \wedge \text{finite } (A - \{F\})$ **using** *hip* **by** *auto*
hence $(A - \{F\}) \cup \{\neg.\neg.F\} \subseteq W \wedge \text{finite } ((A - \{F\}) \cup \{\neg.\neg.F\})$
using *hip2* **by** *auto*
hence *satisfacible* $((A - \{F\}) \cup \{\neg.\neg.F\})$ **using** *hip1* **by** *auto*
hence *satisfacible* $((A - \{F\}) \cup \{F\})$ **using** *satisfacibleUnion1* **by** *blast*
moreover
have $A \subseteq (A - \{F\}) \cup \{F\}$ **by** *auto*
ultimately
show *satisfacible A* **using** *SubSatis* **by** *auto*
qed
qed

lemma satisfacibleUnion2:
assumes *hip1*: *FormulaAlfa F* **and** *hip2*: *satisfacible (A $\cup$ {F})*
shows *satisfacible (A $\cup$ {Comp1 F, Comp2 F})*
proof –
have $\exists I. \forall G \in A \cup \{F\}. \text{valor } I \ G = \text{Verdad}$
using *hip2* **by** (*unfold satisfacible-def, unfold modelo-def, auto*)
then obtain *I* **where** *I*: $\forall G \in A \cup \{F\}. \text{valor } I \ G = \text{Verdad}$ **by** *auto*
hence *1*: $\forall G \in A. \text{valor } I \ G = \text{Verdad}$ **and** *2*: $\text{valor } I \ F = \text{Verdad}$ **by** *auto*
have *tipoFormula F = Alfa* **using** *hip1 noAlfaBeta noAlfaNoNo* **by** *auto*
hence *equivalentes F (Comp1 F $\wedge$. Comp2 F)*
using *2 EquivAlfaComp[of F]* **by** *auto*
hence $\text{valor } I \ (Comp1 \ F \ \wedge. \ Comp2 \ F) = \text{Verdad}$
using *2* **by** (*unfold equivalentes-def, auto*)
hence $\text{valor } I \ (Comp1 \ F) = \text{Verdad} \wedge \text{valor } I \ (Comp2 \ F) = \text{Verdad}$
using *ValoresConjuncion* **by** *auto*
hence $\forall G \in A \cup \{Comp1 \ F, \ Comp2 \ F\}. \text{valor } I \ G = \text{Verdad}$ **using** *1* **by** *auto*
thus *satisfacible (A $\cup$ {Comp1 F, Comp2 F})*
by (*unfold satisfacible-def, unfold modelo-def, auto*)
qed

lemma comp5:
assumes *hip0*: *FormulaAlfa F*
and *hip1*: $\forall (A::'b \text{ formula set}). (A \subseteq W \wedge \text{finite } A) \longrightarrow \text{satisfacible } A$
and *hip2*: $F \in W$
shows $\forall (A::'b \text{ formula set}). (A \subseteq W \cup \{\text{Comp1 } F, \text{Comp2 } F\} \wedge \text{finite } A) \longrightarrow \text{satisfacible } A$
proof (*rule allI, rule impI*)+
fix *A*
assume *hip*: $A \subseteq W \cup \{\text{Comp1 } F, \text{Comp2 } F\} \wedge \text{finite } A$
show *satisfacible A*
proof –
have $A - \{\text{Comp1 } F, \text{Comp2 } F\} \subseteq W \wedge \text{finite } (A - \{\text{Comp1 } F, \text{Comp2 } F\})$
using *hip* **by** *auto*
hence $(A - \{\text{Comp1 } F, \text{Comp2 } F\}) \cup \{F\} \subseteq W \wedge$
 $\text{finite } ((A - \{\text{Comp1 } F, \text{Comp2 } F\}) \cup \{F\})$
using *hip2* **by** *auto*
hence *satisfacible* $((A - \{\text{Comp1 } F, \text{Comp2 } F\}) \cup \{F\})$
using *hip1* **by** *auto*
hence *satisfacible* $((A - \{\text{Comp1 } F, \text{Comp2 } F\}) \cup \{\text{Comp1 } F, \text{Comp2 } F\})$
using *hip0* *satisfacibleUnion2* **by** *auto*
moreover
have $A \subseteq (A - \{\text{Comp1 } F, \text{Comp2 } F\}) \cup \{\text{Comp1 } F, \text{Comp2 } F\}$ **by** *auto*
ultimately
show *satisfacible A* **using** *SubSatis* **by** *auto*
qed
qed

lemma satisfacibleUnion3:
assumes *hip1*: *FormulaBeta F* **and** *hip2*: *satisfacible* $(A \cup \{F\})$
shows *satisfacible* $(A \cup \{\text{Comp1 } F\}) \vee \text{satisfacible } (A \cup \{\text{Comp2 } F\})$
proof –
obtain *I* **where** *I*: $\forall G \in (A \cup \{F\}). \text{valor } I \ G = \text{Verdad}$
using *hip2* **by** (*unfold satisfacible-def, unfold modelo-def, auto*)
hence *S1*: $\forall G \in A. \text{valor } I \ G = \text{Verdad}$
and *S2*: $\text{valor } I \ F = \text{Verdad}$
by *auto*
have *V*: $\text{valor } I \ (\text{Comp1 } F) = \text{Verdad} \vee \text{valor } I \ (\text{Comp2 } F) = \text{Verdad}$
using *hip1 S2* *EquivBetaComp[of F]* *ValoresDisyuncion*
by (*unfold equivalentes-def, auto*)
have $((\forall G \in A. \text{valor } I \ G = \text{Verdad}) \wedge \text{valor } I \ (\text{Comp1 } F) = \text{Verdad}) \vee$
 $((\forall G \in A. \text{valor } I \ G = \text{Verdad}) \wedge \text{valor } I \ (\text{Comp2 } F) = \text{Verdad})$
using *V*
proof (*rule disjE*)

```

assume  $\text{valor } I \text{ (Comp1 } F) = \text{Verdad}$ 
hence  $(\forall G \in A. \text{valor } I \ G = \text{Verdad}) \wedge \text{valor } I \text{ (Comp1 } F) = \text{Verdad}$ 
  using  $S1$  by auto
thus ?thesis by simp
next
  assume  $\text{valor } I \text{ (Comp2 } F) = \text{Verdad}$ 
  hence  $(\forall G \in A. \text{valor } I \ G = \text{Verdad}) \wedge \text{valor } I \text{ (Comp2 } F) = \text{Verdad}$ 
    using  $S1$  by auto
    thus ?thesis by simp
qed
hence  $(\forall G \in A \cup \{\text{Comp1 } F\}. \text{valor } I \ G = \text{Verdad}) \vee$ 
   $(\forall G \in A \cup \{\text{Comp2 } F\}. \text{valor } I \ G = \text{Verdad})$ 
  by auto
hence  $(\exists I. \forall G \in A \cup \{\text{Comp1 } F\}. \text{valor } I \ G = \text{Verdad}) \vee$ 
   $(\exists I. \forall G \in A \cup \{\text{Comp2 } F\}. \text{valor } I \ G = \text{Verdad})$ 
  by auto
thus  $\text{satisfacible } (A \cup \{\text{Comp1 } F\}) \vee \text{satisfacible } (A \cup \{\text{Comp2 } F\})$ 
by  $(\text{unfold satisfacible-def}, \text{unfold modelo-def}, \text{auto})$ 
qed

```

lemma *comp6*:

```

assumes hip0:  $\text{FormulaBeta } F$ 
and hip1:  $\forall (A::'b \text{ formula set}). (A \subseteq W \wedge \text{finite } A) \longrightarrow \text{satisfacible } A$ 
and hip2:  $F \in W$ 
shows  $(\forall (A::'b \text{ formula set}). (A \subseteq W \cup \{\text{Comp1 } F\} \wedge \text{finite } A) \longrightarrow$ 
 $\text{satisfacible } A) \vee$ 
 $(\forall (A::'b \text{ formula set}). (A \subseteq W \cup \{\text{Comp2 } F\} \wedge \text{finite } A) \longrightarrow$ 
 $\text{satisfacible } A)$ 

```

proof –

```

{ assume hip3:  $\neg((\forall (A::'b \text{ formula set}). (A \subseteq W \cup \{\text{Comp1 } F\} \wedge \text{finite } A) \longrightarrow$ 
 $\text{satisfacible } A) \vee$ 
 $(\forall (A::'b \text{ formula set}). (A \subseteq W \cup \{\text{Comp2 } F\} \wedge \text{finite } A) \longrightarrow \text{satisfacible } A))$ 
have False

```

proof –

```

obtain  $A \ B$  where  $A1: A \subseteq W \cup \{\text{Comp1 } F\}$ 
and  $A2: \text{finite } A$ 
and  $A3: \neg \text{satisfacible } A$ 
and  $B1: B \subseteq W \cup \{\text{Comp2 } F\}$ 
and  $B2: \text{finite } B$ 
and  $B3: \neg \text{satisfacible } B$ 
using hip3 by auto
have  $a1: A - \{\text{Comp1 } F\} \subseteq W$ 
and  $a2: \text{finite } (A - \{\text{Comp1 } F\})$ 

```

```

    using A1 and A2 by auto
  hence satisfiable (A - {Comp1 F}) using hip1 by simp
  have b1: B - {Comp2 F} ⊆ W
    and b2: finite (B - {Comp2 F})
    using B1 and B2 by auto
  hence satisfiable (B - {Comp2 F}) using hip1 by simp
  moreover
  have (A - {Comp1 F}) ∪ (B - {Comp2 F}) ∪ {F} ⊆ W
    and finite ((A - {Comp1 F}) ∪ (B - {Comp2 F}) ∪ {F})
    using a1 a2 b1 b2 hip2 by auto
  hence satisfiable ((A - {Comp1 F}) ∪ (B - {Comp2 F}) ∪ {F})
    using hip1 by simp
  hence satisfiable ((A - {Comp1 F}) ∪ (B - {Comp2 F}) ∪ {Comp1 F})
  ∨ satisfiable ((A - {Comp1 F}) ∪ (B - {Comp2 F}) ∪ {Comp2 F})
    using hip0 satisfiableUnion3 by auto
  moreover
  have A ⊆ (A - {Comp1 F}) ∪ (B - {Comp2 F}) ∪ {Comp1 F}
    and B ⊆ (A - {Comp1 F}) ∪ (B - {Comp2 F}) ∪ {Comp2 F}
    by auto
  ultimately
  have satisfiable A ∨ satisfiable B using SubSatis by auto
  thus False using A3 B3 by simp
qed }
thus ?thesis by auto
qed

```

lemma *ConsistenciaCompacidad:*

shows *consistenciaP*{*W*::'b formula set. $\forall A. (A \subseteq W \wedge \text{finite } A) \longrightarrow \text{satisfiable } A$ }

proof (*unfold consistenciaP-def, rule allI, rule impI*)

let *?C* = {*W*::'b formula set. $\forall A. (A \subseteq W \wedge \text{finite } A) \longrightarrow \text{satisfiable } A$ }

fix *W* :: 'b formula set

assume *W* ∈ *?C*

hence *hip*: $\forall A. (A \subseteq W \wedge \text{finite } A) \longrightarrow \text{satisfiable } A$ **by** *simp*

show $(\forall P. \neg (\text{Atomo } P \in W \wedge (\neg.\text{Atomo } P) \in W)) \wedge$

$FF \notin W \wedge$

$\neg.TT \notin W \wedge$

$(\forall F. \neg.\neg.F \in W \longrightarrow W \cup \{F\} \in ?C) \wedge$

$(\forall F. (\text{FormulaAlfa } F) \wedge F \in W \longrightarrow$

$(W \cup \{\text{Comp1 } F, \text{Comp2 } F\} \in ?C)) \wedge$

$(\forall F. (\text{FormulaBeta } F) \wedge F \in W \longrightarrow$

$(W \cup \{\text{Comp1 } F\} \in ?C \vee W \cup \{\text{Comp2 } F\} \in ?C))$

proof –

```

have  $(\forall P. \neg (Atomo\ P \in W \wedge (\neg. Atomo\ P) \in W))$ 
  using hip comp1 by simp
moreover
have  $FF \notin W$  using hip comp2 by auto
moreover
have  $\neg. TT \notin W$  using hip comp3 by auto
moreover
have  $\forall F. (\neg.\neg.F) \in W \longrightarrow W \cup \{F\} \in ?C$ 
proof (rule allI impI)+
  fix  $F$ 
  assume hip1:  $\neg.\neg.F \in W$ 
  show  $W \cup \{F\} \in ?C$  using hip hip1 comp4 by simp
qed
moreover
have
 $\forall F. (FormulaAlfa\ F) \wedge F \in W \longrightarrow (W \cup \{Comp1\ F, Comp2\ F\} \in ?C)$ 
proof (rule allI impI)+
  fix  $F$ 
  assume  $FormulaAlfa\ F \wedge F \in W$ 
  thus  $W \cup \{Comp1\ F, Comp2\ F\} \in ?C$  using hip comp5[of F] by blast
qed
moreover
have  $\forall F. (FormulaBeta\ F) \wedge F \in W \longrightarrow$ 
   $(W \cup \{Comp1\ F\} \in ?C \vee W \cup \{Comp2\ F\} \in ?C)$ 
proof (rule allI impI)+
  fix  $F$ 
  assume  $(FormulaBeta\ F) \wedge F \in W$ 
  thus  $W \cup \{Comp1\ F\} \in ?C \vee W \cup \{Comp2\ F\} \in ?C$ 
    using hip comp6[of F] by blast
qed
ultimately
show ?thesis by auto
qed
qed

theorem TeoremaCompacidad1:
assumes hip1:  $\exists g. enumeracion\ (g:: nat \Rightarrow 'b\ formula)$ 
and hip2:  $\forall A. (A \subseteq (S:: 'b\ formula\ set) \wedge finite\ A) \longrightarrow satisfacible\ A$ 
shows satisfacible S
proof –
let  $?C = \{W:: 'b\ formula\ set. \forall A. (A \subseteq W \wedge finite\ A) \longrightarrow satisfacible\ A\}$ 
have consistenciaP ?C
  using ConsistenciaCompacidad by simp

```

moreover
have $S \in ?C$ **using** *hip2* **by** *simp*
ultimately
show *satisfacible* S **using** *hip1* **and** *TeoremaExistenciaModelos* **by** *auto*
qed

corollary *TeoremaCompacidad2*:
assumes $\forall A. (A \subseteq (S :: \text{nat formula set}) \wedge \text{finite } A) \longrightarrow \text{satisfacible } A$
shows *satisfacible* S
using *assms* **and** *EnumeracionFormulasNat* **and** *TeoremaCompacidad1*
by *auto*

8 Anexo 2. Teoría Coloración

Se formaliza en Isabelle/HOL, utilizando la formalización del Teorema de Compacidad para la Lógica Proposicional, el siguiente teorema:

Un grafo G infinito enumerable es k -colorable si todo subgrafo finito de G es k -colorable.

type-synonym $'v \text{ digrafo} = ('v \text{ set}) \times (('v \times 'v) \text{ set})$

abbreviation $\text{vert} :: 'v \text{ digrafo} \Rightarrow 'v \text{ set}$ ($V[-]$ [80] 80) **where**
 $V[G] \equiv \text{fst } G$

abbreviation $\text{arc} :: 'v \text{ digrafo} \Rightarrow ('v \times 'v) \text{ set}$ ($E[-]$ [80] 80) **where**
 $E[G] \equiv \text{snd } G$

definition $\text{es-grafo} :: 'v \text{ digrafo} \Rightarrow \text{bool}$ **where**
 $\text{es-grafo } G \equiv \forall u v. (u, v) \in E[G] \longrightarrow u \in V[G] \wedge v \in V[G] \wedge u \neq v$

definition $\text{es-subgrafo-inducido} :: 'v \text{ digrafo} \Rightarrow 'v \text{ digrafo} \Rightarrow \text{bool}$ **where**
 $\text{es-subgrafo-inducido } H G \equiv$
 $(V[H] \subseteq V[G]) \wedge E[H] = E[G] \cap ((V[H]) \times (V[H]))$

lemma

assumes $\text{es-grafo } G$ **and** $\text{es-subgrafo-inducido } H G$
shows $\text{es-grafo } H$

proof(*unfold es-grafo-def*)

show $\forall u v. (u, v) \in E[H] \longrightarrow u \in V[H] \wedge v \in V[H] \wedge u \neq v$

proof((*rule allI*)+, *rule impI*)

fix $u v$

assume $(u, v) \in E[H]$

show $u \in V[H] \wedge v \in V[H] \wedge u \neq v$
proof–
have $(u, v) \in E[G] \cap (V[H]) \times (V[H])$ **using** $\langle (u, v) \in E[H] \rangle$ *assms(2)*
by *(unfold es-subgrafo-inducido-def, auto)*
hence 1: $(u, v) \in E[G]$ **and 2:** $u \in V[H] \wedge v \in V[H]$ **by** *auto*
have $u \neq v$ **using 1** *(es-grafo G)* **by** *(unfold es-grafo-def, auto)*
thus $u \in V[H] \wedge v \in V[H] \wedge u \neq v$ **using 2** **by** *auto*
qed
qed
qed

definition *grafo-finito* :: $'v \text{ digrafo} \Rightarrow \text{bool}$ **where**
grafo-finito $G \equiv \text{finite } (V[G]) \wedge \text{finite } (E[G])$

abbreviation *grafo-infinito* :: $'v \text{ digrafo} \Rightarrow \text{bool}$
where *grafo-infinito* $G \equiv \neg \text{grafo-finito } G$

definition *grafo-enumerable* :: $'v \text{ digrafo} \Rightarrow \text{bool}$ **where**
grafo-enumerable $G \equiv \exists g. \text{enumeracion } (g :: \text{nat} \Rightarrow 'v)$

definition *coloramiento* :: $('v \Rightarrow \text{nat}) \Rightarrow \text{nat} \Rightarrow 'v \text{ digrafo} \Rightarrow \text{bool}$ **where**
coloramiento $c \ k \ G \equiv$
 $(\forall u. u \in V[G] \longrightarrow c(u) \leq k) \wedge (\forall u \ v. (u, v) \in E[G] \longrightarrow c(u) \neq c(v))$

definition *colorable* :: $'v \text{ digrafo} \Rightarrow \text{nat} \Rightarrow \text{bool}$ **where**
colorable $G \ k \equiv \exists c. \text{coloramiento } c \ k \ G$

primrec *disyuncion-atomicas* :: $'v \Rightarrow \text{nat} \Rightarrow ('v \times \text{nat}) \text{formula}$ **where**
disyuncion-atomicas $v \ 0 = \text{Atomo } (v, 0)$
 $| \text{disyuncion-atomicas } v \ (\text{Suc } k) =$
 $(\text{Atomo } (v, \text{Suc } k)) \vee. (\text{disyuncion-atomicas } v \ k)$

definition $\mathcal{F}$:: $'v \text{ digrafo} \Rightarrow \text{nat} \Rightarrow (('v \times \text{nat}) \text{formula}) \text{set}$ **where**
 $\mathcal{F} \ G \ k \equiv (\bigcup v \in V[G]. \{\text{disyuncion-atomicas } v \ k\})$

definition $\mathcal{G}$:: $'v \text{ digrafo} \Rightarrow \text{nat} \Rightarrow ('v \times \text{nat}) \text{formula set}$ **where**
 $\mathcal{G} \ G \ k \equiv \{\neg. (\text{Atomo } (v, i) \wedge. \text{Atomo}(v, j))$
 $\quad | \ v \ i \ j. (v \in V[G]) \wedge (0 \leq i \wedge 0 \leq j \wedge i \leq k \wedge j \leq k \wedge i \neq j)\}$

definition $\mathcal{H}$:: $'v \text{ digrafo} \Rightarrow \text{nat} \Rightarrow ('v \times \text{nat}) \text{formula set}$ **where**
 $\mathcal{H} \ G \ k \equiv \{\neg. (\text{Atomo } (u, i) \wedge. \text{Atomo}(v, i))$

$$|u \ v \ i \ . \ (u \in V[G] \wedge v \in V[G] \wedge (u,v) \in E[G]) \wedge (0 \leq i \wedge i \leq k)\}$$

definition $\mathcal{T} :: 'v \text{ digrafo} \Rightarrow \text{nat} \Rightarrow ('v \times \text{nat})\text{formula set}$ **where**
 $\mathcal{T} \ G \ k \equiv (\mathcal{F} \ G \ k) \cup (\mathcal{G} \ G \ k) \cup (\mathcal{H} \ G \ k)$

primrec *vertices-formula* $:: ('v \times \text{nat})\text{formula} \Rightarrow 'v \text{ set}$ **where**

vertices-formula $FF = \{\}$
 $|$ *vertices-formula* $TT = \{\}$
 $|$ *vertices-formula* $(\text{Atomo } P) = \{\text{fst } P\}$
 $|$ *vertices-formula* $(\neg. F) = \text{vertices-formula } F$
 $|$ *vertices-formula* $(F \wedge. G) = \text{vertices-formula } F \cup \text{vertices-formula } G$
 $|$ *vertices-formula* $(F \vee. G) = \text{vertices-formula } F \cup \text{vertices-formula } G$
 $|$ *vertices-formula* $(F \rightarrow. G) = \text{vertices-formula } F \cup \text{vertices-formula } G$

definition *vertices-conjunto-formulas* $:: ('v \times \text{nat})\text{formula set} \Rightarrow 'v \text{ set}$ **where**
vertices-conjunto-formulas $S = (\bigcup F \in S. \text{vertices-formula } F)$

lemma *finito-vertices*:

shows *finite* $(\text{vertices-formula } F)$
by $(\text{induct } F, \text{auto})$

lemma *vertices-disyuncion*:

assumes $F = \text{disyuncion-atomicas } v \ k$ **shows** $\text{vertices-formula } F = \{v\}$

proof –

have $F = \text{disyuncion-atomicas } v \ k \implies \text{vertices-formula } F = \{v\}$

proof $(\text{induct } k \text{ arbitrary: } F)$

case 0

assume $F = \text{disyuncion-atomicas } v \ 0$

hence $F = \text{Atomo } (v, 0)$ **by** *auto*

thus $\text{vertices-formula } F = \{v\}$ **by** *auto*

next

case $(\text{Suc } k)$

have $F = (\text{Atomo } (v, \text{Suc } k)) \vee. (\text{disyuncion-atomicas } v \ k)$

using $\text{Suc}(2)$ **by** *auto*

hence $\text{vertices-formula } F = \text{vertices-formula } (\text{Atomo } (v, \text{Suc } k)) \cup \text{vertices-formula } (\text{disyuncion-atomicas } v \ k)$ **by** *auto*

hence $\text{vertices-formula } F = \{v\} \cup \text{vertices-formula } (\text{disyuncion-atomicas } v \ k)$

by *auto*

hence $\text{vertices-formula } F = \{v\} \cup \{v\}$ **using** $\text{Suc}(1)$ **by** *auto*

thus $\text{vertices-formula } F = \{v\}$ **by** *auto*

qed

thus *?thesis* **using** *assms* **by** *auto*

qed

lemma *vertices-algun:*

shows *vertices-conjunto-formulas* $(\mathcal{F} \ G \ k) \subseteq V[G]$

proof

fix x

assume *hip*: $x \in \text{vertices-conjunto-formulas } (\mathcal{F} \ G \ k)$ **show** $x \in V[G]$

proof–

have $x \in (\bigcup F \in (\mathcal{F} \ G \ k). \text{vertices-formula } F)$ **using** *hip*

by (*unfold vertices-conjunto-formulas-def, auto*)

hence $\exists F \in (\mathcal{F} \ G \ k). x \in \text{vertices-formula } F$ **by** *auto*

then obtain F **where** $F \in (\mathcal{F} \ G \ k)$ **and** $x \in \text{vertices-formula } F$ **by** *auto*

hence $\exists v \in V[G]. F \in \{\text{disyuncion-atomicas } v \ k\}$ **by** (*unfold \mathcal{F}-def, auto*)

then obtain v **where** $v \in V[G]$ **and** $F \in \{\text{disyuncion-atomicas } v \ k\}$ **by** *auto*

hence $F = \text{disyuncion-atomicas } v \ k$ **by** *auto*

hence $\text{vertices-formula } F = \{v\}$

using *vertices-disyuncion* [*OF* $(F = \text{disyuncion-atomicas } v \ k)$] **by** *auto*

hence $x = v$ **using** x **by** *auto*

thus *?thesis* **using** v **by** *auto*

qed

qed

lemma *vertices-maximoC:*

shows *vertices-conjunto-formulas* $(\mathcal{G} \ G \ k) \subseteq V[G]$

proof

fix x

assume *hip*: $x \in \text{vertices-conjunto-formulas } (\mathcal{G} \ G \ k)$ **show** $x \in V[G]$

proof–

have $x \in (\bigcup F \in (\mathcal{G} \ G \ k). \text{vertices-formula } F)$ **using** *hip*

by (*unfold vertices-conjunto-formulas-def, auto*)

hence $\exists F \in (\mathcal{G} \ G \ k). x \in \text{vertices-formula } F$ **by** *auto*

then obtain F **where** $F \in (\mathcal{G} \ G \ k)$ **and** $x \in \text{vertices-formula } F$

by *auto*

hence $\exists v \ i \ j. v \in V[G] \wedge F = \neg.(Atomo \ (v, i) \wedge. Atomo(v,j))$

by (*unfold \mathcal{G}-def, auto*)

then obtain $v \ i \ j$ **where** $v \in V[G]$ **and** $F = \neg.(Atomo \ (v, i) \wedge. Atomo(v,j))$

by *auto*

hence $v: v \in V[G]$ **and** $F = \neg.(Atomo \ (v, i) \wedge. Atomo(v,j))$ **by** *auto*

hence $v: v \in V[G]$ **and** $\text{vertices-formula } F = \{v\}$ **by** *auto*

thus $x \in V[G]$ **using** x **by** *auto*

qed

qed

lemma *vertices-diferentesC:*

shows $vertices\text{-conjunto-formulas}(\mathcal{H} \ G \ k) \subseteq V[G]$
proof
fix x
assume $hip: x \in vertices\text{-conjunto-formulas}(\mathcal{H} \ G \ k)$ **show** $x \in V[G]$
proof–
have $x \in (\bigcup F \in (\mathcal{H} \ G \ k). vertices\text{-formula } F)$ **using** hip
by $(unfold \ vertices\text{-conjunto-formulas-def}, auto)$
hence $\exists F \in (\mathcal{H} \ G \ k) . x \in vertices\text{-formula } F$ **by** $auto$
then obtain F **where** $F \in (\mathcal{H} \ G \ k)$ **and** $x: x \in vertices\text{-formula } F$
by $auto$
hence $\exists u \ v \ i . u \in V[G] \wedge v \in V[G] \wedge F = \neg.(Atomo \ (u, i) \wedge Atomo(v, i))$
by $(unfold \ \mathcal{H}\text{-def}, auto)$
then obtain $u \ v \ i$
where $u \in V[G]$ **and** $v \in V[G]$ **and** $F = \neg.(Atomo \ (u, i) \wedge Atomo(v, i))$
by $auto$
hence $u \in V[G]$ **and** $v \in V[G]$ **and** $F = \neg.(Atomo \ (u, i) \wedge Atomo(v, i))$
by $auto$
hence $u: u \in V[G]$ **and** $v: v \in V[G]$ **and** $vertices\text{-formula } F = \{u, v\}$
by $auto$
hence $x = u \vee x = v$ **using** x **by** $auto$
thus $x \in V[G]$ **using** $u \ v$ **by** $auto$
qed
qed

lemma vv :

shows $vertices\text{-conjunto-formulas} \ (A \cup B) = (vertices\text{-conjunto-formulas} \ A) \cup (vertices\text{-conjunto-formulas} \ B)$
by $(unfold \ vertices\text{-conjunto-formulas-def}, auto)$

lemma $vv1$:

assumes $F \in (\mathcal{F} \ G \ k)$
shows $(vertices\text{-formula } F) \subseteq (vertices\text{-conjunto-formulas} \ (\mathcal{F} \ G \ k))$
proof
fix x
assume $hip: x \in vertices\text{-formula } F$
show $x \in vertices\text{-conjunto-formulas} \ (\mathcal{F} \ G \ k)$
proof–
have $\exists F. F \in (\mathcal{F} \ G \ k) \wedge x \in vertices\text{-formula } F$ **using** $assms \ hip$ **by** $auto$
thus $?thesis$ **by** $(unfold \ vertices\text{-conjunto-formulas-def}, auto)$
qed
qed

lemma $vv2$:

```

assumes  $F \in (\mathcal{G} \ G \ k)$ 
shows  $(\text{vertices-formula } F) \subseteq (\text{vertices-conjunto-formulas } (\mathcal{G} \ G \ k))$ 
proof
  fix  $x$ 
  assume  $hip: x \in \text{vertices-formula } F$ 
  show  $x \in \text{vertices-conjunto-formulas } (\mathcal{G} \ G \ k)$ 
  proof–
    have  $\exists F. F \in (\mathcal{G} \ G \ k) \wedge x \in \text{vertices-formula } F$  using assms hip by auto
    thus ?thesis by (unfold vertices-conjunto-formulas-def, auto)
  qed
qed

```

```

lemma vv3:
  assumes  $F \in (\mathcal{H} \ G \ k)$ 
  shows  $(\text{vertices-formula } F) \subseteq (\text{vertices-conjunto-formulas } (\mathcal{H} \ G \ k))$ 
proof
  fix  $x$ 
  assume  $hip: x \in \text{vertices-formula } F$ 
  show  $x \in \text{vertices-conjunto-formulas } (\mathcal{H} \ G \ k)$ 
  proof–
    have  $\exists F. F \in (\mathcal{H} \ G \ k) \wedge x \in \text{vertices-formula } F$  using assms hip by auto
    thus ?thesis by (unfold vertices-conjunto-formulas-def, auto)
  qed
qed

```

```

lemma aa:
  shows  $\text{vertices-conjunto-formulas } (\mathcal{T} \ G \ k) \subseteq V[G]$ 
proof
  fix  $x$ 
  assume  $hip: x \in \text{vertices-conjunto-formulas } (\mathcal{T} \ G \ k)$  show  $x \in V[G]$ 
  proof–
    have  $x \in \text{vertices-conjunto-formulas } ((\mathcal{F} \ G \ k) \cup (\mathcal{G} \ G \ k) \cup (\mathcal{H} \ G \ k))$ 
    using hip by (unfold T-def, auto)
    hence  $x \in \text{vertices-conjunto-formulas } ((\mathcal{F} \ G \ k) \cup (\mathcal{G} \ G \ k)) \cup$ 
     $\text{vertices-conjunto-formulas}(\mathcal{H} \ G \ k)$ 
    using vv[of (F G k) (G G k)] by auto
    hence  $x \in \text{vertices-conjunto-formulas } ((\mathcal{F} \ G \ k) \cup (\mathcal{G} \ G \ k)) \vee$ 
     $x \in \text{vertices-conjunto-formulas}(\mathcal{H} \ G \ k)$ 
    by auto
    thus ?thesis
  proof(rule disjE)
    assume  $hip: x \in \text{vertices-conjunto-formulas } (\mathcal{F} \ G \ k \cup \mathcal{G} \ G \ k)$ 
    hence  $x \in (\bigcup F \in (\mathcal{F} \ G \ k) \cup (\mathcal{G} \ G \ k). \text{vertices-formula } F)$ 

```

by(*unfold vertices-conjunto-formulas-def*, *auto*)
 then obtain F
 where $F: F \in (\mathcal{F} \ G \ k) \cup (\mathcal{G} \ G \ k)$ and $x: x \in \text{vertices-formula } F$ by *auto*
 from F have $(\text{vertices-formula } F) \subseteq (\text{vertices-conjunto-formulas } (\mathcal{F} \ G \ k))$
 $\vee \text{vertices-formula } F \subseteq (\text{vertices-conjunto-formulas } (\mathcal{G} \ G \ k))$
 using *vv1 vv2* by *blast*
 hence $x \in \text{vertices-conjunto-formulas } (\mathcal{F} \ G \ k) \vee x \in \text{vertices-conjunto-formulas } (\mathcal{G} \ G \ k)$
 using x by *auto*
 thus $x \in V[G]$
 using *vertices-algun[of G k] vertices-maximoC[of G k]* by *auto*
 next
 assume $x \in \text{vertices-conjunto-formulas } (\mathcal{H} \ G \ k)$
 hence
 $x \in (\bigcup F \in (\mathcal{H} \ G \ k). \text{vertices-formula } F)$
 by(*unfold vertices-conjunto-formulas-def*, *auto*)
 then obtain F where $F: F \in (\mathcal{H} \ G \ k)$ and $x: x \in \text{vertices-formula } F$
 by *auto*
 from F have $(\text{vertices-formula } F) \subseteq (\text{vertices-conjunto-formulas } (\mathcal{H} \ G \ k))$
 using *vv3* by *blast*
 hence $x \in \text{vertices-conjunto-formulas } (\mathcal{H} \ G \ k)$ using x by *auto*
 thus $x \in V[G]$ using *vertices-diferentesC[of G k]*
 by *auto*
 qed
 qed
 qed

lemma *vsf*:

assumes $G \subseteq H$

shows $\text{vertices-conjunto-formulas } G \subseteq \text{vertices-conjunto-formulas } H$

using *assms* by(*unfold vertices-conjunto-formulas-def*, *auto*)

lemma *vertices-subconjunto-formulas*:

assumes $S \subseteq (\mathcal{T} \ G \ k)$

shows $\text{vertices-conjunto-formulas } S \subseteq V[G]$

proof–

have $\text{vertices-conjunto-formulas } S \subseteq \text{vertices-conjunto-formulas } (\mathcal{T} \ G \ k)$

using *assms vsf* by *auto*

thus *?thesis* using *aa[of G]* by *auto*

qed

definition *subgrafo-aux* :: '*v* digrafo $\Rightarrow$ '*v* set $\Rightarrow$ '*v* digrafo **where**

subgrafo-aux $G \ V \equiv (V, E[G] \cap (V \times V))$

lemma *subgrafo-inducido*:

assumes *es-grafo* G **and** $S \subseteq (\mathcal{T} \ G \ k)$

shows *es-subgrafo-inducido* (*subgrafo-aux* G (*vertices-conjunto-formulas* S)) G

proof–

let $?V = \text{vertices-conjunto-formulas } S$

let $?H = (?V, E[G] \cap (?V \times ?V))$

have $1: E[?H] = E[G] \cap (?V \times ?V)$ **and** $2: V[?H] = ?V$ **by** *auto*

have $(V[?H] \subseteq V[G])$

using 2 *assms*(2) *vertices-subconjunto-formulas*[*of* $S \ G$]

by *auto*

moreover

have $E[?H] = (E[G] \cap ((V[?H]) \times (V[?H])))$ **using** $1 \ 2$ **by** *auto*

ultimately

have *es-subgrafo-inducido* $?H \ G$ **by**(*unfold es-subgrafo-inducido-def*, *auto*)

thus *?thesis* **by**(*unfold subgrafo-aux-def*, *auto*)

qed

lemma *finito-subgrafo*:

assumes *es-grafo* G **and** $S \subseteq (\mathcal{T} \ G \ k)$ **and** *finite* S

shows *grafo-finito* (*subgrafo-aux* G (*vertices-conjunto-formulas* S))

proof–

let $?V = \text{vertices-conjunto-formulas } S$

let $?H = (?V, E[G] \cap (?V \times ?V))$

have $1: E[?H] = E[G] \cap (?V \times ?V)$ **and** $2: V[?H] = ?V$ **by** *auto*

have $3: \text{finite } ?V$ **using** $\langle \text{finite } S \rangle$ *finito-vertices*

by(*unfold vertices-conjunto-formulas-def*, *auto*)

hence *finite* $(V[?H])$ **using** 2 **by** *auto*

moreover

have *finite* $(E[G] \cap (?V \times ?V))$ **using** 3 **by** *auto*

hence *finite* $(E[?H])$ **using** 1 **by** *auto*

ultimately have *grafo-finito* $?H$ **by**(*unfold grafo-finito-def*, *auto*)

thus *?thesis* **by**(*unfold subgrafo-aux-def*, *auto*)

qed

fun *interpretacion-grafo* :: $'v \text{ digrafo} \Rightarrow ('v \Rightarrow \text{nat}) \Rightarrow (('v \times \text{nat}) \Rightarrow v\text{-verdad})$

where

interpretacion-grafo $G \ f = (\lambda(v,i).(\text{if } v \in V[G] \wedge f(v) = i \text{ then } \text{Verdad} \text{ else } \text{Falso}))$

lemma *valor1*:

assumes $v \in V[G]$ **and** $f(v) \leq k$ **and** $F = \text{disyuncion-atomicas } v \ k$

shows *valor* (*interpretacion-grafo* $G \ f$) $F = \text{Verdad}$

```

proof–
  let ?i = f(v)
  have 0 ≤ ?i by auto
  {have v ∈ V[G] ⇒ 0 ≤ ?i ⇒ ?i ≤ k ⇒ F = disyuncion-atomicas v k ⇒
    valor (interpretacion-grafo G f) F = Verdad
  }
  proof(induct k arbitrary: F)
    case 0
    have ?i = 0 using 0 (2–3) by auto
    hence valor (interpretacion-grafo G f) (Atomo (v, 0)) = Verdad
    using ⟨v ∈ V[G]⟩ by auto
    thus ?case using 0 (4) by auto
  next
  case(Suc k)
  from Suc(1) Suc(2) Suc(3) Suc(4) Suc(5) show ?case
  proof(cases)
    assume (Suc k) = ?i
    hence valor (interpretacion-grafo G f) (Atomo (v, Suc k)) = Verdad
    using Suc(2) Suc(3) Suc(5) by auto
    hence
      valor (interpretacion-grafo G f) (Atomo (v, Suc k)
        ∨.disyuncion-atomicas v k) = Verdad
    using v-disyuncion-def by auto
    thus ?case using Suc(5) by auto
  next
  assume 1: (Suc k) ≠ ?i
  hence valor (interpretacion-grafo G f) (Atomo (v, Suc k)) = Falso
    using Suc(5) by auto
  moreover
  have ?i < (Suc k) using Suc(4) 1 by auto
  hence ?i ≤ k by auto
  hence valor (interpretacion-grafo G f) (disyuncion-atomicas v k) = Verdad
  using Suc(1) Suc(2) Suc(3) Suc(5) by auto
  thus ?case using Suc(5) v-disyuncion-def by auto
  qed
qed
}
thus ?thesis using assms by auto
qed

```

lemma valor-vertice:

```

  assumes valor (interpretacion-grafo G f) (Atomo (v, i)) = Verdad
  shows f(v)=i
proof(rule ccontr)

```

assume $f\ v \neq i$ **hence** $\text{valor}(\text{interpretacion-grafo } G\ f)\ (\text{Atomo } (v, i)) \neq \text{Verdad}$ **by** *auto*
hence $\text{valor}(\text{interpretacion-grafo } G\ f)\ (\text{Atomo } (v, i)) = \text{Falso}$
using $\text{no-verdad}[\text{of } \text{interpretacion-grafo } G\ f\ \text{Atomo } (v, i)]$ **by** *auto*
thus *False* **using** *assms* **by** *simp*
qed

lemma *valor2*:

assumes $i \neq j$ **and** $F = \neg.(\text{Atomo } (v, i) \wedge. \text{Atomo } (v, j))$
shows $\text{valor}(\text{interpretacion-grafo } G\ f)\ F = \text{Verdad}$
proof(*rule ccontr*)
assume $\text{valor}(\text{interpretacion-grafo } G\ f)\ F \neq \text{Verdad}$
hence $\text{valor}(\text{interpretacion-grafo } G\ f)\ (\neg.(\text{Atomo } (v, i) \wedge. \text{Atomo } (v, j))) \neq \text{Verdad}$
using *assms*(2) **by** *auto*
hence $\text{valor}(\text{interpretacion-grafo } G\ f)\ (\neg.(\text{Atomo } (v, i) \wedge. \text{Atomo } (v, j))) = \text{Falso}$ **using**
 $\text{no-verdad}[\text{of } \text{interpretacion-grafo } G\ f\ \neg.(\text{Atomo } (v, i) \wedge. \text{Atomo } (v, j))]$
by *auto*
hence $\text{valor}(\text{interpretacion-grafo } G\ f)\ ((\text{Atomo } (v, i) \wedge. \text{Atomo } (v, j))) = \text{Verdad}$
using *ValoresNegacion1*[*of* $\text{interpretacion-grafo } G\ f\ (\text{Atomo } (v, i) \wedge. \text{Atomo } (v, j))$] **by** *auto*
hence $\text{valor}(\text{interpretacion-grafo } G\ f)\ (\text{Atomo } (v, i)) = \text{Verdad}$ **and**
 $\text{valor}(\text{interpretacion-grafo } G\ f)\ (\text{Atomo } (v, j)) = \text{Verdad}$
using *ValoresConjuncion*[*of* $\text{interpretacion-grafo } G\ f\ \text{Atomo } (v, i)\ \text{Atomo } (v, j)$] **by** *auto*
hence $f(v)=i$ **and** $f(v)=j$ **using** *valor-vertice* **by** *auto*
hence $i=j$ **by** *auto*
thus *False* **using** *assms*(1) **by** *auto*
qed

lemma *valor3*:

assumes $f(u) \neq f(v)$ **and** $F = \neg.(\text{Atomo } (u, i) \wedge. \text{Atomo } (v, i))$
shows $\text{valor}(\text{interpretacion-grafo } G\ f)\ F = \text{Verdad}$
proof(*rule ccontr*)
assume $\text{valor}(\text{interpretacion-grafo } G\ f)\ F \neq \text{Verdad}$
hence
 $\text{valor}(\text{interpretacion-grafo } G\ f)\ (\neg.(\text{Atomo } (u, i) \wedge. \text{Atomo } (v, i))) \neq \text{Verdad}$
using *assms*(2) **by** *auto*
hence $\text{valor}(\text{interpretacion-grafo } G\ f)\ (\neg.(\text{Atomo } (u, i) \wedge. \text{Atomo } (v, i))) = \text{Falso}$
using

```

no-verdad[of interpretacion-grafo  $G f \neg.(Atomo(u, i) \wedge. Atomo(v, i))$ ]
  by auto
hence valor (interpretacion-grafo  $G f$ ) ( $(Atomo(u, i) \wedge. Atomo(v, i))$ ) =
Verdad
  using ValoresNegacion1[of interpretacion-grafo  $G f (Atomo(u, i) \wedge. Atomo$ 
( $v, i$ ))]
  by auto
hence valor (interpretacion-grafo  $G f$ ) ( $Atomo(u, i)$ ) = Verdad and
valor (interpretacion-grafo  $G f$ ) ( $Atomo(v, i)$ ) = Verdad
  using ValoresConjuncion[of interpretacion-grafo  $G f Atomo(u, i) Atomo(v,$ 
 $i)$ ]
  by auto
hence  $f(u)=i$  and  $f(v)=i$  using valor-vertice by auto
hence  $f(u)=f(v)$  by auto
thus False using assms(1) by auto
qed

```

theorem *coloramiento-satisfactible*:

```

assumes es-grafo  $G$  and  $S \subseteq (\mathcal{T} G k)$  and
coloramiento  $f k$  (subgrafo-aux  $G$  (vertices-conjunto-formulas  $S$ ))
shows satisfacible  $S$ 

```

proof–

```

let ?V = vertices-conjunto-formulas  $S$ 
let ?H = subgrafo-aux  $G$  ?V
have (interpretacion-grafo ?H  $f$ ) modelo  $S$ 
proof(unfold modelo-def)
  show  $\forall F \in S. \text{valor}(\text{interpretacion-grafo } ?H f) F = \text{Verdad}$ 
  proof
    fix  $F$  assume  $F \in S$ 
    show valor (interpretacion-grafo ?H  $f$ )  $F = \text{Verdad}$ 
    proof–
      have 1: vertices-formula  $F \subseteq ?V$ 
      proof
        fix  $v$ 
        assume  $v \in (\text{vertices-formula } F)$  thus  $v \in ?V$ 
        using  $\langle F \in S \rangle$  by(unfold vertices-conjunto-formulas-def, auto)
      qed
      have  $F \in (\mathcal{F} G k) \cup (\mathcal{G} G k) \cup (\mathcal{H} G k)$ 
      using  $\langle F \in S \rangle$  assms(2) by(unfold T-def, auto)
      hence  $F \in (\mathcal{F} G k) \vee F \in (\mathcal{G} G k) \vee F \in (\mathcal{H} G k)$  by auto
      thus ?thesis
    proof(rule disjE)
      assume  $F \in (\mathcal{F} G k)$ 

```

hence $\exists v \in V[G]. F = \text{disyuncion-atomicas } v \ k$ **by**(*unfold $\mathcal{F}$ -def, auto*)
then obtain v
where $v: v \in V[G]$ **and** $F: F = \text{disyuncion-atomicas } v \ k$
by *auto*
have $v \in ?V$ **using** F *vertices-disyuncion[of F] 1* **by** *auto*
hence $v \in V[?H]$ **by**(*unfold subgrafo-aux-def, auto*)
hence $f(v) \leq k$ **using** *coloramiento-def[of $f \ k \ ?H$] assms($\mathcal{I}$)* **by** *auto*
thus *?thesis* **using** F *valor1[OF $\langle v \in V[?H] \rangle$]* **by** *auto*
next
assume $F \in (\mathcal{G} \ G \ k) \vee F \in (\mathcal{H} \ G \ k)$
thus *?thesis*
proof(*rule disjE*)
assume $F \in (\mathcal{G} \ G \ k)$
hence $\exists v. \exists i. \exists j. F = \neg.(Atomo \ (v, i) \wedge. Atomo(v, j)) \wedge (i \neq j)$
by(*unfold $\mathcal{G}$ -def, auto*)
then obtain $v \ i \ j$
where $F = \neg.(Atomo \ (v, i) \wedge. Atomo(v, j))$ **and** $(i \neq j)$
by *auto*
thus *valor (interpretacion-grafo ?H f) F = Verdad*
using *valor2[OF $\langle i \neq j \rangle \langle F = \neg.(Atomo \ (v, i) \wedge. Atomo(v, j)) \rangle$]*
by *auto*
next
assume $F \in (\mathcal{H} \ G \ k)$
hence $\exists u. \exists v. \exists i. (F = \neg.(Atomo \ (u, i) \wedge. Atomo(v, i)) \wedge (u, v) \in E[G])$
by(*unfold $\mathcal{H}$ -def, auto*)
then obtain $u \ v \ i$
where $F: F = \neg.(Atomo \ (u, i) \wedge. Atomo(v, i))$ **and** $uv: (u, v) \in E[G]$
by *auto*
have *vertices-formula* $F = \{u, v\}$ **using** F **by** *auto*
hence $\{u, v\} \subseteq ?V$ **using** *1* **by** *auto*
hence $(u, v) \in E[?H]$ **using** uv **by**(*unfold subgrafo-aux-def, auto*)
hence $f(u) \neq f(v)$ **using** *coloramiento-def[of $f \ k \ ?H$] assms($\mathcal{I}$)*
by *auto*
show *?thesis*
using *valor3[OF $\langle f(u) \neq f(v) \rangle \langle F = \neg.(Atomo \ (u, i) \wedge. Atomo(v, i)) \rangle$]*
by *auto*
qed
qed
qed
qed
thus *satisfacible S* **by**(*unfold satisfacible-def, auto*)
qed

```

theorem coloramiento-satisfactible1:
  assumes coloramiento f k G
  shows satisfacible (T G k)
proof–
  let  $?S = (\mathcal{T} \ G \ k)$ 
  have  $\forall F \in ?S. \text{valor } (\text{interpretacion-grafo } G \ f) \ F = \text{Verdad}$ 
  proof
    fix  $F$  assume  $F \in ?S$ 
    show  $\text{valor } (\text{interpretacion-grafo } G \ f) \ F = \text{Verdad}$ 
    proof–
      have  $F \in (\mathcal{F} \ G \ k) \cup (\mathcal{G} \ G \ k) \cup (\mathcal{H} \ G \ k)$ 
      using  $\langle F \in ?S \rangle$  by (unfold T-def, auto)
      hence  $F \in (\mathcal{F} \ G \ k) \vee F \in (\mathcal{G} \ G \ k) \vee F \in (\mathcal{H} \ G \ k)$  by auto
      thus ?thesis
    proof(rule disjE)
      assume  $F \in \mathcal{F} \ G \ k$ 
      hence  $\exists v \in V[G]. \ F = \text{disyuncion-atomicas } v \ k$  by (unfold F-def, auto)
      then obtain  $v$ 
      where  $v: v \in V[G]$  and  $F: F = \text{disyuncion-atomicas } v \ k$  by auto
      have  $f(v) \leq k$ 
      using  $v$  and coloramiento-def[of f k G] assms by auto
      thus ?thesis using  $F \text{ valor1}[OF \ \langle v \in V[G] \rangle]$  by blast
    next
      assume  $F \in (\mathcal{G} \ G \ k) \vee F \in (\mathcal{H} \ G \ k)$ 
      thus ?thesis
    proof(rule disjE)
      assume  $F \in (\mathcal{G} \ G \ k)$ 
      hence  $\exists v. \exists i. \exists j. \ F = \neg.(\text{Atomo } (v, i) \wedge. \text{Atomo}(v, j)) \wedge (i \neq j)$ 
      by (unfold G-def, auto)
      then obtain  $v \ i \ j$ 
      where  $F = \neg.(\text{Atomo } (v, i) \wedge. \text{Atomo}(v, j))$  and  $(i \neq j)$  by auto
      thus  $\text{valor } (\text{interpretacion-grafo } G \ f) \ F = \text{Verdad}$ 
      using  $\text{valor2}[OF \ \langle i \neq j \rangle \ \langle F = \neg.(\text{Atomo } (v, i) \wedge. \text{Atomo}(v, j)) \rangle]$  by auto
    next
      assume  $F \in (\mathcal{H} \ G \ k)$ 
      hence  $\exists u. \exists v. \exists i. (F = \neg.(\text{Atomo } (u, i) \wedge. \text{Atomo}(v, i)) \wedge (u, v) \in E[G])$ 
      by (unfold H-def, auto)
      then obtain  $u \ v \ i$ 
      where  $F: F = \neg.(\text{Atomo } (u, i) \wedge. \text{Atomo}(v, i))$  and  $uv: (u, v) \in E[G]$ 
      by auto
      have  $f(u) \neq f(v)$ 

```

```

    using uv and coloramiento-def[of f k G] assms by auto
    show ?thesis
      using valor3[OF ⟨f(u) ≠ f(v)⟩ ⟨F = ¬.(Atomo (u, i) ∧. Atomo(v,i))⟩]
      by auto
    qed
  qed
  qed
  hence (interpretacion-grafo G f) modelo ?S by(unfold modelo-def)
  thus satisfacible ?S by( unfold satisfacible-def,auto)
qed

```

lemma *diag-nat*:

```

  shows  $\forall y z. \exists x. (y, z) = \text{diag } x$ 
  using enumeracion-nat.nat by(unfold enumeracion-def,auto)

```

lemma *EnumFormulasVertices*:

```

  assumes  $\exists g. \text{enumeracion } (g:: \text{nat} \Rightarrow 'v)$ 
  shows  $\exists f. \text{enumeracion } (f:: \text{nat} \Rightarrow ('v \times \text{nat})\text{formula})$ 

```

proof–

```

  from assms obtain g where e: enumeracion (g:: nat =>'v) by auto
  have enumeracion (( $\lambda m. (g(\text{fst}(\text{diag } m)), \text{snd}(\text{diag } m))$ ):: nat =>('v × nat))
  proof(unfold enumeracion-def)
    show  $\forall y::('v \times \text{nat}). \exists m. y = (g (\text{fst } (\text{diag } m)), \text{snd } (\text{diag } m))$ 

```

proof

```

  fix y::('v × nat)

```

```

  show  $\exists m. y = (g (\text{fst } (\text{diag } m)), \text{snd } (\text{diag } m))$ 

```

proof–

```

  have  $y = ((\text{fst } y), (\text{snd } y))$  by auto
  from e have  $\forall w::'v. \exists n. w = (g n)$  by(unfold enumeracion-def, auto)
  hence  $\exists n. (\text{fst } y) = (g n)$  by auto
  then obtain n where n:  $(\text{fst } y) = (g n)$  by auto
  have  $\exists m. (n, (\text{snd } y)) = \text{diag } m$  using diag-nat by auto
  hence  $\exists m. (n, (\text{snd } y)) = (\text{fst } (\text{diag } m), \text{snd } (\text{diag } m))$  by simp
  hence  $\exists m. ((\text{fst } y), (\text{snd } y)) = (g(\text{fst } (\text{diag } m)), \text{snd } (\text{diag } m))$ 
    using n by blast
  thus  $\exists m. y = (g (\text{fst } (\text{diag } m)), \text{snd } (\text{diag } m))$  by auto

```

qed

qed

qed

```

  thus  $\exists f. \text{enumeracion } (f:: \text{nat} \Rightarrow ('v \times \text{nat})\text{formula})$ 

```

```

    using EnumeracionFormulasP1 by auto

```

qed

```

fun coloramiento-grafo :: (('v × nat) ⇒ v-verdad) ⇒ nat ⇒ ('v ⇒ nat)
  where
  coloramiento-grafo I k = (λv.(THE i. (valor I (Atomo (v,i)) = Verdad) ∧ 0≤i
    ∧ i≤k))

```

lemma unicidad:

```

  assumes (valor I (Atomo (v, i)) = Verdad ∧ 0≤i ∧ i ≤ k)
  and ∀j. (0≤j ∧ j≤k ∧ i≠j) ⟶ (valor I (¬.(Atomo (v, i) ∧. Atomo(v,j))) =
    Verdad)
  shows ∀j. (0≤j ∧ j≤k ∧ i≠j) ⟶ valor I (Atomo (v, j)) = Falso
proof(rule allI, rule impI)
  fix j
  assume hip: 0≤j ∧ j≤k ∧ i≠j
  show valor I (Atomo (v, j)) = Falso
proof(rule ccontr)
  assume valor I (Atomo (v, j)) ≠ Falso
  hence valor I (Atomo (v, j)) = Verdad using CasosValor by blast
  hence 1: valor I (Atomo (v, i) ∧. Atomo(v,j)) = Verdad
    using assms(1) v-conjuncion-def by auto
  have valor I (¬.(Atomo (v, i) ∧. Atomo(v,j))) = Verdad
    using hip assms(2) by auto
  hence valor I (Atomo (v, i) ∧. Atomo(v,j)) = Falso
    using ValoresNegacion2 by blast
  thus False using 1 by auto
qed
qed

```

lemma existencia:

```

  assumes (valor I (Atomo (v, i)) = Verdad ∧ 0≤i ∧ i ≤ k)
  and ∀j. (0≤j ∧ j≤k ∧ i≠j) ⟶ valor I (Atomo (v, j)) = Falso
shows (∀x. (valor I (Atomo (v, x)) = Verdad ∧ 0≤x ∧ x ≤ k) ⟶ x = i)
proof(rule allI)
  fix x
  show valor I (Atomo (v, x)) = Verdad ∧ 0 ≤ x ∧ x ≤ k ⟶ x = i
proof(rule impI)
  assume hip: valor I (Atomo (v, x)) = Verdad ∧ 0≤x ∧ x ≤ k show x = i
proof(rule ccontr)
  assume 1: x ≠ i
  have 0≤x ∧ x ≤ k using hip by auto
  hence valor I (Atomo (v, x)) = Falso using 1 assms(2) by auto
  thus False using hip by auto
qed

```

qed
qed

lemma *exist-unicidad1*:

assumes $(\text{valor } I \text{ (Atomo } (v, i)) = \text{Verdad} \wedge 0 \leq i \wedge i \leq k)$
and $\forall j. (0 \leq j \wedge j \leq k \wedge i \neq j) \longrightarrow (\text{valor } I (\neg.(\text{Atomo } (v, i) \wedge. \text{Atomo}(v, j))) = \text{Verdad})$
shows $(\forall x. (\text{valor } I \text{ (Atomo } (v, x)) = \text{Verdad} \wedge 0 \leq x \wedge x \leq k) \longrightarrow x = i)$
using *assms unicidad[of I v i k] existencia[of I v i k]* **by** *blast*

lemma *exist-unicidad2*:

assumes $(\text{valor } I \text{ (Atomo } (v, i)) = \text{Verdad} \wedge 0 \leq i \wedge i \leq k)$ **and**
 $(\wedge x. (\text{valor } I \text{ (Atomo } (v, x)) = \text{Verdad} \wedge 0 \leq x \wedge x \leq k) \implies x = i)$
shows $(\text{THE } a. (\text{valor } I \text{ (Atomo } (v, a)) = \text{Verdad} \wedge 0 \leq a \wedge a \leq k)) = i$
using *assms* **by** *(rule the-equality)*

lemma *exist-unicidad*:

assumes $(\text{valor } I \text{ (Atomo } (v, i)) = \text{Verdad} \wedge 0 \leq i \wedge i \leq k)$ **and**
 $\forall j. (0 \leq j \wedge j \leq k \wedge i \neq j) \longrightarrow (\text{valor } I (\neg.(\text{Atomo } (v, i) \wedge. \text{Atomo}(v, j))) = \text{Verdad})$
shows $(\text{THE } a. (\text{valor } I \text{ (Atomo } (v, a)) = \text{Verdad} \wedge 0 \leq a \wedge a \leq k)) = i$
using *assms exist-unicidad1[of I v i k] exist-unicidad2[of I v i k]* **by** *blast*

lemma *elem-max-un-color*:

assumes $v \in V[G]$
shows $\forall i j. (0 \leq i \wedge 0 \leq j \wedge i \leq k \wedge j \leq k \wedge i \neq j) \longrightarrow (\neg.(\text{Atomo } (v, i) \wedge. \text{Atomo}(v, j)) \in (\mathcal{G} \ G \ k))$
proof(*rule allI*)+
fix $i j$
show $0 \leq i \wedge 0 \leq j \wedge i \leq k \wedge j \leq k \wedge i \neq j \longrightarrow \neg.(\text{Atomo } (v, i) \wedge. \text{Atomo}(v, j)) \in (\mathcal{G} \ G \ k)$
proof(*rule impI*)
assume $0 \leq i \wedge 0 \leq j \wedge i \leq k \wedge j \leq k \wedge i \neq j$
thus $\neg.(\text{Atomo } (v, i) \wedge. \text{Atomo}(v, j)) \in (\mathcal{G} \ G \ k)$
using $\langle v \in V[G] \rangle$ **by**(*unfold G-def, auto*)
qed
qed

lemma *elem-dif-colores*:

assumes $u \in V[G]$ **and** $v \in V[G]$ **and** $(u, v) \in E[G]$
shows $\forall i. (0 \leq i \wedge i \leq k) \longrightarrow (\neg.(\text{Atomo } (u, i) \wedge. \text{Atomo}(v, i)) \in (\mathcal{H} \ G \ k))$
proof(*rule allI*)
fix i

```

show  $0 \leq i \wedge i \leq k \longrightarrow (\neg.(Atomo(u, i) \wedge Atomo(v, i)) \in (\mathcal{H} \ G \ k))$ 
proof(rule impI)
  assume  $0 \leq i \wedge i \leq k$ 
  thus  $\neg.(Atomo(u, i) \wedge Atomo(v, i)) \in (\mathcal{H} \ G \ k)$ 
    using assms by(unfold H-def, auto)
qed
qed

```

lemma *valor-atomo*:

```

assumes (valor I (disyuncion-atomicas u k)) = Verdad
shows  $\exists i. (valor \ I \ (Atomo \ (u, i)) = Verdad) \wedge 0 \leq i \wedge i \leq k$ 
proof–
  have (valor I (disyuncion-atomicas u k)) = Verdad  $\implies$ 
 $\exists i. (valor \ I \ (Atomo \ (u, i)) = Verdad) \wedge 0 \leq i \wedge i \leq k$ 
proof(induct k)
  case(0)
  assume (valor I (disyuncion-atomicas u 0)) = Verdad
  thus  $\exists i. valor \ I \ (Atomo \ (u, i)) = Verdad \wedge 0 \leq i \wedge i \leq 0$  by auto
  next
  case(Suc k)
  from Suc(1) Suc(2) show ?case
  proof–
    have valor I (Atomo (u, (Suc k)))  $\vee$  (disyuncion-atomicas u k) = Verdad
      using Suc(2) by auto
    hence valor I (Atomo (u, (Suc k))) = Verdad  $\vee$ 
      (valor I (disyuncion-atomicas u k)) = Verdad
      using ValoresDisyuncion[of I (Atomo (u, (Suc k)))] by auto
    thus ?case
  proof(rule disjE)
    assume valor I (Atomo (u, (Suc k))) = Verdad
    thus ?case by(rule-tac x= Suc k in exI, auto)
    next
    assume valor I (disyuncion-atomicas u k) = Verdad
    thus ?case using Suc(1) by auto
  qed
qed
qed
thus ?thesis using assms by auto
qed

```

lemma *funcion-coloramiento*:

```

assumes  $u \in V[G]$  and I modelo (T G k)
shows  $\exists ! i. (valor \ I \ (Atomo \ (u, i)) = Verdad \wedge 0 \leq i \wedge i \leq k) \wedge coloramiento-grafo$ 

```

$I k u = i$

proof–

from $\langle u \in V[G] \rangle$

have $\text{disyuncion-atomicas } u \ k \in \mathcal{F} \ G \ k$ **by** $(\text{induct}, \text{unfold } \mathcal{F}\text{-def}, \text{auto})$

hence $\text{disyuncion-atomicas } u \ k \in \mathcal{T} \ G \ k$ **by** $(\text{unfold } \mathcal{T}\text{-def}, \text{auto})$

hence $(\text{valor } I \ (\text{disyuncion-atomicas } u \ k)) = \text{Verdad}$

using $\text{assms}(2)$ $\text{modelo-def}[\text{of } I \ \mathcal{T} \ G \ k]$ **by** auto

hence $\exists i. (\text{valor } I \ (\text{Atomo } (u, i)) = \text{Verdad} \wedge 0 \leq i \wedge i \leq k)$

using valor-atomo **by** auto

then obtain i **where** $i: (\text{valor } I \ (\text{Atomo } (u, i)) = \text{Verdad}) \wedge 0 \leq i \wedge i \leq k$

by auto

moreover

have $\forall i \ j. (0 \leq i \wedge 0 \leq j \wedge i \leq k \wedge j \leq k \wedge i \neq j) \longrightarrow$

$(\neg. (\text{Atomo } (u, i) \wedge \text{Atomo}(u, j)) \in (\mathcal{G} \ G \ k))$

using $\langle u \in V[G] \rangle$ $\text{elem-max-un-color}[\text{of } u]$ **by** auto

hence $\forall j. (0 \leq j \wedge j \leq k \wedge i \neq j) \longrightarrow (\neg. (\text{Atomo } (u, i) \wedge \text{Atomo}(u, j)) \in \mathcal{T} \ G \ k)$

using i **by** $(\text{unfold } \mathcal{T}\text{-def}, \text{auto})$

hence

$\forall j. (0 \leq j \wedge j \leq k \wedge i \neq j) \longrightarrow (\text{valor } I \ (\neg. (\text{Atomo } (u, i) \wedge \text{Atomo}(u, j)))) = \text{Verdad})$

using $\text{assms}(2)$ $\text{modelo-def}[\text{of } I \ \mathcal{T} \ G \ k]$ **by** blast

hence $(\text{THE } a. (\text{valor } I \ (\text{Atomo } (u, a)) = \text{Verdad} \wedge 0 \leq a \wedge a \leq k)) = i$

using i $\text{exist-unicidad}[\text{of } I \ u]$ **by** blast

hence $\text{coloramiento-grafo } I \ k \ u = i$ **by** auto

hence

$(\text{valor } I \ (\text{Atomo } (u, i)) = \text{Verdad} \wedge 0 \leq i \wedge i \leq k) \wedge$

$\text{coloramiento-grafo } I \ k \ u = i$

using i **by** auto

thus $?thesis$ **by** auto

qed

lemma $\mathcal{H}1$:

assumes $(\text{valor } I \ (\text{Atomo } (u, a)) = \text{Verdad} \wedge 0 \leq a \wedge a \leq k)$ **and**

$(\text{valor } I \ (\text{Atomo } (v, b)) = \text{Verdad} \wedge 0 \leq b \wedge b \leq k)$

and $\forall i. (0 \leq i \wedge i \leq k) \longrightarrow (\text{valor } I \ (\neg. (\text{Atomo } (u, i) \wedge \text{Atomo}(v, i)))) = \text{Verdad})$

shows $a \neq b$

proof $(\text{rule } ccontr)$

assume $\neg a \neq b$

hence $a = b$ **by** auto

hence $\text{valor } I \ (\text{Atomo } (u, a)) = \text{Verdad}$ **and** $\text{valor } I \ (\text{Atomo } (v, a)) = \text{Verdad}$

using assms **by** auto

hence $\text{valor } I \text{ (Atomo } (u, a) \wedge \text{Atomo}(v, a)) = \text{Verdad}$ **using** $v\text{-conjuncion-def}$
by *auto*
hence $\text{valor } I \text{ } (\neg.(\text{Atomo } (u, a) \wedge \text{Atomo}(v, a))) = \text{Falso}$ **using** $v\text{-negacion-def}$
by *auto*
moreover
have $0 \leq a \wedge a \leq k$ **using** $\text{assms}(1)$ **by** *auto*
hence $\text{valor } I \text{ } (\neg.(\text{Atomo } (u, a) \wedge \text{Atomo}(v, a))) = \text{Verdad}$ **using** $\text{assms}(3)$ **by**
auto
finally show *False* **by** *auto*
qed

lemma *colores-distintos*:

assumes $\text{es-grafo } G$ **and** $(u, v) \in E[G]$ **and** $I: I \text{ modelo } (\mathcal{T} \ G \ k)$
shows $\text{coloramiento-grafo } I \ k \ u \neq \text{coloramiento-grafo } I \ k \ v$
proof–
have $u \neq v$ **and** $u \in V[G]$ **and** $v \in V[G]$ **using** $\langle (u, v) \in E[G] \rangle \langle \text{es-grafo } G \rangle$
by $(\text{unfold } \text{es-grafo-def}, \text{auto})$
have $\exists! i. (\text{valor } I \text{ (Atomo } (u, i)) = \text{Verdad} \wedge 0 \leq i \wedge i \leq k) \wedge \text{coloramiento-grafo}$
 $I \ k \ u = i$
using $\text{funcion-coloramiento}[OF \ \langle u \in V[G] \rangle \ I]$ **by** *blast*
then obtain i **where** $i1: (\text{valor } I \text{ (Atomo } (u, i)) = \text{Verdad} \wedge 0 \leq i \wedge i \leq k)$ **and**
 $i2: \text{coloramiento-grafo } I \ k \ u = i$
by *auto*
have $\exists! j. (\text{valor } I \text{ (Atomo } (v, j)) = \text{Verdad} \wedge 0 \leq j \wedge j \leq k) \wedge \text{coloramiento-grafo}$
 $I \ k \ v = j$
using $\text{funcion-coloramiento}[OF \ \langle v \in V[G] \rangle \ I]$ **by** *blast*
then obtain j **where** $j1: (\text{valor } I \text{ (Atomo } (v, j)) = \text{Verdad} \wedge 0 \leq j \wedge j \leq k)$ **and**
 $j2: \text{coloramiento-grafo } I \ k \ v = j$ **by** *auto*
have $\forall i. (0 \leq i \wedge i \leq k) \longrightarrow (\neg.(\text{Atomo } (u, i) \wedge \text{Atomo}(v, i)) \in \mathcal{H} \ G \ k)$
using $\langle u \in V[G] \rangle \langle v \in V[G] \rangle \langle (u, v) \in E[G] \rangle$ **by** $(\text{unfold } \mathcal{H}\text{-def}, \text{auto})$
hence $\forall i. (0 \leq i \wedge i \leq k) \longrightarrow \neg.(\text{Atomo } (u, i) \wedge \text{Atomo}(v, i)) \in \mathcal{T} \ G \ k$
by $(\text{unfold } \mathcal{T}\text{-def}, \text{auto})$
hence $\forall i. (0 \leq i \wedge i \leq k) \longrightarrow (\text{valor } I \text{ } (\neg.(\text{Atomo } (u, i) \wedge \text{Atomo}(v, i))) =$
 $\text{Verdad})$
using $\text{assms}(2) \ I \text{ modelo-def}[of \ I \ \mathcal{T} \ G \ k]$ **by** *blast*
hence $i \neq j$ **using** $i1 \ j1 \ \mathcal{H}1[of \ I \ u \ i \ k \ v \ j]$ **by** *blast*
thus *?thesis* **using** $i2 \ j2$ **by** *auto*
qed

theorem *satisfactible-coloramiento*:

assumes $\text{es-grafo } G$ **and** $\text{satisfacible } (\mathcal{T} \ G \ k)$
shows $\text{colorable } G \ k$
proof $(\text{unfold } \text{colorable-def})$

```

show  $\exists f. \text{coloramiento } f \ k \ G$ 
proof–
  from assms(2) have  $\exists I. I \text{ modelo } (\mathcal{T} \ G \ k)$  by (unfold satisfacible-def)
  then obtain I where  $I: I \text{ modelo } (\mathcal{T} \ G \ k)$  by auto
  hence  $\text{coloramiento } (\text{coloramiento-grafo } I \ k) \ k \ G$ 
  proof(unfold coloramiento-def)
    show
       $(\forall u. u \in V[G] \longrightarrow (\text{coloramiento-grafo } I \ k \ u) \leq k) \wedge (\forall u \ v. (u, v) \in E[G] \longrightarrow \text{coloramiento-grafo } I \ k \ u \neq \text{coloramiento-grafo } I \ k \ v)$ 
    proof(rule conjI)
      show  $\forall u. u \in V[G] \longrightarrow \text{coloramiento-grafo } I \ k \ u \leq k$ 
      proof(rule allI, rule impI)
        fix u
        assume  $u \in V[G]$ 
        show  $\text{coloramiento-grafo } I \ k \ u \leq k$ 
        using funcion-coloramiento[OF  $\langle u \in V[G] \rangle I$ ] by blast
      qed
    next
      show
         $\forall u \ v. (u, v) \in E[G] \longrightarrow \text{coloramiento-grafo } I \ k \ u \neq \text{coloramiento-grafo } I \ k \ v$ 
      proof(rule allI, rule allI, rule impI)
        fix u v
        assume  $(u, v) \in E[G]$ 
        thus  $\text{coloramiento-grafo } I \ k \ u \neq \text{coloramiento-grafo } I \ k \ v$ 
        using colores-distintos[OF  $\langle \text{es-grafo } G \rangle \langle (u, v) \in E[G] \rangle I$ ] by blast
      qed
    qed
  qed
thus  $\exists f. \text{coloramiento } f \ k \ G$  by auto
qed

```

theorem *coloracion*:

```

assumes es-grafo ( $G::('vertices \ set) \times (('vertices \times 'vertices) \ set)$ )
and  $\exists g. \text{enumeracion } (g:: \text{nat} \Rightarrow 'vertices)$ 
and  $\forall H. (\text{es-subgrafo-inducido } H \ G \wedge \text{grafo-finito } H \longrightarrow \text{colorable } H \ k)$ 
shows  $\text{colorable } G \ k$ 
proof–
  have  $\forall S. S \subseteq (\mathcal{T} \ G \ k) \wedge (\text{finite } S) \longrightarrow \text{satisfacible } S$ 
  proof(rule allI, rule impI)
    fix S assume  $S \subseteq (\mathcal{T} \ G \ k) \wedge (\text{finite } S)$ 
    hence hip1:  $S \subseteq (\mathcal{T} \ G \ k)$  and hip2:  $\text{finite } S$  by auto

```

```

show satisfacible S
proof –
  let ?V = vertices-conjunto-formulas S
  let ?H = (?V, E[G] ∩ (?V × ?V))
  have es-subgrafo-inducido ?H G
    using assms(1) hip1 subgrafo-inducido[of G S k]
    by(unfold subgrafo-aux-def, auto)
  moreover
  have grafo-finito ?H
    using assms(1) hip1 hip2 finito-subgrafo[of G S k]
    by(unfold subgrafo-aux-def, auto)
  ultimately
  have colorable ?H k using assms(3) by auto
  hence  $\exists f. \text{coloramiento } f \text{ } k \text{ } ?H$  by(unfold colorable-def, auto)
  then obtain f where coloramiento f k ?H by auto
  thus satisfacible S using coloramiento-satisfactible[OF assms(1) hip1]
    by(unfold subgrafo-aux-def, auto)
  qed
qed
hence satisfacible (T G k) using
  TeoremaCompacidad1[OF EnumFormulasVertices[OF
    ( $\exists g. \text{enumeracion } (g :: \text{nat} \Rightarrow 'v)$ )], of (T G k)]
  by auto
  thus ?thesis using assms(1) satisfactible-coloramiento by blast
qed

end

```

9 Anexo 3. Teoría König

Se formaliza en Isabelle/HOL, utilizando el Teorema de Compacidad para la Lógica Proposicional, el Lema de König para árboles enumerables:

Todo árbol infinito enumerable de ramificación finita tiene una rama infinita.

```

type-synonym 'a rel = ('a × 'a) set

```

```

definition irrefl-en :: 'a set  $\Rightarrow$  'a rel  $\Rightarrow$  bool
where irrefl-en A r  $\equiv (\forall x \in A. (x, x) \notin r)$ 

```

```

definition trans-en :: 'a set  $\Rightarrow$  'a rel  $\Rightarrow$  bool
where trans-en A r  $\equiv$ 
  ( $\forall x \in A. \forall y \in A. \forall z \in A. (x, y) \in r \wedge (y, z) \in r \longrightarrow (x, z) \in r$ )

```

definition *total-en* :: 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *total-en* A r $\equiv (\forall x \in A. \forall y \in A. x \neq y \longrightarrow (x, y) \in r \vee (y, x) \in r)$

definition *minimo* :: 'a set $\Rightarrow$ 'a $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *minimo* A a r $\equiv (a \in A \wedge (\forall x \in A. x \neq a \longrightarrow (a, x) \in r))$

definition *predecesores-de* :: 'a set $\Rightarrow$ 'a $\Rightarrow$ 'a rel $\Rightarrow$ 'a set
where *predecesores-de* A a r $\equiv \{x \in A. (x, a) \in r\}$

definition *altura* :: 'a set $\Rightarrow$ 'a $\Rightarrow$ 'a rel $\Rightarrow$ nat
where *altura* A a r $\equiv \text{card } (\text{predecesores-de } A \ a \ r)$

definition *nivel* :: 'a set $\Rightarrow$ 'a rel $\Rightarrow$ nat $\Rightarrow$ 'a set
where *nivel* A r n $\equiv \{x \in A. \text{altura } A \ x \ r = n\}$

definition *sucesores-inmed-de* :: 'a set $\Rightarrow$ 'a $\Rightarrow$ 'a rel $\Rightarrow$ 'a set
where *sucesores-inmed-de* A a r $\equiv$
 $\{x \in A. (a, x) \in r \wedge \text{altura } A \ x \ r = (\text{altura } A \ a \ r) + 1\}$

definition *orden-parcial-estricto-en* :: 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *orden-parcial-estricto-en* A r $\equiv \text{irrefl-en } A \ r \wedge \text{trans-en } A \ r$

definition *orden-lineal-en* :: 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *orden-lineal-en* A r $\equiv \text{orden-parcial-estricto-en } A \ r \wedge \text{total-en } A \ r$

lemma *minimo-elemento*:

assumes *orden-parcial-estricto-en* A r **and** *minimo* A a r **and** $r = \{\}$

shows $A = \{a\}$

proof(*rule ccontr*)

assume *hip*: $A \neq \{a\}$ **show** False

proof(*cases*)

assume *hip1*: $A = \{\}$

have $a \in A$ **using** $\langle \text{minimo } A \ a \ r \rangle$ **by**(*unfold minimo-def, auto*)

thus False **using** *hip1* **by** *auto*

next

assume $A \neq \{\}$

hence $\exists x. x \neq a \wedge x \in A$ **using** *hip* **by** *auto*

then obtain x **where** $x \neq a \wedge x \in A$ **by** *auto*

hence $(a, x) \in r$ **using** $\langle \text{minimo } A \ a \ r \rangle$ **by**(*unfold minimo-def, auto*)

hence $r \neq \{\}$ **by** *auto*

thus False **using** $\langle r = \{\} \rangle$ **by** *auto*

qed

qed

lemma *unico-minimo*:

assumes *orden-parcial-estricto-en* A r **and** *minimo* A a r **and** *minimo* A b r
shows $a=b$

proof(*rule ccontr*)

assume *hip*: $a \neq b$

have $a \in A$ **and** $b \in A$ **using** *assms*(2-3) **by**(*unfold minimo-def, auto*)

show *False*

proof(*cases*)

assume $r = \{\}$

hence $A = \{a\} \wedge A = \{b\}$ **using** *assms*(1-3) *minimo-elemento*[*of* A r] **by**
auto

thus *False* **using** *hip* **by** *auto*

next

assume $r \neq \{\}$

hence $1: (a,b) \in r \wedge (b,a) \in r$ **using** *hip* *assms*(2-3)

by(*unfold minimo-def, auto*)

have *irr*: *irrefl-en* A r **and** *tran*: *trans-en* A r

using *assms*(1) **by**(*unfold orden-parcial-estricto-en-def, auto*)

have $(a,a) \in r$ **using** $\langle a \in A \rangle \langle b \in A \rangle 1$ *tran* **by**(*unfold trans-en-def, blast*)

thus *False* **using** $\langle a \in A \rangle$ *irr* **by**(*unfold irrefl-en-def, blast*)

qed

qed

lemma *predecesores-minimo1*:

assumes *minimo* A a r **and** *orden-parcial-estricto-en* A r

shows *predecesores-de* A a $r = \{\}$

proof(*rule ccontr*)

have *irr*: *irrefl-en* A r **and** *tran*: *trans-en* A r **using** *assms*(2)

by(*unfold orden-parcial-estricto-en-def, auto*)

assume $1: \text{predecesores-de } A \ a \ r \neq \{\}$ **show** *False*

proof—

have $\exists x \in A. (x,a) \in r$ **using** 1 **by**(*unfold predecesores-de-def, auto*)

then obtain x **where** $x \in A$ **and** $(x,a) \in r$ **by** *auto*

hence $x \neq a$ **using** *irr* **by** (*unfold irrefl-en-def, auto*)

hence $(a,x) \in r$ **using** $\langle x \in A \rangle \langle \text{minimo } A \ a \ r \rangle$ **by**(*unfold minimo-def, auto*)

have $a \in A$ **using** $\langle \text{minimo } A \ a \ r \rangle$ **by**(*unfold minimo-def, auto*)

hence $(a,a) \in r$ **using** $\langle (a,x) \in r \rangle \langle (x,a) \in r \rangle \langle x \in A \rangle$ *tran*

by(*unfold trans-en-def, blast*)

thus *False* **using** $\langle (a,a) \in r \rangle \langle a \in A \rangle$ *irr* *irrefl-en-def*

by (*unfold irrefl-en-def, auto*)

qed

qed

```

lemma predecesores-minimo2:
  assumes orden-parcial-estricto-en A r and minimo A a r
  shows  $\forall x \in A. (\text{predecesores-de } A \ x \ r = \{\}) \longleftrightarrow (x=a)$ 
proof
  fix x
  assume x  $\in$  A
  show  $(\text{predecesores-de } A \ x \ r = \{\}) \longleftrightarrow (x = a)$ 
proof-
  have 1:  $a \in A$  using  $\langle \text{minimo } A \ a \ r \rangle$  by(unfold minimo-def, auto)
  have 2:  $(\text{predecesores-de } A \ x \ r = \{\}) \longrightarrow (x=a)$ 
  proof(rule impI)
    assume h:  $\text{predecesores-de } A \ x \ r = \{\}$  show  $x=a$ 
    proof(rule ccontr)
      assume  $x \neq a$ 
      hence  $(a,x) \in r$  using  $\langle x \in A \rangle \langle \text{minimo } A \ a \ r \rangle$ 
        by(unfold minimo-def, auto)
      hence  $a \in \text{predecesores-de } A \ x \ r$ 
        using 1 by(unfold predecesores-de-def, auto)
      thus False using h by auto
    qed
  qed
  have 3:  $x=a \longrightarrow (\text{predecesores-de } A \ x \ r = \{\})$ 
  proof(rule impI)
    assume  $x=a$ 
    thus  $\text{predecesores-de } A \ x \ r = \{\}$ 
      using assms predecesores-minimo1[of A a] by auto
  qed
  show ?thesis using 2 3 by auto
qed

```

```

lemma altura-minimo:
  assumes orden-parcial-estricto-en A r and minimo A a r
  shows  $\text{altura } A \ a \ r = 0$ 
proof-
  have  $a \in A$  using  $\langle \text{minimo } A \ a \ r \rangle$  by(unfold minimo-def, auto)
  hence  $\text{predecesores-de } A \ a \ r = \{\}$ 
    using assms predecesores-minimo2[of A r] by auto
  thus  $\text{altura } A \ a \ r = 0$  by(unfold altura-def, auto)
qed

```

```

lemma nivel-cero:

```

assumes *orden-parcial-estricto-en* A r
and *minimo* A a r **and** $\forall x \in A. \text{finite } (\text{predecesores-de } A \ x \ r)$
shows $(\text{nivel } A \ r \ 0) = \{a\}$
proof–
have $\forall x \in A. (\text{card } (\text{predecesores-de } A \ x \ r) = 0) \longleftrightarrow (x=a)$
using *assms predecesores-minimo2*[of $A \ r \ a$] *card-eq-0-iff* **by** *auto*
hence $1: \forall x \in A. (\text{altura } A \ x \ r = 0) \longleftrightarrow (x=a)$
by(*unfold altura-def, auto*)
have $a \in A$ **using** $\langle \text{minimo } A \ a \ r \rangle$ **by**(*unfold minimo-def, auto*)
thus *?thesis* **using** *assms 1 nivel-def*[of $A \ r \ 0$] **by** *auto*
qed

lemma *minimo-predecesor*:
assumes *minimo* A a r
shows $\forall x \in A. x \neq a \longrightarrow a \in \text{predecesores-de } A \ x \ r$
proof
fix x
assume $x \in A$
show $x \neq a \longrightarrow a \in \text{predecesores-de } A \ x \ r$
proof(*rule impI*)
assume $x \neq a$
show $a \in \text{predecesores-de } A \ x \ r$
proof–
have $\langle a, x \rangle \in r$ **using** $\langle x \in A \rangle \langle x \neq a \rangle \langle \text{minimo } A \ a \ r \rangle$
by(*unfold minimo-def, auto*)
hence $a \in A$ **using** $\langle \text{minimo } A \ a \ r \rangle$ **by**(*unfold minimo-def, auto*)
thus $a \in \text{predecesores-de } A \ x \ r$ **using** $\langle \langle a, x \rangle \in r \rangle$
by(*unfold predecesores-de-def, auto*)
qed
qed
qed

lemma *contenencia-orden-estricto*:
assumes *orden-parcial-estricto-en* A r **and** $B \subseteq A$
shows *orden-parcial-estricto-en* B r
proof–
have *irrefl-en* A r **and** *trans-en* A r
using $\langle \text{orden-parcial-estricto-en } A \ r \rangle$
by(*unfold orden-parcial-estricto-en-def, auto*)
have $1: \text{irrefl-en } B \ r$
proof(*unfold irrefl-en-def*)
show $\forall x \in B. (x, x) \notin r$
proof

```

    fix x
    assume  $x \in B$ 
    hence  $x \in A$  using  $\langle B \subseteq A \rangle$  by auto
    thus  $(x, x) \notin r$  using  $\langle \text{irrefl-en } A \ r \rangle$ 
    by (unfold irrefl-en-def, auto)
  qed
qed
have 2: trans-en B r
proof (unfold trans-en-def)
  show  $\forall x \in B. \forall y \in B. \forall z \in B. (x, y) \in r \wedge (y, z) \in r \longrightarrow (x, z) \in r$ 
  proof
    fix x assume  $x \in B$ 
    show  $\forall y \in B. \forall z \in B. (x, y) \in r \wedge (y, z) \in r \longrightarrow (x, z) \in r$ 
    proof
      fix y assume  $y \in B$ 
      show  $\forall z \in B. (x, y) \in r \wedge (y, z) \in r \longrightarrow (x, z) \in r$ 
      proof
        fix z assume  $z \in B$ 
        show  $(x, y) \in r \wedge (y, z) \in r \longrightarrow (x, z) \in r$ 
        proof (rule impI)
          assume hip:  $(x, y) \in r \wedge (y, z) \in r$ 
          show  $(x, z) \in r$ 
          proof-
            have  $x \in A$  and  $y \in A$  and  $z \in A$  using  $\langle x \in B \rangle \langle y \in B \rangle \langle z \in B \rangle \langle B \subseteq A \rangle$ 
            by auto
            thus  $(x, z) \in r$  using hip  $\langle \text{trans-en } A \ r \rangle$  by (unfold trans-en-def, blast)
          qed
        qed
      qed
    qed
  qed
  thus orden-parcial-estricto-en B r
  using 1 2 by (unfold orden-parcial-estricto-en-def, auto)
qed

lemma contenencia-orden-total:
  assumes total-en A r and  $B \subseteq A$ 
  shows total-en B r
proof (unfold total-en-def)
  show  $\forall x \in B. \forall y \in B. x \neq y \longrightarrow (x, y) \in r \vee (y, x) \in r$ 
  proof
    fix x

```

```

assume  $x \in B$  show  $\forall y \in B. x \neq y \longrightarrow (x, y) \in r \vee (y, x) \in r$ 
proof
  fix  $y$ 
  assume  $y \in B$ 
  show  $x \neq y \longrightarrow (x, y) \in r \vee (y, x) \in r$ 
  proof(rule impI)
    assume  $x \neq y$ 
    show  $(x, y) \in r \vee (y, x) \in r$ 
    proof–
      have  $x \in A \wedge y \in A$  using  $\langle x \in B \rangle \langle y \in B \rangle \langle B \subseteq A \rangle$  by auto
      thus  $(x, y) \in r \vee (y, x) \in r$ 
      using  $\langle x \neq y \rangle \langle \text{total-en } A \ r \rangle$  by(unfold total-en-def, auto)
    qed
  qed
qed
qed
qed

```

```

definition maximo :: ' $a$  set  $\Rightarrow$  ' $a$   $\Rightarrow$  ' $a$  rel  $\Rightarrow$  bool
  where maximo  $A$   $a$   $r \equiv (a \in A \wedge (\forall x \in A. x \neq a \longrightarrow (x, a) \in r))$ 

```

lemma *maximo-orden-estricto*:

```

assumes orden-parcial-estricto-en  $A$   $r$  and  $A \neq \{\}$  and total-en  $A$   $r$ 
and finite  $A$ 
shows  $(\exists a. \text{maximo } A \ a \ r)$ 

```

proof–

```

have orden-parcial-estricto-en  $A$   $r \implies A \neq \{\} \implies \text{total-en } A \ r \implies \text{finite } A$ 
 $\implies (\exists a. \text{maximo } A \ a \ r)$  using assms(4)

```

proof(*induct* A *rule:finite-induct*)

case *empty*

then show *?case* **by** *auto*

next

case (*insert* x A)

show $(\exists a. \text{maximo } (\text{insert } x \ A) \ a \ r)$

proof(*cases* $A = \{\}$)

case *True*

hence *insert* x $A = \{x\}$ **by** *simp*

hence *maximo* (*insert* x A) x r **by**(*unfold maximo-def, auto*)

then show *?thesis* **by** *auto*

next

case *False*

assume $A \neq \{\}$

show $\exists a. \text{maximo } (\text{insert } x \ A) \ a \ r$

proof–

have 1: *orden-parcial-estricto-en* A r
using *insert*(4) *contenencia-orden-estricto* **by** *auto*
have 2: *total-en* A r **using** *insert*(6) *contenencia-orden-total* **by** *auto*
have $\exists a. \text{maximo } A \ a \ r$ **using** 1 $\langle A \neq \{\} \rangle$ *insert*(1) 2 *insert*(3) **by** *auto*
then obtain a **where** $a: \text{maximo } A \ a \ r$ **by** *auto*
hence $a \in A$ **and** $\forall y \in A. y \neq a \longrightarrow (y, a) \in r$ **by**(*unfold maximo-def*, *auto*)
have 3: $a \in (\text{insert } x \ A)$ **using** $\langle a \in A \rangle$ **by** *auto*
have 4: $a \neq x$ **using** $\langle a \in A \rangle$ **and** $\langle x \notin A \rangle$ **by** *auto*
have $x \in (\text{insert } x \ A)$ **by** *auto*
hence $(a, x) \in r \vee (x, a) \in r$ **using** 3 4 $\langle \text{total-en } (\text{insert } x \ A) \ r \rangle$
by(*unfold total-en-def*, *auto*)
thus $\exists a. \text{maximo } (\text{insert } x \ A) \ a \ r$
proof(*rule disjE*)
have *trans-en* $(\text{insert } x \ A) \ r$ **using** *insert*(4)
by(*unfold orden-parcial-estricto-en-def*, *auto*)
assume *casoa*: $(a, x) \in r$
have $\forall z \in (\text{insert } x \ A). z \neq x \longrightarrow (z, x) \in r$
proof
fix z
assume *hip1*: $z \in (\text{insert } x \ A)$
show $z \neq x \longrightarrow (z, x) \in r$
proof(*rule impI*)
assume $z \neq x$
hence *hip2*: $z \in A$ **using** $\langle z \in (\text{insert } x \ A) \rangle$ **by** *auto*
thus $(z, x) \in r$
proof(*cases*)
assume $z = a$
thus $(z, x) \in r$ **using** $\langle (a, x) \in r \rangle$ **by** *auto*
next
assume $z \neq a$
hence $(z, a) \in r$ **using** $\langle z \in A \rangle \langle \forall y \in A. y \neq a \longrightarrow (y, a) \in r \rangle$ **by** *auto*
have $a \in (\text{insert } x \ A)$ **and** $z \in (\text{insert } x \ A)$ **and** $x \in (\text{insert } x \ A)$
using $\langle a \in A \rangle \langle z \in A \rangle$ **by** *auto*
thus $(z, x) \in r$
using $\langle (z, a) \in r \rangle \langle (a, x) \in r \rangle \langle \text{trans-en } (\text{insert } x \ A) \ r \rangle$
by(*unfold trans-en-def*, *blast*)
qed
qed
qed
thus $\exists a. \text{maximo } (\text{insert } x \ A) \ a \ r$
using $\langle x \in (\text{insert } x \ A) \rangle$ **by**(*unfold maximo-def*, *auto*)
next

```

assume casob:  $(x, a) \in r$ 
have  $\forall z \in (\text{insert } x \ A). z \neq a \longrightarrow (z, a) \in r$ 
proof
  fix  $z$ 
  assume hip1:  $z \in (\text{insert } x \ A)$ 
  show  $z \neq a \longrightarrow (z, a) \in r$ 
  proof(rule impI)
    assume  $z \neq a$  show  $(z, a) \in r$ 
    proof-
      have  $z \in A \vee z = x$  using  $\langle z \in (\text{insert } x \ A) \rangle$  by auto
      thus  $(z, a) \in r$ 
      proof(rule disjE)
        assume  $z \in A$ 
        thus  $(z, a) \in r$ 
        using  $\langle z \neq a \rangle \langle \forall y \in A. y \neq a \longrightarrow (y, a) \in r \rangle$  by auto
      next
        assume  $z = x$ 
        thus  $(z, a) \in r$  using  $\langle (x, a) \in r \rangle$  by auto
      qed
    qed
  qed
thus  $\exists a. \text{maximo } (\text{insert } x \ A) \ a \ r$ 
  using  $\langle a \in (\text{insert } x \ A) \rangle$  by(unfold maximo-def, auto)
qed
qed
qed
qed
thus ?thesis using assms by auto
qed

```

lemma *finito-union*:

```

fixes  $S :: 'a \Rightarrow 'a \text{ set}$ 
assumes  $\forall x. \text{finite } (S \ x) \text{ and } \text{finite } A$ 
shows  $\text{finite } (\bigcup a \in A. (S \ a))$  using assms by auto

```

lemma *niveles-distintos*:

```

assumes  $k > 0$ 
shows  $(\text{nivel } A \ r \ n) \cap (\text{nivel } A \ r \ (n+k)) = \{\}$ 
proof(rule ccontr)
  assume  $\text{nivel } A \ r \ n \cap \text{nivel } A \ r \ (n+k) \neq \{\}$ 
  hence  $\exists x. x \in (\text{nivel } A \ r \ n) \cap \text{nivel } A \ r \ (n+k)$  by auto
  then obtain  $x$  where  $x \in (\text{nivel } A \ r \ n) \cap \text{nivel } A \ r \ (n+k)$  by auto

```

hence $x \in A \wedge \text{altura } A \ x \ r = n$ **and** $x \in A \wedge \text{altura } A \ x \ r = n+k$
by (*unfold nivel-def*, *auto*)
thus *False* **using** $\langle k > 0 \rangle$ **by** *auto*
qed

lemma *niveles-distintosa*:

assumes $n \neq m$
shows $(\text{nivel } A \ r \ n) \cap (\text{nivel } A \ r \ m) = \{\}$
proof–
have $n < m \vee m < n$ **using** *assms* **by** *auto*
thus *?thesis*
proof(*rule disjE*)
assume $n < m$
hence $\exists k. k > 0 \wedge m = n + k$ **by** *arith*
thus *?thesis* **using** *niveles-distintos[of - A r]* **by** *auto*
next
assume $m < n$
hence $\exists k. k > 0 \wedge n = m + k$ **by** *arith*
thus *?thesis* **using** *niveles-distintos[of - A r]* **by** *auto*
qed
qed

definition *arbol* :: $'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow \text{bool}$

where $\text{arbol } A \ r \equiv$
 $r \subseteq A \times A \wedge r \neq \{\} \wedge (\text{orden-parcial-estricto-en } A \ r) \wedge (\exists a. \text{minimo } A \ a \ r) \wedge$
 $(\forall a \in A. \text{finite } (\text{predecesores-de } A \ a \ r) \wedge (\text{total-en } (\text{predecesores-de } A \ a \ r) \ r))$

definition *arbol-finito* :: $'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow \text{bool}$

where
 $\text{arbol-finito } A \ r \equiv \text{arbol } A \ r \wedge \text{finite } A$

abbreviation *arbol-infinito* :: $'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow \text{bool}$

where
 $\text{arbol-infinito } A \ r \equiv \text{arbol } A \ r \wedge \neg \text{finite } A$

definition *arbol-enumerable* :: $'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow \text{bool}$ **where**

$\text{arbol-enumerable } A \ r \equiv \exists g. \text{enumeracion } (g :: \text{nat} \Rightarrow 'a)$

definition *ramificacion-finita* :: $'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow \text{bool}$

where $\text{ramificacion-finita } A \ r \equiv (\forall x \in A. \text{finite } (\text{sucesores-inmed-de } A \ x \ r))$

definition *sub-orden-lineal-en* :: $'a \text{ set} \Rightarrow 'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow \text{bool}$

where $\text{sub-orden-lineal-en } B \ A \ r \equiv B \subseteq A \wedge (\text{orden-parcial-estricto-en } A \ r) \wedge$

(total-en B r)

definition *rama* :: 'a set $\Rightarrow$ 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *rama* B A $r \equiv$
(sub-orden-lineal-en B A r) $\wedge$
($\forall C. B \subseteq C \wedge$ sub-orden-lineal-en C A $r \longrightarrow B = C$)

definition *rama-finita*:: 'a set $\Rightarrow$ 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *rama-finita* B A $r \equiv$ *rama* B A $r \wedge$ finite B

definition *rama-infinita*:: 'a set $\Rightarrow$ 'a set $\Rightarrow$ 'a rel $\Rightarrow$ bool
where *rama-infinita* B A $r \equiv$ *rama* B A $r \wedge \neg$ finite B

lemma *arbol*:

assumes *arbol* A r
shows
 $r \subseteq A \times A$ **and** $r \neq \{\}$
and orden-parcial-estricto-en A r
and $\exists a. \text{minimo } A$ a r
and ($\forall a \in A. \text{finite } (\text{predecesores-de } A$ a $r) \wedge (\text{total-en } (\text{predecesores-de } A$ a $r)$
 $r)$)
using $\langle \text{arbol } A$ $r \rangle$ **by**(unfold *arbol-def*, *auto*)

lemma *no-vacio*:

assumes *arbol* A r **shows** $A \neq \{\}$
proof–
have $\exists a. \text{minimo } A$ a r **using** $\langle \text{arbol } A$ $r \rangle$ *arbol*[of A r] **by** *auto*
hence $\exists a. a \in A$ **by**(unfold *minimo-def*, *auto*)
thus $A \neq \{\}$ **by** *auto*
qed

lemma *orden-estricto-pred*:

assumes *arbol* A r
shows $\forall x \in A. \text{orden-parcial-estricto-en } (\text{predecesores-de } A$ x $r)$ r
proof–
have *irrefl-en* A r **and** *trans-en* A r **using** $\langle \text{arbol } A$ $r \rangle$
by(unfold *arbol-def*, unfold *orden-parcial-estricto-en-def*, *auto*)
thus ?thesis
proof(unfold *orden-parcial-estricto-en-def*)
show $\forall x \in A. \text{irrefl-en } (\text{predecesores-de } A$ x $r)$ $r \wedge$
 $\text{trans-en } (\text{predecesores-de } A$ x $r)$ r
proof
fix x

```

assume  $x \in A$ 
show  $\text{irrefl-en } (\text{predecesores-de } A \ x \ r) \ r \wedge \text{trans-en } (\text{predecesores-de } A \ x \ r) \ r$ 
proof–
  have 1:  $\text{irrefl-en } (\text{predecesores-de } A \ x \ r) \ r$ 
  proof(unfold irrefl-en-def)
    show  $\forall y \in (\text{predecesores-de } A \ x \ r). (y, y) \notin r$ 
    proof
      fix  $y$ 
      assume  $y \in (\text{predecesores-de } A \ x \ r)$ 
      hence  $y \in A$  by(unfold predecesores-de-def, auto)
      thus  $(y, y) \notin r$  using  $\langle \text{irrefl-en } A \ r \rangle$  by(unfold irrefl-en-def, auto)
    qed
  qed
  have 2:  $\text{trans-en } (\text{predecesores-de } A \ x \ r) \ r$ 
  proof(unfold trans-en-def)
    let  $?B = (\text{predecesores-de } A \ x \ r)$ 
    show  $\forall w \in ?B. \forall y \in ?B. \forall z \in ?B. (w, y) \in r \wedge (y, z) \in r \longrightarrow (w, z) \in r$ 
    proof
      fix  $w$  assume  $w \in ?B$ 
      show  $\forall y \in ?B. \forall z \in ?B. (w, y) \in r \wedge (y, z) \in r \longrightarrow (w, z) \in r$ 
      proof
        fix  $y$  assume  $y \in ?B$ 
        show  $\forall z \in ?B. (w, y) \in r \wedge (y, z) \in r \longrightarrow (w, z) \in r$ 
        proof
          fix  $z$  assume  $z \in ?B$ 
          show  $(w, y) \in r \wedge (y, z) \in r \longrightarrow (w, z) \in r$ 
          proof(rule impI)
            assume  $\text{hip}: (w, y) \in r \wedge (y, z) \in r$ 
            show  $(w, z) \in r$ 
            proof–
              have  $w \in A$  and  $y \in A$  and  $z \in A$  using  $\langle w \in ?B \rangle \langle y \in ?B \rangle \langle z \in ?B \rangle$ 
              by(unfold predecesores-de-def, auto)
              thus  $(w, z) \in r$ 
              using  $\text{hip}$   $\langle \text{trans-en } A \ r \rangle$  by(unfold trans-en-def, blast)
            qed
          qed
        qed
      qed
    qed
  qed
show
   $\text{irrefl-en } (\text{predecesores-de } A \ x \ r) \ r \wedge \text{trans-en } (\text{predecesores-de } A \ x \ r) \ r$ 
using 1 2 by auto

```

qed
 qed
 qed
 qed

lemma *maximo-predecesores*:

assumes *arbol* A r **and** *minimo* A a r

shows $\forall x \in A. x \neq a \longrightarrow (\exists b. \text{maximo } (\text{predecesores-de } A \ x \ r) \ b \ r)$

proof

fix x

assume $x \in A$

show $x \neq a \longrightarrow (\exists b. \text{maximo } (\text{predecesores-de } A \ x \ r) \ b \ r)$

proof(*rule impI*)

assume $x \neq a$

show $(\exists b. \text{maximo } (\text{predecesores-de } A \ x \ r) \ b \ r)$

proof–

have 1: *orden-parcial-estricto-en* $(\text{predecesores-de } A \ x \ r) \ r$

using $\langle \text{arbol } A \ r \rangle \langle x \in A \rangle$ *orden-estricto-pred* **by** *auto*

have 2: *total-en* $(\text{predecesores-de } A \ x \ r) \ r$ **and**

3: *finite* $(\text{predecesores-de } A \ x \ r)$ **and** $r \subseteq A \times A$

using $\langle \text{arbol } A \ r \rangle \langle x \in A \rangle$ **by**(*unfold arbol-def, auto*)

have 4: $(\text{predecesores-de } A \ x \ r) \neq \{\}$

using $\langle r \subseteq A \times A \rangle \langle \text{minimo } A \ a \ r \rangle \langle x \in A \rangle \langle x \neq a \rangle$

minimo-predecesor[*of* A a] **by** *auto*

have 5: $A \neq \{\}$ **using** $\langle \text{arbol } A \ r \rangle$ *no-vacio* **by** *auto*

show $(\exists b. \text{maximo } (\text{predecesores-de } A \ x \ r) \ b \ r)$

using 1 2 3 4 5 *maximo-orden-estricto* **by** *auto*

qed

qed

qed

lemma *pertenece-conjunto*:

assumes *arbol* A r **and** *card* $(\text{predecesores-de } A \ x \ r) = n+1$

shows $x \in A$

proof–

have $r \subseteq A \times A$ **using** $\langle \text{arbol } A \ r \rangle$ **by**(*unfold arbol-def, auto*)

have $(\text{predecesores-de } A \ x \ r) \neq \{\}$ **using** *assms*(2) **by** *auto*

hence $\exists y \in A. (y, x) \in r$ **by** (*unfold predecesores-de-def, auto*)

thus $x \in A$ **using** $\langle r \subseteq A \times A \rangle$ **by** *auto*

qed

lemma *predecesor-inmediato*:

assumes *arbol* A r

and $\text{card}(\text{predecesores-de } A \ x \ r) = n+1$ **and**
 $\text{maximo}(\text{predecesores-de } A \ x \ r) \ b \ r$
shows $\text{altura } A \ b \ r = n$
proof–
have $\text{trans-en } A \ r$ **and** $r \subseteq A \times A$ **and** $\text{irrefl-en } A \ r$
using $\langle \text{arbol } A \ r \rangle$
by $(\text{unfold arbol-def}, \text{unfold orden-parcial-estricto-en-def}, \text{auto})$
have $x \in A$ **using** $\text{assms}(1) \text{ assms}(2) \text{ pertenece-conjunto}$ **by** auto
have $\text{orden-parcial-estricto-en}(\text{predecesores-de } A \ x \ r) \ r$
using $\langle x \in A \rangle \langle \text{arbol } A \ r \rangle \text{orden-estricto-pred}[\text{of } A \ r]$ **by** auto
hence $\text{irrefl-en}(\text{predecesores-de } A \ x \ r) \ r$ **and**
 $\text{trans-en}(\text{predecesores-de } A \ x \ r) \ r$
by $(\text{unfold orden-parcial-estricto-en-def}, \text{auto})$
have $b \in (\text{predecesores-de } A \ x \ r)$
using $\langle \text{maximo}(\text{predecesores-de } A \ x \ r) \ b \ r \rangle$ **by** $(\text{unfold maximo-def}, \text{auto})$
have $\text{total-en}(\text{predecesores-de } A \ x \ r)$
using $\langle x \in A \rangle \langle \text{arbol } A \ r \rangle$ **by** $(\text{unfold arbol-def}, \text{auto})$
have $\text{card}(\text{predecesores-de } A \ x \ r) > 0$ **using** $\text{assms}(2)$ **by** auto
hence $1: \text{finite}(\text{predecesores-de } A \ x \ r)$ **using** card-gt-0-iff **by** blast
have $2: b \in (\text{predecesores-de } A \ x \ r)$
using $\text{assms}(3)$ **by** $(\text{unfold maximo-def}, \text{auto})$
hence $\text{card}((\text{predecesores-de } A \ x \ r) - \{b\}) = n$
using $1 \ \langle \text{card}(\text{predecesores-de } A \ x \ r) = n+1 \rangle$
 $\text{card-Diff-singleton}[\text{of}(\text{predecesores-de } A \ x \ r) \ b]$ **by** auto
have $(\text{predecesores-de } A \ b \ r) = ((\text{predecesores-de } A \ x \ r) - \{b\})$
proof (rule equalityI)
show $(\text{predecesores-de } A \ b \ r) \subseteq (\text{predecesores-de } A \ x \ r - \{b\})$
proof
fix y
assume $y \in (\text{predecesores-de } A \ b \ r)$
hence $y \in A$ **and** $(y, b) \in r$ **by** $(\text{unfold predecesores-de-def}, \text{auto})$
hence $y \neq b$ **using** $\langle \text{irrefl-en } A \ r \rangle$ **by** $(\text{unfold irrefl-en-def}, \text{auto})$
have $(b, x) \in r$ **using** 2 **by** $(\text{unfold predecesores-de-def}, \text{auto})$
hence $b \in A$ **using** $\langle r \subseteq A \times A \rangle$ **by** auto
have $(y, x) \in r$ **using** $\langle x \in A \rangle \langle y \in A \rangle \langle b \in A \rangle \langle (y, b) \in r \rangle \langle (b, x) \in r \rangle \langle \text{trans-en } A \ r \rangle$
by $(\text{unfold trans-en-def}, \text{blast})$
show $y \in (\text{predecesores-de } A \ x \ r - \{b\})$
using $\langle y \in A \rangle \langle (y, x) \in r \rangle \langle y \neq b \rangle$ **by** $(\text{unfold predecesores-de-def}, \text{auto})$
qed
next
show $(\text{predecesores-de } A \ x \ r - \{b\}) \subseteq (\text{predecesores-de } A \ b \ r)$
proof
fix y

assume $hip: y \in (\text{predecesores-de } A \ x \ r - \{b\})$
hence $y \neq b$ **and** $y \in A$ **by** $(\text{unfold predecesores-de-def}, \text{auto})$
have $(y, b) \in r$ **using** $hip \ \langle \text{maximo } (\text{predecesores-de } A \ x \ r) \ b \ r \rangle$
by $(\text{unfold maximo-def}, \text{auto})$
thus $y \in (\text{predecesores-de } A \ b \ r)$ **using** $\langle y \in A \rangle$
by $(\text{unfold predecesores-de-def}, \text{auto})$
qed
qed
hence $3: \text{card } (\text{predecesores-de } A \ b \ r) = \text{card } (\text{predecesores-de } A \ x \ r - \{b\})$
by auto
have $\text{finite } (\text{predecesores-de } A \ x \ r)$ **using** $\langle x \in A \rangle \ \langle \text{arbol } A \ r \rangle$ **by** $(\text{unfold arbol-def}, \text{auto})$
hence $\text{card } (\text{predecesores-de } A \ x \ r - \{b\}) = \text{card } (\text{predecesores-de } A \ x \ r) - 1$
using $2 \ \text{card-Suc-Diff1}$ **by** auto
hence $\text{card } (\text{predecesores-de } A \ b \ r) = n$
using $3 \ \langle \text{card } (\text{predecesores-de } A \ x \ r) = n + 1 \rangle$ **by** auto
thus $\text{altura } A \ b \ r = n$ **by** $(\text{unfold altura-def}, \text{auto})$
qed

lemma *altura*:

assumes $\text{arbol } A \ r$ **and** $\text{altura } A \ x \ r = n + 1$
shows $\exists y. (y, x) \in r \wedge \text{altura } A \ y \ r = n$
proof –
have $1: \text{card } (\text{predecesores-de } A \ x \ r) = n + 1$
using $\text{assms}(2)$ **by** $(\text{unfold altura-def}, \text{auto})$
have $\exists a. \text{minimo } A \ a \ r$ **using** $\langle \text{arbol } A \ r \rangle$ **by** $(\text{unfold arbol-def}, \text{auto})$
then obtain a **where** $a: \text{minimo } A \ a \ r$ **by** auto
have $\text{orden-parcial-estricto-en } A \ r$ **using** $\langle \text{arbol } A \ r \rangle \ \text{arbol}[\text{of } A \ r]$ **by** auto
hence $\text{altura } A \ a \ r = 0$ **using** $a \ \text{altura-minimo}[\text{of } A \ r]$ **by** auto
hence $x \neq a$ **using** $\text{assms}(2)$ **by** auto
have $x \in A$ **using** $\langle \text{arbol } A \ r \rangle \ 1 \ \text{pertenece-conjunto}$ **by** auto
hence $(\exists b. \text{maximo } (\text{predecesores-de } A \ x \ r) \ b \ r)$
using $\langle x \neq a \rangle \ \langle \text{arbol } A \ r \rangle \ a \ \text{maximo-predecesores}[\text{of } A \ r \ a]$ **by** auto
then obtain b **where** $b: (\text{maximo } (\text{predecesores-de } A \ x \ r) \ b \ r)$ **by** auto
hence $(b, x) \in r$ **by** $(\text{unfold maximo-def}, \text{unfold predecesores-de-def}, \text{auto})$
thus $\exists y. (y, x) \in r \wedge \text{altura } A \ y \ r = n$
using $\langle \text{arbol } A \ r \rangle \ 1 \ b \ \text{predecesor-inmediato}[\text{of } A \ r]$ **by** auto
qed

lemma *nivel*:

assumes $\text{arbol } A \ r$ **and** $x \in (\text{nivel } A \ r \ (n + 1))$
shows $\exists y. (y, x) \in r \wedge y \in (\text{nivel } A \ r \ n)$
proof –
have $\text{altura } A \ x \ r = n + 1$

using $\langle x \in (\text{nivel } A \ r \ (n+1)) \rangle$ **by** $(\text{unfold nivel-def}, \text{auto})$
hence $\exists y. (y, x) \in r \wedge \text{altura } A \ y \ r = n$
using $\langle \text{arbol } A \ r \rangle$ **altura** $[\text{of } A \ r]$ **by** auto
then obtain y **where** $y: (y, x) \in r \wedge \text{altura } A \ y \ r = n$ **by** auto
have $r \subseteq A \times A$ **using** $\langle \text{arbol } A \ r \rangle$ **by** $(\text{unfold arbol-def}, \text{auto})$
hence $y \in A$ **using** y **by** auto
hence $(y, x) \in r \wedge y \in (\text{nivel } A \ r \ n)$ **using** y **by** $(\text{unfold nivel-def}, \text{auto})$
thus $?thesis$ **by** auto
qed

primrec $\text{nivel1} :: 'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow \text{nat} \Rightarrow 'a \text{ set}$ **where**
 $\text{nivel1 } A \ r \ 0 = \{a. (\text{minimo } A \ a \ r)\}$
 $| \text{nivel1 } A \ r \ (\text{Suc } n) = (\bigcup a \in (\text{nivel1 } A \ r \ n). \text{sucesores-inmed-de } A \ a \ r)$

lemma nivel1-cero :

assumes $\text{orden-parcial-estricto-en } A \ r$ **and** $\text{minimo } A \ a \ r$
shows $(\text{nivel1 } A \ r \ 0) = \{a\}$

proof–

have $a \in (\text{nivel1 } A \ r \ 0)$ **using** $\langle \text{minimo } A \ a \ r \rangle$ **by** auto

hence $1: \{a\} \subseteq (\text{nivel1 } A \ r \ 0)$ **by** auto

have $2: (\text{nivel1 } A \ r \ 0) \subseteq \{a\}$

proof

{fix x

assume $x \in (\text{nivel1 } A \ r \ 0)$

hence $\text{minimo } A \ x \ r$ **by** auto

hence $x = a$ **using** $\text{assms unico-minimo}[\text{of } A \ r]$ **by** auto

thus $x \in \{a\}$ **by** auto }

qed

thus $(\text{nivel1 } A \ r \ 0) = \{a\}$ **using** $1 \ 2$ **by** auto

qed

lemma altura-nivel :

assumes $\text{orden-parcial-estricto-en } A \ r$ **and** $\text{minimo } A \ a \ r$

and $x \in \text{nivel1 } A \ r \ n$

shows $\text{altura } A \ x \ r = n$

proof–

have

$\llbracket \text{orden-parcial-estricto-en } A \ r; \text{minimo } A \ a \ r; x \in \text{nivel1 } A \ r \ n \rrbracket \implies$
 $\text{altura } A \ x \ r = n$

proof $(\text{induct } n \text{ arbitrary: } x)$

case 0

then show $\text{altura } A \ x \ r = 0$

proof–

```

    have minimo A x r using ⟨x ∈ nivel1 A r 0⟩ by auto
    thus altura A x r = 0
      using ⟨orden-parcial-estricto-en A r⟩ altura-minimo[of A r]
      by auto
  qed
next
case (Suc n)
then show ?case
proof-
  have x ∈ (⋃ a ∈ (nivel1 A r n). (sucesores-inmed-de A a r))
    using Suc(4) by auto
  then obtain a
    where hip1: a ∈ (nivel1 A r n) and hip2: x ∈ (sucesores-inmed-de A a r)
    by auto
  hence 1: altura A a r = n using Suc(1-3) by auto
  have altura A x r = (altura A a r)+1
    using hip2 by (unfold sucesores-inmed-de-def, auto)
  thus altura A x r = Suc n using 1 by auto
qed
qed
thus ?thesis using assms by auto
qed

lemma igual-niveles:
  assumes arbol A r
  shows nivel1 A r n = nivel A r n
proof(induct n)
  have 1: orden-parcial-estricto-en A r and
    2: ∀ x ∈ A. finite (predecesores-de A x r)
    using ⟨arbol A r⟩ arbol[of A r] by auto
  have ∃ a. minimo A a r using ⟨arbol A r⟩ by (unfold arbol-def, auto)
  then obtain a where a: minimo A a r by auto
  case 0
  then show nivel1 A r 0 = nivel A r 0
  proof-
    have nivel1 A r 0 = {a} using 1 a nivel1-cero[of A r] by auto
    moreover
    have nivel A r 0 = {a} using 1 2 a nivel-cero[of A r] by auto
    ultimately
    show nivel1 A r 0 = nivel A r 0 by auto
  qed
next
case (Suc n)

```

```

assume  $nivel1\ A\ r\ n = nivel\ A\ r\ n$ 
show  $nivel1\ A\ r\ (Suc\ n) = nivel\ A\ r\ (Suc\ n)$ 
proof(rule equalityI)
  show  $nivel1\ A\ r\ (Suc\ n) \subseteq nivel\ A\ r\ (Suc\ n)$ 
  proof(rule subsetI)
    fix  $x$ 
    assume  $hip$ :  $x \in nivel1\ A\ r\ (Suc\ n)$  show  $x \in nivel\ A\ r\ (Suc\ n)$ 
    proof–
      have
         $nivel1\ A\ r\ (Suc\ n) = (\bigcup a \in (nivel1\ A\ r\ n). (sucesores-inmed-de\ A\ a\ r))$ 
        by simp
      hence  $x \in (\bigcup a \in (nivel1\ A\ r\ n). (sucesores-inmed-de\ A\ a\ r))$ 
        using  $hip$  by auto
      then obtain  $a$  where  $hip1$ :  $a \in (nivel1\ A\ r\ n)$  and
         $hip2$ :  $x \in (sucesores-inmed-de\ A\ a\ r)$  by auto
      have  $(a,x) \in r \wedge altura\ A\ x\ r = (altura\ A\ a\ r) + 1$ 
        using  $hip2$  by(unfold sucesores-inmed-de-def, auto)
      moreover
        have  $\exists b. minimo\ A\ b\ r$  using  $\langle arbol\ A\ r \rangle$  by(unfold arbol-def, auto)
      then obtain  $b$  where  $b$ :  $minimo\ A\ b\ r$  by auto
      have  $1$ :  $r \subseteq A \times A$  and  $orden-parcial-estricto-en\ A\ r$ 
        using  $\langle arbol\ A\ r \rangle$  by(unfold arbol-def, auto)
      hence  $altura\ A\ a\ r = n$  using  $b\ hip1\ altura-nivel[of\ A\ r]$  by auto
      ultimately
        have  $(a,x) \in r \wedge altura\ A\ x\ r = n + 1$  by auto
        hence  $x \in A \wedge altura\ A\ x\ r = n + 1$  using  $\langle r \subseteq A \times A \rangle$  by auto
        thus  $x \in nivel\ A\ r\ (Suc\ n)$  by(unfold nivel-def, auto)
      qed
    qed
  next
    show  $nivel\ A\ r\ (Suc\ n) \subseteq nivel1\ A\ r\ (Suc\ n)$ 
    proof(rule subsetI)
      fix  $x$ 
      assume  $hip$ :  $x \in nivel\ A\ r\ (Suc\ n)$  show  $x \in nivel1\ A\ r\ (Suc\ n)$ 
      proof–
        have  $1$ :  $x \in A \wedge altura\ A\ x\ r = n + 1$  using  $hip$  by(unfold nivel-def, auto)
        hence  $\exists y. (y,x) \in r \wedge altura\ A\ y\ r = n$ 
        using assms altura[of A r] by auto
        then obtain  $y$  where  $y1$ :  $(y,x) \in r$  and  $y2$ :  $altura\ A\ y\ r = n$  by auto
        hence  $x \in (sucesores-inmed-de\ A\ y\ r)$ 
          using  $1$  by(unfold sucesores-inmed-de-def, auto)
        moreover
          have  $r \subseteq A \times A$  using  $\langle arbol\ A\ r \rangle$  by(unfold arbol-def, auto)

```

```

    have  $y \in A$  using  $y1$   $\langle r \subseteq A \times A \rangle$  by auto
    hence  $y \in \text{nivel } A \ r \ n$  using  $y2$  by (unfold nivel-def, auto)
    hence  $y \in \text{nivel1 } A \ r \ n$  using Suc by auto
    ultimately
    show  $x \in \text{nivel1 } A \ r \ (\text{Suc } n)$  by auto
  qed
qed
qed
qed
lemma pertenece-nivel:
  assumes  $x \in \text{nivel1 } A \ r \ n$ 
  shows  $x \in A$ 
proof-
  have  $x \in \text{nivel1 } A \ r \ n \implies x \in A$ 
  proof(induct n)
    case 0
    show  $x \in A$  using  $\langle x \in \text{nivel1 } A \ r \ 0 \rangle$  minimo-def[of  $A \ x \ r$ ] by auto
  next
    case (Suc n)
    then show  $x \in A$ 
    proof-
      have  $\exists a \in (\text{nivel1 } A \ r \ n). x \in \text{sucesores-inmed-de } A \ a \ r$ 
      using  $\langle x \in \text{nivel1 } A \ r \ (\text{Suc } n) \rangle$  by auto
      then obtain a where  $a1: a \in (\text{nivel1 } A \ r \ n)$  and
         $a2: x \in \text{sucesores-inmed-de } A \ a \ r$  by auto
      show  $x \in A$  using  $a2$  sucesores-inmed-de-def[of  $A \ a \ r$ ] by auto
    qed
  qed
  thus  $x \in A$  using assms by auto
qed

```

```

lemma finito-nivel1a:
  assumes  $\forall x \in A. \text{finite } (\text{sucesores-inmed-de } A \ x \ r)$  and  $\text{finite } (\text{nivel1 } A \ r \ n)$ 
  shows  $\text{finite } (\bigcup a \in (\text{nivel1 } A \ r \ n). \text{sucesores-inmed-de } A \ a \ r)$ 
proof
  show  $\text{finite } (\text{nivel1 } A \ r \ n)$  using assms(2) by simp
next
  fix x
  assume hip:  $x \in \text{nivel1 } A \ r \ n$  show  $\text{finite } (\text{sucesores-inmed-de } A \ x \ r)$ 
  proof-
    have  $x \in A$  using hip pertenece-nivel[of  $x \ A \ r$ ] by auto
    thus  $\text{finite } (\text{sucesores-inmed-de } A \ x \ r)$  using assms(1) by auto
  qed

```

qed

qed

lemma *finito-nivel1*:

assumes *finite* (*nivel1* *A* *r* 0) and *ramificacion-finita* *A* *r*

shows *finite* (*nivel1* *A* *r* *n*)

proof(induct *n*)

case 0

show *finite* (*nivel1* *A* *r* 0) using *assms* by auto

next

case (*Suc* *n*)

then show ?case

proof –

have 1: $\forall x \in A. \text{finite} (\text{sucesores-inmed-de } A \ x \ r)$

using *assms* by (unfold *ramificacion-finita-def*, auto)

hence *finite* ($\bigcup a \in (\text{nivel1 } A \ r \ n). \text{sucesores-inmed-de } A \ a \ r$)

using *Suc*(1) *finito-nivel1a*[of *A* *r*] by auto

thus *finite* (*nivel1* *A* *r* (*Suc* *n*)) by auto

qed

qed

lemma *finito-nivel*:

assumes *arbol* *A* *r* and *ramificacion-finita* *A* *r*

shows *finite* (*nivel* *A* *r* *n*)

proof–

have 1: *orden-parcial-estricto-en* *A* *r* using $\langle \text{arbol } A \ r \rangle$ *arbol*[of *A* *r*] by auto

have $\exists a. \text{minimo } A \ a \ r$ using $\langle \text{arbol } A \ r \rangle$ *arbol*[of *A* *r*] by auto

then obtain *a* where *minimo* *A* *a* *r* by auto

hence *finite* (*nivel1* *A* *r* 0)

using 1 *nivel1-cero*[of *A* *r*] by auto

hence *finite* (*nivel1* *A* *r* *n*)

using $\langle \text{ramificacion-finita } A \ r \rangle$ *finito-nivel1*[of *A* *r*] by auto

thus ?thesis using $\langle \text{arbol } A \ r \rangle$ *igual-niveles*[of *A* *r* *n*] by auto

qed

lemma *finito-nivel-a*:

assumes *arbol* *A* *r* and $\forall n. \text{finite} (\text{nivel } A \ r \ n)$

shows *ramificacion-finita* *A* *r*

proof(unfold *ramificacion-finita-def*)

show $\forall x \in A. \text{finite} (\text{sucesores-inmed-de } A \ x \ r)$

proof

fix *x*

assume $x \in A$

```

show finite (sucesores-inmed-de  $A$   $x$   $r$ ) using ramificacion-finita-def
proof–
  let  $?n = (\text{altura } A \ x \ r)$ 
  have (sucesores-inmed-de  $A$   $x$   $r$ )  $\subseteq$  (nivel  $A$   $r$  ( $?n+1$ ))
    using sucesores-inmed-de-def[of  $A$   $x$   $r$ ] nivel-def[of  $A$   $r$   $?n+1$ ] by auto
  thus finite (sucesores-inmed-de  $A$   $x$   $r$ ) using assms(2) by(simp add: finite-subset)

qed
qed
qed

lemma vacio-predec:
  assumes  $\forall x \in A. (x, y) \notin r$ 
  shows predecesores-de  $A$   $y$   $r = \{\}$ 
  using assms by(unfold predecesores-de-def, auto)

lemma nivel-elemento:
   $\forall x \in A. \exists n. x \in \text{nivel } A \ r \ n$ 
proof
  fix  $x$ 
  assume hip:  $x \in A$  show  $\exists n. x \in \text{nivel } A \ r \ n$ 
  proof–
    let  $?n = \text{altura } A \ x \ r$ 
    have  $x \in \text{nivel } A \ r \ ?n$  using  $\langle x \in A \rangle$  by (unfold nivel-def, auto)
    thus  $\exists n. x \in \text{nivel } A \ r \ n$  by auto
  qed
qed

lemma union-niveles:
  shows  $A = (\bigcup n. \text{nivel } A \ r \ n)$ 
proof(rule equalityI)
  show  $A \subseteq (\bigcup n. \text{nivel } A \ r \ n)$ 
  proof(rule subsetI)
    fix  $x$ 
    assume hip:  $x \in A$  show  $x \in (\bigcup n. \text{nivel } A \ r \ n)$ 
    proof–
      have  $\exists n. x \in \text{nivel } A \ r \ n$ 
        using hip nivel-elemento[of  $A$ ] by auto
      then obtain  $n$  where  $x \in \text{nivel } A \ r \ n$  by auto
      thus ?thesis by auto
    qed
  qed
qed
next

```

```

show  $(\bigcup n. \text{nivel } A \ r \ n) \subseteq A$ 
proof(rule subsetI)
  fix  $x$ 
  assume  $hip: x \in (\bigcup n. \text{nivel } A \ r \ n)$  show  $x \in A$ 
  proof–
    obtain  $n$  where  $x \in \text{nivel } A \ r \ n$  using  $hip$  by auto
    thus  $x \in A$  by(unfold nivel-def, auto)
  qed
qed
qed

lemma camino1:
  assumes  $\text{arbol } A \ r$  and  $x \in (\text{nivel } A \ r \ (n+1))$ 
  shows  $\forall k. (0 \leq k \wedge k \leq n) \longrightarrow (\exists y. (y, x) \in r \wedge y \in (\text{nivel } A \ r \ k))$ 
proof–
  have  $\text{arbol } A \ r \implies x \in (\text{nivel } A \ r \ (n+1)) \implies$ 
 $\forall k. (0 \leq k \wedge k \leq n) \longrightarrow (\exists y. (y, x) \in r \wedge y \in (\text{nivel } A \ r \ k))$ 
  proof(induction  $n$  arbitrary:  $x$ )
    have  $r \subseteq A \times A$  and  $1: \text{orden-parcial-estricto-en } A \ r$ 
    and  $\exists a. \text{minimo } A \ a \ r$ 
    and  $2: \forall x \in A. \text{finite } (\text{predecesores-de } A \ x \ r)$ 
    using  $\langle \text{arbol } A \ r \rangle \text{ arbol}[of \ A \ r]$  by auto
    case  $0$ 
    show  $\forall k. 0 \leq k \wedge k \leq 0 \longrightarrow (\exists y. (y, x) \in r \wedge y \in \text{nivel } A \ r \ k)$ 
    proof
      fix  $k$ 
      show  $0 \leq k \wedge k \leq 0 \longrightarrow (\exists y. (y, x) \in r \wedge y \in \text{nivel } A \ r \ k)$ 
      proof(rule impI)
        assume  $hip: 0 \leq k \wedge k \leq 0$ 
        show  $(\exists y. (y, x) \in r \wedge y \in \text{nivel } A \ r \ k)$ 
        proof–
          have  $k=0$  using  $hip$  by auto
          thus  $(\exists y. (y, x) \in r \wedge y \in \text{nivel } A \ r \ k)$ 
          using  $\langle \text{arbol } A \ r \rangle \langle x \in (\text{nivel } A \ r \ (0 + 1)) \rangle \text{ nivel}[of \ A \ r]$  by auto
        qed
      qed
    qed
  next
  case  $(\text{Suc } n)$ 
  show  $\forall k. 0 \leq k \wedge k \leq \text{Suc } n \longrightarrow (\exists y. (y, x) \in r \wedge y \in \text{nivel } A \ r \ k)$ 
proof(rule allI, rule impI)
  fix  $k$ 
  assume  $hip: 0 \leq k \wedge k \leq \text{Suc } n$ 

```

```

show  $(\exists y. (y, x) \in r \wedge y \in \text{nivel } A \ r \ k)$ 
proof-
  have  $(0 \leq k \wedge k \leq n) \vee k = \text{Suc } n$  using hip by auto
  thus ?thesis
  proof(rule disjE)
    assume hip1:  $0 \leq k \wedge k \leq n$ 
    have  $\exists y. (y, x) \in r \wedge y \in (\text{nivel } A \ r \ (n+1))$ 
    using  $\langle \text{arbol } A \ r \rangle \text{nivel } \langle x \in \text{nivel } A \ r \ (\text{Suc } n + 1) \rangle$  by auto
    then obtain y where y1:  $(y, x) \in r$  and y2:  $y \in (\text{nivel } A \ r \ (n+1))$ 
    by auto
    have  $\forall k. 0 \leq k \wedge k \leq n \longrightarrow (\exists z. (z, y) \in r \wedge z \in \text{nivel } A \ r \ k)$ 
    using y2 Suc(1-3) by auto
    hence  $(\exists z. (z, y) \in r \wedge z \in \text{nivel } A \ r \ k)$ 
    using hip1 by auto
    then obtain z where z1:  $(z, y) \in r$  and z2:  $z \in (\text{nivel } A \ r \ k)$  by auto
    have  $r \subseteq A \times A$  and orden-parcial-estricto-en A r
    using  $\langle \text{arbol } A \ r \rangle \text{arbol}$  by auto
    hence  $z \in A$  and  $y \in A$  and  $x \in A$ 
    using  $\langle r \subseteq A \times A \rangle \langle (z, y) \in r \rangle \langle (y, x) \in r \rangle$  by auto
    have trans-en A r using orden-parcial-estricto-en A r
    by(unfold orden-parcial-estricto-en-def, auto)
    hence  $(z, x) \in r$  using  $\langle z \in A \rangle \langle y \in A \rangle$  and  $\langle x \in A \rangle \langle (z, y) \in r \rangle \langle (y, x) \in r \rangle$ 
    by(unfold trans-en-def, blast)
    thus  $(\exists y. (y, x) \in r \wedge y \in \text{nivel } A \ r \ k)$ 
    using z2 by auto
  next
    assume  $k = \text{Suc } n$ 
    thus  $\exists y. (y, x) \in r \wedge y \in (\text{nivel } A \ r \ k)$ 
    using  $\langle \text{arbol } A \ r \rangle \text{nivel } \langle x \in \text{nivel } A \ r \ (\text{Suc } n + 1) \rangle$  by auto
  qed
qed
qed
qed
thus ?thesis using assms by auto
qed

lemma nivel1:
  assumes arbol A r
  shows  $(\text{nivel } A \ r \ (n+1)) \neq \{\}$   $\longrightarrow (\forall k. (0 \leq k \wedge k \leq n) \longrightarrow (\text{nivel } A \ r \ k) \neq \{\})$ 
proof(rule impI)
  assume hip:  $(\text{nivel } A \ r \ (n+1)) \neq \{\}$ 
  show  $(\forall k. (0 \leq k \wedge k \leq n) \longrightarrow (\text{nivel } A \ r \ k) \neq \{\})$ 
  proof-

```

have $\exists x. x \in (\text{nivel } A \ r \ (n+1))$ **using** *hip* **by** *auto*
then obtain x **where** $x: x \in (\text{nivel } A \ r \ (n+1))$ **by** *auto*
thus *?thesis* **using** *assms camino1*[*of A r*] **by** *blast*
qed
qed

lemma *nivel2*:

assumes *arbol A r*
shows $((\text{nivel } A \ r \ (n+1)) = \{\}) \longrightarrow (\forall k. k > (n+1) \longrightarrow (\text{nivel } A \ r \ k) = \{\})$
proof(*rule ccontr*)
assume *hip*: $\neg ((\text{nivel } A \ r \ (n+1)) = \{\}) \longrightarrow (\forall k > (n+1). \text{nivel } A \ r \ k = \{\})$
show *False*
proof–
have $((\text{nivel } A \ r \ (n+1)) = \{\}) \wedge \neg (\forall k > (n+1). \text{nivel } A \ r \ k = \{\})$
using *hip* **by** *auto*
hence *1*: $(\text{nivel } A \ r \ (n+1)) = \{\}$ **and** *2*: $\exists k > (n+1). (\text{nivel } A \ r \ k) \neq \{\}$
by *auto*
obtain z **where** *z1*: $z > (n+1)$ **and** *z2*: $(\text{nivel } A \ r \ z) \neq \{\}$
using *2* **by** *auto*
have $z > 0$ **using** $\langle z > (n+1) \rangle$ **by** *auto*
hence $(\text{nivel } A \ r \ ((z-1)+1)) \neq \{\}$
using *z2* **by** *simp*
hence $\forall k. (0 \leq k \wedge k \leq (z-1)) \longrightarrow (\text{nivel } A \ r \ k) \neq \{\}$
using *z2* $\langle \text{arbol } A \ r \rangle$ *nivel1*[*of A r z-1*]
by *auto*
hence $(\text{nivel } A \ r \ (n+1)) \neq \{\}$
using $\langle z > (n+1) \rangle$ **by** *auto*
thus *False* **using** *1* **by** *auto*
qed
qed

lemma *arbol-finito1*:

assumes *arbol A r* **and** $\forall k. k > m \longrightarrow (\text{nivel } A \ r \ k) = \{\}$
shows $A = (\bigcup n \in \{0..m\}. \text{nivel } A \ r \ n)$
proof–
have $a: A = (\bigcup n. \text{nivel } A \ r \ n)$ **using** *union-niveles*[*of A r*] **by** *auto*
have $(\bigcup n. \text{nivel } A \ r \ n) = (\bigcup n \in \{0..m\}. \text{nivel } A \ r \ n)$
proof(*rule equalityI*)
show $(\bigcup n. \text{nivel } A \ r \ n) \subseteq (\bigcup n \in \{0..m\}. \text{nivel } A \ r \ n)$
proof(*rule subsetI*)
fix x
assume *hip*: $x \in (\bigcup n. \text{nivel } A \ r \ n)$
show $x \in (\bigcup n \in \{0..m\}. \text{nivel } A \ r \ n)$

```

proof–
  have  $\exists n. x \in \text{nivel } A \text{ } r \text{ } n$ 
  using hip nivel-elemento[of A] by auto
  then obtain n where  $n: x \in \text{nivel } A \text{ } r \text{ } n$  by auto
  have  $n \in \{0..m\}$ 
  proof(rule ccontr)
    assume  $1: n \notin \{0..m\}$ 
    show False
  proof–
    have  $n > m$  using 1 by auto
    thus False using assms(2) n by auto
  qed
qed
thus  $x \in (\bigcup n \in \{0..m\}. \text{nivel } A \text{ } r \text{ } n)$  using n by auto
qed
qed
next
  show  $(\bigcup n \in \{0..m\}. \text{nivel } A \text{ } r \text{ } n) \subseteq (\bigcup n. \text{nivel } A \text{ } r \text{ } n)$  by auto
qed
thus  $A = (\bigcup n \in \{0..m\}. \text{nivel } A \text{ } r \text{ } n)$  using a by auto
qed

```

lemma *arbol-finito*:

assumes *arbol* *A r* **and** *ramificacion-finita* *A r*
and $\exists n. (\forall k. k > n \longrightarrow (\text{nivel } A \text{ } r \text{ } k) = \{\})$
shows *finite* *A*

```

proof–
  obtain m where  $m: (\forall k. k > m \longrightarrow (\text{nivel } A \text{ } r \text{ } k) = \{\})$ 
  using assms(3) by auto
  hence  $1: A = (\bigcup n \in \{0..m\}. \text{nivel } A \text{ } r \text{ } n)$ 
  using assms(1) assms(3) arbol-finito1[of A r m] by auto
  have  $\forall n. \text{finite } (\text{nivel } A \text{ } r \text{ } n)$ 
  using assms(1–2) finito-nivel by auto
  hence  $\forall n \in \{0..m\}. \text{finite } (\text{nivel } A \text{ } r \text{ } n)$  by auto
  hence finite  $(\bigcup n \in \{0..m\}. \text{nivel } A \text{ } r \text{ } n)$  by auto
  thus finite A using 1 by auto
qed

```

lemma *niveles-no-vacios*:

assumes *arbol-infinito* *A r* **and** *ramificacion-finita* *A r*
shows $\forall n. \text{nivel } A \text{ } r \text{ } n \neq \{\}$

```

proof(rule ccontr)
  assume hip:  $\neg (\forall n. \text{nivel } A \text{ } r \text{ } n \neq \{\})$ 

```

```

show False
proof-
  have arbol  $A$   $r$  using  $\langle \text{arbol-infinito } A \ r \rangle$  by auto
  have  $(\exists n. \text{nivel } A \ r \ n = \{\})$  using hip by auto
  then obtain  $n$  where  $n: \text{nivel } A \ r \ n = \{\}$  by auto
  thus False
proof(cases  $n$ )
  case 0
  then show False
proof-
  have  $\exists a. \text{minimo } A \ a \ r$  using  $\langle \text{arbol } A \ r \rangle \text{ arbol}[of \ A \ r]$  by auto
  then obtain  $a$  where  $a: \text{minimo } A \ a \ r$  by auto
  have orden-parcial-estricto-en  $A \ r$ 
  and  $\forall x \in A. \text{finite } (\text{predecesores-de } A \ x \ r)$ 
    using  $\langle \text{arbol } A \ r \rangle \text{ arbol}[of \ A \ r]$  by auto
  hence  $\text{nivel } A \ r \ n = \{a\}$ 
    using  $a \ \langle n=0 \rangle \text{nivel-cero}[of \ A \ r \ a]$  by auto
  thus False using  $\langle \text{nivel } A \ r \ n = \{\} \rangle$  by auto
qed
next
case (Suc nat)
fix  $m$ 
assume hip:  $n = \text{Suc } m$  show False
proof-
  have 1:  $\text{nivel } A \ r \ (\text{Suc } m) = \{\}$ 
    using hip  $n$  by auto
  have  $(\forall k. k > (m+1) \longrightarrow (\text{nivel } A \ r \ k) = \{\})$ 
    using  $\langle \text{arbol } A \ r \rangle \ 1 \ \text{nivel2}[of \ A \ r \ m]$  by auto
  hence 1:  $(\exists n. \forall k. k > n \longrightarrow (\text{nivel } A \ r \ k) = \{\})$  by auto
  hence 2: finite  $A$ 
    using  $\langle \text{arbol } A \ r \rangle \ 1 \ \langle \text{ramificacion-finita } A \ r \rangle \text{ arbol-finito}[of \ A \ r]$  by auto
  have 3:  $\neg \text{finite } A$  using  $\langle \text{arbol-infinito } A \ r \rangle$  by auto
  show False using 2 3 by auto
qed
qed
qed
qed

lemma no-ciclos:
  assumes arbol  $A \ r$  and  $(x,z) \in r$  and  $(y,z) \in r$  and  $x \neq y$ 
  shows  $(x,y) \in r \vee (y,x) \in r$ 
proof-
  have  $r \subseteq A \times A$  using  $\langle \text{arbol } A \ r \rangle$  by(unfold arbol-def, auto)

```

hence $x \in A$ and $y \in A$ and $z \in A$ using $\langle (x,z) \in r \rangle$ and $\langle (y,z) \in r \rangle$ by *auto*
 hence 1: $x \in \text{predecesores-de } A \text{ } z \text{ } r$ and 2: $y \in \text{predecesores-de } A \text{ } z \text{ } r$
 using *assms* by(*unfold predecesores-de-def*, *auto*)
 have (*total-en* (*predecesores-de* $A \text{ } z \text{ } r$) r)
 using $\langle \text{arbol } A \text{ } r \rangle \langle z \in A \rangle$ by(*unfold arbol-def*, *auto*)
 thus ?thesis using 1 2 $\langle x \neq y \rangle$ *total-en-def*[of *predecesores-de* $A \text{ } z \text{ } r$] by *auto*
 qed

lemma *contenencia-predecesores*:

assumes $r \subseteq A \times A$ and *orden-parcial-estricto-en* $A \text{ } r$ and $\langle x, y \rangle \in r$
 shows (*predecesores-de* $A \text{ } x \text{ } r$) $\subseteq$ (*predecesores-de* $A \text{ } y \text{ } r$)

proof—

have *irrefl-en* $A \text{ } r$ and *trans-en* $A \text{ } r$
 using *assms*(2) by (*unfold orden-parcial-estricto-en-def*, *auto*)
 have 1: (*predecesores-de* $A \text{ } x \text{ } r$) $\subseteq$ (*predecesores-de* $A \text{ } y \text{ } r$)
proof(*rule subsetI*)
 fix z
 assume $z \in \text{predecesores-de } A \text{ } x \text{ } r$
 hence $z \in A$ and $\langle z, x \rangle \in r$ by(*unfold predecesores-de-def*, *auto*)
 have $x \in A$ and $y \in A$ using $\langle (x,y) \in r \rangle \langle r \subseteq A \times A \rangle$ by *auto*
 hence $\langle z, y \rangle \in r$
 using $\langle z \in A \rangle \langle y \in A \rangle \langle x \in A \rangle \langle \langle z, x \rangle \in r \rangle \langle \langle x, y \rangle \in r \rangle \langle \text{trans-en } A \text{ } r \rangle$
 by (*unfold trans-en-def*, *blast*)
 thus $z \in \text{predecesores-de } A \text{ } y \text{ } r$
 using $\langle z \in A \rangle$ by(*unfold predecesores-de-def*, *auto*)

qed

have 2: $x \in \text{predecesores-de } A \text{ } y \text{ } r$
 using $\langle r \subseteq A \times A \rangle \langle \langle x, y \rangle \in r \rangle$ by(*unfold predecesores-de-def*, *auto*)

have 3: $x \notin \text{predecesores-de } A \text{ } x \text{ } r$

proof(*rule ccontr*)

assume $\neg x \notin \text{predecesores-de } A \text{ } x \text{ } r$
 hence $x \in \text{predecesores-de } A \text{ } x \text{ } r$ by *auto*
 hence $x \in A \wedge \langle x, x \rangle \in r$
 by(*unfold predecesores-de-def*, *auto*)
 thus *False* using *irrefl-en* $A \text{ } r$
 by (*unfold irrefl-en-def*, *auto*)

qed

have (*predecesores-de* $A \text{ } x \text{ } r$) $\neq$ (*predecesores-de* $A \text{ } y \text{ } r$)

using 2 3 by *auto*

thus ?thesis using 1 by *auto*

qed

lemma *alturas-diferentes*:

assumes $r \subseteq A \times A$ **and** *orden-parcial-estricto-en* A r **and** $(x,y) \in r$
and *finite* (*predecesores-de* A y r)
shows *altura* A x r $<$ *altura* A y r
proof–
have *card*(*predecesores-de* A x r) $<$ *card*(*predecesores-de* A y r)
using *assms* *contenencia-predecesores*[*of* r A x y] *psubset-card-mono* **by** *auto*
thus ?thesis **by**(*unfold* *altura-def*, *auto*)
qed

lemma *pertenecen-niveles-diferentes*:

assumes $r \subseteq A \times A$ **and** *orden-parcial-estricto-en* A r **and** $(x,y) \in r$
and $x \in (\text{nivel } A \text{ } r \text{ } n)$ **and** $y \in (\text{nivel } A \text{ } r \text{ } m)$
and *finite* (*predecesores-de* A y r)
shows $\text{nivel } A \text{ } r \text{ } n \neq \text{nivel } A \text{ } r \text{ } m$
proof(*rule ccontr*)
assume $\neg \text{nivel } A \text{ } r \text{ } n \neq \text{nivel } A \text{ } r \text{ } m$
hence $\text{nivel } A \text{ } r \text{ } n = \text{nivel } A \text{ } r \text{ } m$ **by** *auto*
hence $x \in (\text{nivel } A \text{ } r \text{ } m)$ **using** $\langle x \in (\text{nivel } A \text{ } r \text{ } n) \rangle$ **by** *auto*
hence 1: *altura* A x $r = m$ **by**(*unfold* *nivel-def*, *auto*)
have *altura* A y $r = m$ **using** $\langle y \in (\text{nivel } A \text{ } r \text{ } m) \rangle$ **by**(*unfold* *nivel-def*, *auto*)
hence *altura* A x $r = \text{altura } A \text{ } y \text{ } r$ **using** 1 **by** *auto*
thus *False*
using *assms* *alturas-diferentes*[*of* r A x y] **by** (*unfold* *nivel-def*, *auto*)
qed

lemma *altura-predecesor*:

assumes $r \subseteq A \times A$ **and** *orden-parcial-estricto-en* A r
and $x \in \text{predecesores-de } A \text{ } y \text{ } r$ **and** $y \in (\text{nivel } A \text{ } r \text{ } n)$
and $x \in (\text{nivel } A \text{ } r \text{ } m)$
and *finite* (*predecesores-de* A y r)
shows $m < n$
proof–
have $(x,y) \in r$ **using** $\langle x \in \text{predecesores-de } A \text{ } y \text{ } r \rangle$
by (*unfold* *predecesores-de-def*, *auto*)
thus ?thesis
using *assms* *alturas-diferentes*[*of* r A x y] **by**(*unfold* *nivel-def*, *auto*)
qed

lemma *niveles-diferentes-arbol*:

assumes *arbol* A r **and** $x \in (\text{predecesores-de } A \text{ } z \text{ } r)$
and $y \in (\text{predecesores-de } A \text{ } z \text{ } r)$
and $x \neq y$ **and** $x \in (\text{nivel } A \text{ } r \text{ } n)$ **and** $y \in (\text{nivel } A \text{ } r \text{ } m)$
shows $\text{nivel } A \text{ } r \text{ } n \cap \text{nivel } A \text{ } r \text{ } m = \{\}$

proof–

have $(x,y) \in r \vee (y,x) \in r$
using *assms no-ciclos*[of A] **by**(*unfold predecesores-de-def*, *auto*)
thus $\text{nivel } A \ r \ n \cap \text{nivel } A \ r \ m = \{\}$
proof(*rule disjE*)
assume $(x, y) \in r$
have $r \subseteq A \times A$ **and** $1: \text{orden-parcial-estricto-en } A \ r$
using $\langle \text{arbol } A \ r \rangle$ **by**(*unfold arbol-def*, *auto*)
hence $x \in A$ **and** $y \in A$ **and** $2: x \in (\text{predecesores-de } A \ y \ r)$
using $\langle (x, y) \in r \rangle$ **by**(*unfold predecesores-de-def*, *auto*)
have $3: \text{finite } (\text{predecesores-de } A \ y \ r)$
using $\langle y \in A \rangle \langle \text{arbol } A \ r \rangle$ **by**(*unfold arbol-def*, *auto*)
hence $n < m$
using *assms* $\langle r \subseteq A \times A \rangle \ 1 \ 2 \ 3$ *altura-predecesor*[of $r \ A \ x \ y \ m \ n$]
by *auto*
hence $\exists k > 0. m = n + k$ **by** *arith*
then obtain k **where** $k: k > 0$ **and** $m: m = n + k$ **by** *auto*
thus ?thesis **using** *niveles-distintos*[OF k , of A]
by *auto*

next

assume $(y, x) \in r$
have $r \subseteq A \times A$ **and** $1: \text{orden-parcial-estricto-en } A \ r$
using $\langle \text{arbol } A \ r \rangle$ **by**(*unfold arbol-def*, *auto*)
hence $x \in A$ **and** $y \in A$ **and** $2: y \in (\text{predecesores-de } A \ x \ r)$
using $\langle (y, x) \in r \rangle$
by(*unfold predecesores-de-def*, *auto*)
have $3: \text{finite } (\text{predecesores-de } A \ x \ r)$
using $\langle x \in A \rangle \langle \text{arbol } A \ r \rangle$
by(*unfold arbol-def*, *auto*)
hence $m < n$
using *assms* $\langle r \subseteq A \times A \rangle \ 1 \ 2 \ 3$ *altura-predecesor*[of $r \ A \ y \ x \ n \ m$]
by *auto*
hence $\exists k > 0. n = m + k$ **by** *arith*
then obtain k **where** $k: k > 0$ **and** $m: n = m + k$ **by** *auto*
thus ?thesis **using** *niveles-distintos*[OF k , of A] **by** *auto*

qed

qed

lemma *niveles-diferentes-arbol1*:

assumes *arbol* $A \ r$ **and** $(x,z) \in r$ **and** $(y,z) \in r$
and $x \neq y$ **and** $x \in (\text{nivel } A \ r \ n)$ **and** $y \in (\text{nivel } A \ r \ m)$
shows $\text{nivel } A \ r \ n \cap \text{nivel } A \ r \ m = \{\}$

proof–

have $r \subseteq A \times A$ **using** $\langle \text{arbol } A \ r \rangle$ **arbol** **by** *auto*
hence $x \in A$ **and** $y \in A$ **using** $\langle r \subseteq A \times A \rangle \langle (x, z) \in r \rangle \langle (y, z) \in r \rangle$ **by** *auto*
hence $x \in (\text{predecesores-de } A \ z \ r)$ **and** $y \in (\text{predecesores-de } A \ z \ r)$
using $\langle (x, z) \in r \rangle$ **and** $\langle (y, z) \in r \rangle$ **by** $(\text{unfold predecesores-de-def}, \text{auto})$
thus *?thesis*
using *assms niveles-diferentes-arbol*[*of* $A \ r$] **by** *blast*
qed

definition *conj-a-lista* :: $'a \text{ set} \Rightarrow 'a \text{ list}$
where *conj-a-lista* $s = (\text{SOME } l. \text{ set } l = s)$

lemma *conj-conj-a-list*:
 $\text{finite } s \implies \text{set } (\text{conj-a-lista } s) = s$
unfolding *conj-a-lista-def* **by** $(\text{metis } (\text{mono-tags}) \text{ finite-list some-eq-ex})$

lemma *lista-conjunto*:
assumes *arbol* $A \ r$ **and** *ramificacion-finita* $A \ r$
shows $\text{set } (\text{conj-a-lista } (\text{nivel } A \ r \ n)) = (\text{nivel } A \ r \ n)$
using *assms finito-nivel*[*of* $A \ r \ n$] *conj-conj-a-list* **by** *auto*

primrec *disyuncion-nodos* :: $'a \text{ list} \Rightarrow 'a \text{ formula}$ **where**
 $\text{disyuncion-nodos } [] = \text{FF}$
 $|\ \text{disyuncion-nodos } (v \# D) = (\text{Atomo } v) \vee. (\text{disyuncion-nodos } D)$

lemma *valor-verdad-disyuncion-nodos*:
assumes $v \in \text{set } l$ **and** *valor* $I \ (\text{Atomo } v) = \text{Verdad}$
shows *valor* $I \ (\text{disyuncion-nodos } l) = \text{Verdad}$

proof–

have $v \in \text{set } l \implies \text{valor } I \ (\text{Atomo } v) = \text{Verdad} \implies$
 $\text{valor } I \ (\text{disyuncion-nodos } l) = \text{Verdad}$

proof(*induct* l)

case *Nil*

then show *?case* **by** *auto*

next

case $(\text{Cons } a \ l)$

then show *valor* $I \ (\text{disyuncion-nodos } (a \# l)) = \text{Verdad}$

proof–

have $v = a \vee v \neq a$ **by** *auto*

thus *valor* $I \ (\text{disyuncion-nodos } (a \# l)) = \text{Verdad}$

proof(*rule* *disjE*)

assume $v = a$

hence $1: \text{disyuncion-nodos } (a \# l) = (\text{Atomo } v) \vee. (\text{disyuncion-nodos } l)$

by *auto*

```

    have valor I ((Atomo v)  $\vee$ . (disyuncion-nodos l)) = Verdad
      using Cons(3) by (unfold valor-def, unfold v-disyuncion-def, auto)
    thus ?thesis using 1 by auto
  next
    assume v  $\neq$  a
    hence v  $\in$  set l using Cons(2) by auto
    hence valor I (disyuncion-nodos l) = Verdad
      using Cons(1) Cons(3) by auto
    thus ?thesis
      by (unfold valor-def, unfold v-disyuncion-def, auto)
  qed
qed
qed
thus ?thesis using assms by auto
qed

```

lemma *valor-verdad-disyuncion-formulas:*

```

  assumes arbol A r and ramificacion-finita A r
  and v  $\in$  (nivel A r n)  $\wedge$  valor I (Atomo v) = Verdad
  and F = disyuncion-nodos(conj-a-lista (nivel A r n))
  shows valor I F = Verdad

```

proof–

```

  have set (conj-a-lista (nivel A r n)) = (nivel A r n)
    using lista-conjunto assms(1–2) by auto
  hence v  $\in$  set (conj-a-lista (nivel A r n))
    using assms(3) by auto
  thus valor I F = Verdad
    using assms(3–4) valor-verdad-disyuncion-nodos by auto

```

qed

definition $\mathcal{F}' :: 'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow ('a \text{ formula}) \text{ set}$ **where**

$$\mathcal{F}' A r \equiv (\bigcup n. \{ \text{disyuncion-nodos}(\text{conj-a-lista } (\text{nivel } A r n)) \})$$

definition $\mathcal{G}' :: 'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow ('a \text{ formula}) \text{ set}$ **where**

$$\mathcal{G}' A r \equiv \{ (\text{Atomo } u) \rightarrow. (\text{Atomo } v) \mid u v. u \in A \wedge v \in A \wedge (v, u) \in r \}$$

definition $\mathcal{H}n :: 'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow \text{nat} \Rightarrow ('a \text{ formula}) \text{ set}$ **where**

$$\mathcal{H}n A r n \equiv \{ \neg. ((\text{Atomo } u) \wedge. (\text{Atomo } v)) \mid u v. u \in (\text{nivel } A r n) \wedge v \in (\text{nivel } A r n) \wedge u \neq v \}$$

definition $\mathcal{H}' :: 'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow ('a \text{ formula}) \text{ set}$ **where**

$$\mathcal{H}' A r \equiv \bigcup n. \mathcal{H}n A r n$$

definition $\mathcal{T}' :: 'a \text{ set} \Rightarrow 'a \text{ rel} \Rightarrow ('a \text{ formula}) \text{ set}$ **where**

$$\mathcal{T}' A r \equiv (\mathcal{F}' A r) \cup (\mathcal{G}' A r) \cup (\mathcal{H}' A r)$$

primrec *nodos-formula* :: 'v formula $\Rightarrow$ 'v set **where**

nodos-formula $FF = \{\}$
| *nodos-formula* $TT = \{\}$
| *nodos-formula* $(\text{Atomo } P) = \{P\}$
| *nodos-formula* $(\neg. F) = \text{nodos-formula } F$
| *nodos-formula* $(F \wedge. G) = \text{nodos-formula } F \cup \text{nodos-formula } G$
| *nodos-formula* $(F \vee. G) = \text{nodos-formula } F \cup \text{nodos-formula } G$
| *nodos-formula* $(F \rightarrow. G) = \text{nodos-formula } F \cup \text{nodos-formula } G$

definition *nodos-conjunto-formulas* :: 'v formula set $\Rightarrow$ 'v set **where**

nodos-conjunto-formulas $S = (\bigcup F \in S. \text{nodos-formula } F)$

definition *maximo-nivel* :: 'v set $\Rightarrow$ 'v rel $\Rightarrow$ 'v formula set $\Rightarrow$ nat **where**

maximo-nivel $A r S = \text{Max } (\bigcup x \in \text{nodos-conjunto-formulas } S. \{\text{altura } A x r\})$

lemma *nodo-formula*:

assumes $v \in \text{set } l$

shows $v \in \text{nodos-formula } (\text{disyuncion-nodos } l)$

proof–

have $v \in \text{set } l \implies v \in \text{nodos-formula } (\text{disyuncion-nodos } l)$

proof(*induct* l)

case *Nil*

then show ?case **by** *auto*

next

case (*Cons* $a l$)

show $v \in \text{nodos-formula } (\text{disyuncion-nodos } (a \# l))$

proof–

have $v = a \vee v \neq a$ **by** *auto*

thus $v \in \text{nodos-formula } (\text{disyuncion-nodos } (a \# l))$

proof(*rule* *disjE*)

assume $v = a$

hence 1: $\text{disyuncion-nodos } (a \# l) = (\text{Atomo } v) \vee. (\text{disyuncion-nodos } l)$

by *auto*

have $v \in \text{nodos-formula } ((\text{Atomo } v) \vee. (\text{disyuncion-nodos } l))$ **by** *auto*

thus ?thesis **using** 1 **by** *auto*

next

assume $v \neq a$

hence $v \in \text{set } l$ **using** *Cons*(2) **by** *auto*

hence $v \in \text{nodos-formula } (\text{disyuncion-nodos } l)$

using *Cons*(1) *Cons*(2) **by** *auto*

thus ?thesis **by** *auto*

```

qed
qed
qed
thus ?thesis using assms by auto
qed

```

lemma *nodo-disyuncion-formulas*:

```

assumes arbol A r and ramificacion-finita A r and v ∈ (nivel A r n)
and F = disyuncion-nodos(conj-a-lista (nivel A r n))
shows v ∈ nodos-formula F

```

proof–

```

have set (conj-a-lista (nivel A r n)) = (nivel A r n)
using lista-conjunto assms(1–2) by auto
hence v ∈ set (conj-a-lista (nivel A r n))
using assms(3) by auto
thus v ∈ nodos-formula F
using assms(3–4) nodo-formula by auto

```

qed

definition *nodo-sig-nivel-max*:: $'v \text{ set} \Rightarrow 'v \text{ rel} \Rightarrow 'v \text{ formula set} \Rightarrow 'v$

```

where nodo-sig-nivel-max A r S =
(SOME u. u ∈ (nivel A r ((maximo-nivel A r S)+1)))

```

fun *interpretacion-camino* :: $'v \text{ set} \Rightarrow 'v \text{ rel} \Rightarrow 'v \Rightarrow ('v \Rightarrow v\text{-verdad})$ **where**
interpretacion-camino A r v = $(\lambda u. (\text{if } (u,v) \in r \text{ then Verdad else Falso}))$

lemma *finito-nodos-formula*:

```

finite (nodos-formula F) by(induct F, auto)

```

lemma *finito-conjunto-nodos*:

```

assumes finite S
shows finite (nodos-conjunto-formulas S)
using assms finito-nodos-formula
by (unfold nodos-conjunto-formulas-def, auto)

```

lemma *nodo-nivel-maximo*:

```

assumes arbol-infinito A r and ramificacion-finita A r
shows  $\exists u. u \in (\text{nivel } A \text{ r } ((\text{maximo-nivel } A \text{ r } S)+1))$ 
using assms niveles-no-vacios[of A r] by(unfold nivel-def, auto)

```

lemma *maximo1*:

```

assumes finite S and u ∈ nodos-conjunto-formulas S
shows (altura A u r) ≤ (maximo-nivel A r S)

```

proof–

have $(altura\ A\ u\ r) \in (\bigcup_{x \in \text{nodos-conjunto-formulas } S} \{altura\ A\ x\ r\})$
using *assms*(2) **by** *auto*
thus $(altura\ A\ u\ r) \leq (maximo-nivel\ A\ r\ S)$
using $\langle \text{finite } S \rangle$ *finito-conjunto-nodos*[of *S*]
by(*unfold maximo-nivel-def*, *auto*)

qed

lemma *negacion-implicacion*:

assumes *valor I* $(F \rightarrow G) = \text{Falso}$
shows *valor I* $F = \text{Verdad} \wedge \text{valor I } G = \text{Falso}$

proof(*rule ccontr*)

assume $\neg (\text{valor I } F = \text{Verdad} \wedge \text{valor I } G = \text{Falso})$
hence $\text{valor I } F = \text{Falso} \vee \text{valor I } G = \text{Verdad}$
using *CasosValor* **by** *auto*
hence *valor I* $(F \rightarrow G) = \text{Verdad}$
by(*unfold valor-def*, *unfold v-implicacion-def*, *auto*)
thus *False* **using** *assms* **by** *auto*

qed

lemma *valor-interpretacion-camino*:

assumes *valor* $(\text{interpretacion-camino } A\ r\ v) (\text{Atomo } u) = \text{Verdad}$
shows $(u, v) \in r$

proof(*rule ccontr*)

assume $(u, v) \notin r$
hence *valor* $(\text{interpretacion-camino } A\ r\ v) (\text{Atomo } u) = \text{Falso}$
by(*unfold valor-def*, *auto*)
thus *False* **using** *assms* **by** *auto*

qed

lemma *camino-satisfacible*:

assumes *arbol-infinito* $A\ r$
and *ramificacion-finita* $A\ r$ **and** $S \subseteq (\mathcal{T}'\ A\ r)$
and *finite* S

shows *satisfacible* S

proof–

let $?m = (maximo-nivel\ A\ r\ S) + 1$
let $?nivel = nivel\ A\ r\ ?m$
let $?u = \text{nodo-sig-nivel-max } A\ r\ S$
let $?F = \text{Atomo } (?u, ?m)$
have 1: *arbol* $A\ r$ **using** $\langle \text{arbol-infinito } A\ r \rangle$ **by** *auto*
have $r \subseteq A \times A$ **and** *orden-parcial-estricto-en* $A\ r$
using $\langle \text{arbol } A\ r \rangle$ *arbol* **by** *auto*

```

have trans-en A r
  using  $\langle \text{orden-parcial-estricto-en } A \ r \rangle$ 
  by(unfold orden-parcial-estricto-en-def, auto)
have  $\exists u. u \in \text{nivel } A \ r \ ?m$ 
  using assms(1-2) nodo-nivel-maximo by auto
then obtain u where u: u ∈ nivel A r ?m by auto
hence nivelu: ?u ∈ nivel A r ?m
  using someI by (unfold nodo-sig-nivel-max-def, auto)
hence ?u ∈ A by(unfold nivel-def, auto)
have (interpretacion-camino A r ?u) modelo S
proof(unfold modelo-def)
  show  $\forall F \in S. \text{valor}(\text{interpretacion-camino } A \ r \ ?u) \ F = \text{Verdad}$ 
  proof
    fix F assume F ∈ S
    show valor (interpretacion-camino A r ?u) F = Verdad
    proof–
      have  $F \in (\mathcal{F}' \ A \ r) \cup (\mathcal{G}' \ A \ r) \cup (\mathcal{H}' \ A \ r)$ 
      using  $\langle S \subseteq \mathcal{T}' \ A \ r \rangle \langle F \in S \rangle$  assms(2) by(unfold T'-def, auto)
      hence  $F \in (\mathcal{F}' \ A \ r) \vee F \in (\mathcal{G}' \ A \ r) \vee F \in (\mathcal{H}' \ A \ r)$  by auto
      thus ?thesis
      proof(rule disjE)
        assume  $F \in (\mathcal{F}' \ A \ r)$ 
        hence  $\exists n. F = \text{disyuncion-nodos}(\text{conj-a-lista}(\text{nivel } A \ r \ n))$ 
          by(unfold F'-def, auto)
        then obtain n
          where n: F = disyuncion-nodos(conj-a-lista (nivel A r n))
          by auto
        have  $\exists v. v \in (\text{nivel } A \ r \ n)$ 
          using assms(1-2) niveles-no-vacios[of A r] by auto
        then obtain v where v: v ∈ (nivel A r n) by auto
        hence  $v \in \text{nodos-formula } F$ 
          using n nodo-disyuncion-formulas[OF 1 assms(2) v, of F ]
          by auto
        hence a: v ∈ nodos-conjunto-formulas S
          using  $\langle F \in S \rangle$  by(unfold nodos-conjunto-formulas-def, blast)
        hence b: (altura A v r) ≤ (maximo-nivel A r S)
          using  $\langle \text{finite } S \rangle$  maximo1[of S v] by auto
        have (altura A v r) = n
          using v by(unfold nivel-def, auto)
        hence  $n < ?m$ 
          using  $\langle \text{finite } S \rangle$  a maximo1[of S v A r]
          by(unfold maximo-nivel-def, auto)
        hence  $(\exists y. (y, ?u) \in r \wedge y \in (\text{nivel } A \ r \ n))$ 

```

using $\text{nivelu } \langle \text{arbol } A \ r \rangle \text{ camino1 } [of \ A \ r]$
 by *auto*
 then obtain y where $y1: (y, ?u) \in r$ and $y2: y \in (\text{nivel } A \ r \ n)$
 by *auto*
 hence $\text{valor } (\text{interpretacion-camino } A \ r \ ?u) \ (\text{Atomo } y) = \text{Verdad}$
 by *auto*
 thus $\text{valor } (\text{interpretacion-camino } A \ r \ ?u) \ F = \text{Verdad}$
 using $1 \ \text{assms}(2) \ y2 \ n \ \text{valor-verdad-disyuncion-formulas} [of \ A \ r \ y]$
 by *auto*
 next
 assume $F \in \mathcal{G}' \ A \ r \vee F \in \mathcal{H}' \ A \ r$
 thus $\text{valor } (\text{interpretacion-camino } A \ r \ ?u) \ F = \text{Verdad}$
 proof(*rule disjE*)
 assume $F \in \mathcal{G}' \ A \ r$
 hence $\exists u. \exists v. u \in A \wedge v \in A \wedge (v, u) \in r \wedge$
 $(F = (\text{Atomo } u) \rightarrow. (\text{Atomo } v))$
 by (*unfold G'-def, auto*)
 then obtain $u \ v$ where $u \in A$ and $v \in A$ and $(v, u) \in r$
 and $F: (F = (\text{Atomo } u) \rightarrow. (\text{Atomo } v))$ by *auto*
 show $\text{valor } (\text{interpretacion-camino } A \ r \ ?u) \ F = \text{Verdad}$
 proof(*rule ccontr*)
 assume $\neg(\text{valor } (\text{interpretacion-camino } A \ r \ ?u) \ F = \text{Verdad})$
 hence $\text{valor } (\text{interpretacion-camino } A \ r \ ?u) \ F = \text{Falso}$
 using *CasosValor* by *auto*
 hence $\text{valor } (\text{interpretacion-camino } A \ r \ ?u) \ (\text{Atomo } u) = \text{Verdad} \wedge$
 $\text{valor } (\text{interpretacion-camino } A \ r \ ?u) \ (\text{Atomo } v) = \text{Falso}$
 using $F \ \text{negacion-implicacion}$ by *blast*
 hence $1: \text{valor } (\text{interpretacion-camino } A \ r \ ?u) \ (\text{Atomo } u) = \text{Verdad}$
 and $2: \text{valor } (\text{interpretacion-camino } A \ r \ ?u) \ (\text{Atomo } v) = \text{Falso}$
 by *auto*
 have $(u, ?u) \in r$ using $1 \ \text{valor-interpretacion-camino}$ by *auto*
 hence $(v, ?u) \in r$
 using $\langle u \in A \rangle \langle v \in A \rangle \langle ?u \in A \rangle \langle (v, u) \in r \rangle \langle \text{trans-en } A \ r \rangle$
 by (*unfold trans-en-def, blast*)
 hence $\text{valor } (\text{interpretacion-camino } A \ r \ ?u) \ (\text{Atomo } v) = \text{Verdad}$
 by *auto*
 thus *False* using 2 by *auto*
 qed
 next
 assume $F \in \mathcal{H}' \ A \ r$
 hence $\exists n. F \in \mathcal{H}n \ A \ r \ n$ by (*unfold H'-def, auto*)
 then obtain n where $F \in \mathcal{H}n \ A \ r \ n$ by *auto*
 hence

```

 $\exists u. \exists v. F = \neg.((Atomo\ u) \wedge. (Atomo\ v)) \wedge u \in (nivel\ A\ r\ n) \wedge$ 
 $v \in (nivel\ A\ r\ n) \wedge u \neq v$ 
  by (unfold  $\mathcal{H}n$ -def, auto)
then obtain  $u\ v$  where  $F: F = \neg.((Atomo\ u) \wedge. (Atomo\ v))$ 
and  $u \in (nivel\ A\ r\ n)$  and  $v \in (nivel\ A\ r\ n)$  and  $u \neq v$ 
  by auto
show  $valor\ (interpretacion-camino\ A\ r\ ?u)\ F = Verdad$ 
proof(rule ccontr)
  assume  $valor\ (interpretacion-camino\ A\ r\ ?u)\ F \neq Verdad$ 
  hence  $valor\ (interpretacion-camino\ A\ r\ ?u)\ F = Falso$ 
    using CasosValor by auto
  hence
     $valor\ (interpretacion-camino\ A\ r\ ?u)((Atomo\ u) \wedge.$ 
     $(Atomo\ v)) = Verdad$ 
    using  $F\ ValoresNegacion1$  by blast
  hence  $valor\ (interpretacion-camino\ A\ r\ ?u)(Atomo\ u) = Verdad \wedge$ 
   $valor\ (interpretacion-camino\ A\ r\ ?u)(Atomo\ v) = Verdad$ 
    using ValoresConjuncion by blast
  hence  $(u, ?u) \in r$  and  $(v, ?u) \in r$ 
    using valor-interpretacion-camino by auto
  hence  $a: (nivel\ A\ r\ n) \cap (nivel\ A\ r\ n) = \{\}$ 
    using  $\langle arbol\ A\ r \rangle\ \langle u \in (nivel\ A\ r\ n) \rangle\ \langle v \in (nivel\ A\ r\ n) \rangle\ \langle u \neq v \rangle$ 
    niveles-diferentes-arbol1[of  $A\ r$ ]
    by blast
  have  $(nivel\ A\ r\ n) \neq \{\}$ 
    using  $\langle v \in (nivel\ A\ r\ n) \rangle$  by auto
  thus False using  $a$  by auto
qed
qed
qed
qed
qed
qed
thus satisfacible S by(unfold satisfacible-def, auto)
qed

```

definition $\mathcal{B}:: 'a\ set \Rightarrow ('a \Rightarrow v-verdad) \Rightarrow 'a\ set$ **where**
 $\mathcal{B}\ A\ I \equiv \{u \mid u. u \in A \wedge valor\ I\ (Atomo\ u) = Verdad\}$

lemma *valor-disyuncion-lista1*:

assumes $valor\ I\ (disyuncion-nodos\ (a \# l)) = Verdad$

shows $valor\ I\ (Atomo\ a) = Verdad \vee valor\ I\ (disyuncion-nodos\ l) = Verdad$

proof–

```

have disyuncion-nodos (a # l) = (Atomo a) ∨. (disyuncion-nodos l)
  by auto
hence valor I ((Atomo a) ∨. (disyuncion-nodos l)) = Verdad
  using assms by auto
thus ?thesis using ValoresDisyuncion by blast
qed

lemma valor-disyuncion-lista:
  assumes valor I (disyuncion-nodos l) = Verdad
  shows ∃ x. x ∈ set l ∧ valor I (Atomo x) = Verdad
proof-
  have valor I (disyuncion-nodos l) = Verdad ⇒
    ∃ x. x ∈ set l ∧ valor I (Atomo x) = Verdad
  proof(induct l)
    case Nil
    then show ?case by auto
  next
    case (Cons a l)
    show ∃ x. x ∈ set (a # l) ∧ valor I (Atomo x) = Verdad
    proof-
      have valor I (Atomo a) = Verdad ∨ valor I (disyuncion-nodos l) = Verdad
        using Cons(2) valor-disyuncion-lista1[of I] by auto
      thus ?thesis
    proof(rule disjE)
      assume valor I (Atomo a) = Verdad
      thus ?thesis by auto
    next
      assume valor I (disyuncion-nodos l) = Verdad
      thus ?thesis
        using Cons by auto
    qed
  qed
qed
thus ?thesis using assms by auto
qed

```

```

lemma interseccion-rama-nivel1:
  assumes arbol-infinito A r and ramificacion-finita A r
  and I: ∀ F ∈ (F' A r). valor I F = Verdad
  shows ∀ n. ∃ x. x ∈ nivel A r n ∧ x ∈ (B A I) using niveles-no-vacios
proof-
  fix n
  have ∀ n. valor I (disyuncion-nodos(conj-a-lista (nivel A r n))) = Verdad

```

using I by (unfold $\mathcal{F}'$ -def, auto)
 hence 1:
 $\forall n. \exists x. x \in \text{set } (\text{conj-a-lista } (\text{nivel } A \text{ r } n)) \wedge \text{valor } I (\text{Atomo } x) = \text{Verdad}$
 using *valor-disyuncion-lista* by auto
 have *arbol* $A \text{ r}$
 using *arbol-infinito* $A \text{ r}$ by auto
 hence $\forall n. \text{set } (\text{conj-a-lista } (\text{nivel } A \text{ r } n)) = \text{nivel } A \text{ r } n$
 using *assms(1-2) lista-conjunto* by auto
 hence $\forall n. \exists x. x \in \text{nivel } A \text{ r } n \wedge \text{valor } I (\text{Atomo } x) = \text{Verdad}$
 using 1 by auto
 hence $\forall n. \exists x. x \in \text{nivel } A \text{ r } n \wedge x \in A \wedge \text{valor } I (\text{Atomo } x) = \text{Verdad}$
 by (unfold *nivel-def*, auto)
 thus ?thesis using *B-def[of A I]* by auto
 qed

lemma *interseccion-rama-nivel2*:

assumes $I: \forall F \in (\mathcal{H}' A \text{ r}). \text{valor } I F = \text{Verdad}$
 and $x \in (\mathcal{B} A I)$ and $y \in (\mathcal{B} A I)$ and $x \neq y$ and $n: x \in \text{nivel } A \text{ r } n$
 and $m: y \in \text{nivel } A \text{ r } m$
 shows $n \neq m$
 proof (rule ccontr)
 assume $\neg n \neq m$
 hence $n=m$ by auto
 have $x \in A$ and $y \in A$ and $v1: \text{valor } I (\text{Atomo } x) = \text{Verdad}$
 and $v2: \text{valor } I (\text{Atomo } y) = \text{Verdad}$
 using $\langle x \in (\mathcal{B} A I) \rangle \langle y \in (\mathcal{B} A I) \rangle$ by (unfold *B-def*, auto)
 have $\neg. ((\text{Atomo } x) \wedge. (\text{Atomo } y)) \in (\mathcal{H}n A \text{ r } n)$
 using $\langle x \in A \rangle \langle y \in A \rangle \langle x \neq y \rangle n \text{ m } \langle n=m \rangle$
 by (unfold *Hn-def*, auto)
 hence $\neg. ((\text{Atomo } x) \wedge. (\text{Atomo } y)) \in (\mathcal{H}' A \text{ r})$
 by (unfold *H'-def*, auto)
 hence $\text{valor } I (\neg. ((\text{Atomo } x) \wedge. (\text{Atomo } y))) = \text{Verdad}$
 using I by auto
 moreover
 have $\text{valor } I ((\text{Atomo } x) \wedge. (\text{Atomo } y)) = \text{Verdad}$
 using $v1 \text{ v2 } v\text{-conjuncion-def}$ by auto
 hence $\text{valor } I (\neg. ((\text{Atomo } x) \wedge. (\text{Atomo } y))) = \text{Falso}$
 using *v-negacion-def* by auto
 ultimately
 show *False* by auto
 qed

lemma *interseccion-rama-nivel*:

assumes *arbol-infinito* $A \ r$ **and** *ramificacion-finita* $A \ r$
and $I; \forall F \in (\mathcal{F}' A \ r) \cup (\mathcal{H}' A \ r). \text{valor } I \ F = \text{Verdad}$
shows $\forall n. \exists u. (\mathcal{B} A \ I) \cap \text{nivel } A \ r \ n = \{u\}$
proof
fix n
show $\exists u. (\mathcal{B} A \ I) \cap \text{nivel } A \ r \ n = \{u\}$
proof–
have $\exists u. u \in \text{nivel } A \ r \ n \wedge u \in (\mathcal{B} A \ I)$
using *assms interseccion-rama-nivel1*[*of* $A \ r \ I$] **by** *auto*
then obtain u **where** $u: u \in \text{nivel } A \ r \ n \wedge u \in (\mathcal{B} A \ I)$ **by** *auto*
hence $1: \{u\} \subseteq (\mathcal{B} A \ I) \cap \text{nivel } A \ r \ n$ **by** *blast*
have $2: (\mathcal{B} A \ I) \cap \text{nivel } A \ r \ n \subseteq \{u\}$
proof(*rule subsetI*)
fix x
assume $x \in (\mathcal{B} A \ I) \cap \text{nivel } A \ r \ n$
hence $2: x \in (\mathcal{B} A \ I) \wedge x \in \text{nivel } A \ r \ n$ **by** *auto*
have $u = x$
proof(*rule ccontr*)
assume $u \neq x$
hence $n \neq n$
using $u \ 2 \ I$ *interseccion-rama-nivel2*[*of* $A \ r$] **by** *blast*
thus *False* **by** *auto*
qed
thus $x \in \{u\}$ **by** *auto*
qed
have $(\mathcal{B} A \ I) \cap \text{nivel } A \ r \ n = \{u\}$
using $1 \ 2$ **by** *auto*
thus $\exists u. (\mathcal{B} A \ I) \cap \text{nivel } A \ r \ n = \{u\}$ **by** *auto*
qed
qed

lemma *predecesor-rama*:

assumes $I: \forall F \in (\mathcal{G}' A \ r). \text{valor } I \ F = \text{Verdad}$
and $y \in (\mathcal{B} A \ I)$ **and** $(x, y) \in r$ **and** $x \in A$ **and** $y \in A$
shows $x \in (\mathcal{B} A \ I)$
proof–
have $(\text{Atomo } y) \rightarrow. (\text{Atomo } x) \in \mathcal{G}' A \ r$
using $\langle x \in A \rangle \ \langle y \in A \rangle \ \langle (x, y) \in r \rangle$ **by** (*unfold* $\mathcal{G}'\text{-def}$, *auto*)
hence $\text{valor } I \ ((\text{Atomo } y) \rightarrow. (\text{Atomo } x)) = \text{Verdad}$
using I **by** *auto*
moreover
have $\text{valor } I \ (\text{Atomo } y) = \text{Verdad}$
using $\langle y \in (\mathcal{B} A \ I) \rangle$ **by** (*unfold* $\mathcal{B}\text{-def}$, *auto*)

```

ultimately
have valor I (Atomo x) = Verdad
  using v-implicacion-def by auto
thus  $x \in (\mathcal{B} \ A \ I)$  using  $\langle x \in A \rangle$  by (unfold  $\mathcal{B}$ -def, auto)
qed

lemma rama:
  assumes arbol-infinito A r and ramificacion-finita A r
  and I:  $\forall F \in (\mathcal{T}' \ A \ r). \text{valor } I \ F = \text{Verdad}$ 
shows rama ( $\mathcal{B} \ A \ I$ ) A r
proof (unfold rama-def)
  let ?B = ( $\mathcal{B} \ A \ I$ )
  have arbol A r
  using  $\langle \text{arbol-infinito } A \ r \rangle$  by auto
  have  $\forall F \in (\mathcal{F}' \ A \ r) \cup (\mathcal{G}' \ A \ r) \cup (\mathcal{H}' \ A \ r). \text{valor } I \ F = \text{Verdad}$ 
  using I by (unfold  $\mathcal{T}'$ -def)
  hence I1:  $\forall F \in (\mathcal{F}' \ A \ r). \text{valor } I \ F = \text{Verdad}$ 
  and I2:  $\forall F \in (\mathcal{G}' \ A \ r). \text{valor } I \ F = \text{Verdad}$ 
  and I3:  $\forall F \in (\mathcal{H}' \ A \ r). \text{valor } I \ F = \text{Verdad}$ 
  by auto
  have 0: sub-orden-lineal-en ?B A r
  proof (unfold sub-orden-lineal-en-def)
    have 1:  $?B \subseteq A$  by (unfold  $\mathcal{B}$ -def, auto)
    have 2: orden-parcial-estricto-en A r
    using  $\langle \text{arbol } A \ r \rangle$  arbol[of A r] by auto
    have total-en ?B r
    proof (unfold total-en-def)
      show  $\forall x \in ?B. \forall y \in ?B. x \neq y \longrightarrow (x, y) \in r \vee (y, x) \in r$ 
      proof
        fix x
        assume  $x \in ?B$ 
        show  $\forall y \in ?B. x \neq y \longrightarrow (x, y) \in r \vee (y, x) \in r$ 
        proof
          fix y
          assume  $y \in ?B$ 
          show  $x \neq y \longrightarrow (x, y) \in r \vee (y, x) \in r$ 
          proof (rule impI)
            assume  $x \neq y$ 
            have  $x \in A$  and  $y \in A$  and v1: valor I (Atomo x) = Verdad
            and v2: valor I (Atomo y) = Verdad
            using  $\langle x \in ?B \rangle \langle y \in ?B \rangle$  by (unfold  $\mathcal{B}$ -def, auto)
            have  $(\exists n. x \in \text{nivel } A \ r \ n)$  and  $(\exists m. y \in \text{nivel } A \ r \ m)$ 
            using  $\langle x \in A \rangle$  and  $\langle y \in A \rangle$  nivel-elemento[of A r]

```

```

    by auto
  then obtain  $n\ m$ 
  where  $n: x \in \text{nivel } A\ r\ n$  and  $m: y \in \text{nivel } A\ r\ m$ 
    by auto
  have  $n \neq m$ 
    using  $I3\ \langle x \in ?B \rangle\ \langle y \in ?B \rangle\ \langle x \neq y \rangle\ n\ m$ 
      interseccion-rama-nivel2[of  $A\ r$ ]
    by auto
  hence  $n < m \vee m < n$  by auto
  thus  $(x, y) \in r \vee (y, x) \in r$ 
  proof(rule disjE)
    assume  $n < m$ 
    have  $(x, y) \in r$ 
    proof(rule ccontr)
      assume  $(x, y) \notin r$ 
      have  $\exists z. (z, y) \in r \wedge z \in \text{nivel } A\ r\ n$ 
        using  $\langle \text{arbol } A\ r \rangle\ \langle y \in \text{nivel } A\ r\ m \rangle\ \langle n < m \rangle$ 
          camino1[of  $A\ r\ y\ m-1$ ]
      by auto
      then obtain  $z$  where  $z1: (z, y) \in r$  and  $z2: z \in \text{nivel } A\ r\ n$ 
        by auto
      have  $z \in A$  using  $\langle \text{arbol } A\ r \rangle\ \text{arbol } z1$  by auto
      hence  $z \in (B\ A\ I)$ 
        using  $I2\ \langle y \in A \rangle\ \langle y \in ?B \rangle\ \langle (z, y) \in r \rangle\ \text{predecesor-rama}[of\ A\ r\ I\ y\ z]$ 
      by auto
      have  $x \neq z$  using  $\langle (x, y) \notin r \rangle\ \langle (z, y) \in r \rangle$  by auto
      hence  $n \neq z$ 
        using  $I3\ \langle x \in ?B \rangle\ \langle z \in ?B \rangle\ n\ z2\ \text{interseccion-rama-nivel2}[of\ A\ r]$ 
      by blast
      thus False by auto
    qed
  thus  $(x, y) \in r \vee (y, x) \in r$  by auto
next
  assume  $m < n$ 
  have  $(y, x) \in r$ 
  proof(rule ccontr)
    assume  $(y, x) \notin r$ 
    have  $\exists z. (z, x) \in r \wedge z \in \text{nivel } A\ r\ m$ 
      using  $\langle \text{arbol } A\ r \rangle\ \langle x \in \text{nivel } A\ r\ n \rangle\ \langle m < n \rangle$ 
        camino1[of  $A\ r\ x\ n-1$ ]
    by auto
    then obtain  $z$  where  $z1: (z, x) \in r$  and  $z2: z \in \text{nivel } A\ r\ m$ 
      by auto

```

```

have  $z \in A$  using  $\langle \text{arbol } A \ r \rangle$   $\text{arbol } z1$  by auto
hence  $z \in (\mathcal{B} \ A \ I)$ 
  using  $I2 \ \langle x \in A \rangle \ \langle x \in ?B \rangle \ \langle (z, x) \in r \rangle$   $\text{predecesor-rama}[of \ A \ r \ I \ x \ z]$ 
  by auto
have  $y \neq z$  using  $\langle (y, x) \notin r \rangle \ \langle (z, x) \in r \rangle$  by auto
hence  $m \neq n$ 
  using  $I3 \ \langle y \in ?B \rangle \ \langle z \in ?B \rangle \ m \ z2$   $\text{interseccion-rama-nivel2}[of \ A \ r \ ]$ 
  by blast
thus False by auto
qed
thus  $(x, y) \in r \vee (y, x) \in r$  by auto
qed
qed
qed
qed
qed
thus  $?B \subseteq A \wedge \text{orden-parcial-estricto-en } A \ r \wedge \text{total-en } ?B \ r$ 
  using 1 2 by auto
qed
have  $\mathcal{A}: (\forall C. ?B \subseteq C \wedge \text{sub-orden-lineal-en } C \ A \ r \longrightarrow ?B = C)$ 
proof
  fix C
  show  $?B \subseteq C \wedge \text{sub-orden-lineal-en } C \ A \ r \longrightarrow ?B = C$ 
  proof(rule impI)
    assume  $?B \subseteq C \wedge \text{sub-orden-lineal-en } C \ A \ r$ 
    hence  $?B \subseteq C$  and  $\text{sub-orden-lineal-en } C \ A \ r$  by auto
    have  $C \subseteq ?B$ 
    proof(rule subsetI)
      fix x
      assume  $x \in C$ 
      have  $C \subseteq A$ 
        using  $\langle \text{sub-orden-lineal-en } C \ A \ r \rangle$ 
        by(unfold sub-orden-lineal-en-def, auto)
      hence  $x \in A$  using  $\langle x \in C \rangle$  by auto
      have  $\exists n. x \in \text{nivel } A \ r \ n$ 
        using  $\langle x \in A \rangle$   $\text{nivel-elemento}[of \ A]$  by auto
      then obtain n where  $n: x \in \text{nivel } A \ r \ n$  by auto
      have  $\exists u. (\mathcal{B} \ A \ I) \cap \text{nivel } A \ r \ n = \{u\}$ 
        using  $\text{assms}(1,2) \ I1 \ I3$   $\text{interseccion-rama-nivel}[of \ A \ r]$ 
        by blast
      then obtain u where  $i: (\mathcal{B} \ A \ I) \cap \text{nivel } A \ r \ n = \{u\}$ 
        by auto
      hence  $u \in A$  and  $u: u \in \text{nivel } A \ r \ n$ 

```

```

by(unfold nivel-def, auto)
have x=u
proof(rule ccontr)
  assume hip: x≠u
  have u∈(B A I) using i by auto
  hence u∈C using ⟨?B ⊆ C⟩ by auto
  have total-en C r
    using ⟨sub-orden-lineal-en C A r⟩ sub-orden-lineal-en-def[of C A r]
    by blast
  hence (x,u)∈r ∨ (u,x)∈r
    using hip ⟨x∈C⟩ ⟨u∈C⟩ ⟨sub-orden-lineal-en C A r⟩
    by(unfold total-en-def,auto)
  thus False
proof(rule disjE)
  assume (x,u)∈r
  have r ⊆ A × A and orden-parcial-estricto-en A r
  and finite (predecesores-de A u r)
    using ⟨u∈A⟩ ⟨arbol A r⟩ arbol[of A r] by auto
  hence (nivel A r n) ≠ (nivel A r n)
    using ⟨(x,u)∈r⟩ ⟨x ∈ nivel A r n⟩ ⟨u ∈ nivel A r n⟩
    pertenecen-niveles-diferentes[of r A ] by blast
  thus False by auto
next
  assume (u,x)∈r
  have r ⊆ A × A and orden-parcial-estricto-en A r
  and finite (predecesores-de A x r)
    using ⟨x∈A⟩ ⟨arbol A r⟩ arbol[of A r] by auto
  hence (nivel A r n) ≠ (nivel A r n)
    using ⟨(u,x)∈r⟩ ⟨u ∈ nivel A r n⟩ ⟨x ∈ nivel A r n⟩
    pertenecen-niveles-diferentes[of r A ] by blast
  thus False by auto
qed
qed
thus x ∈ ?B using i by auto
qed
thus ?B = C using ⟨?B ⊆ C⟩ by blast
qed
qed
thus sub-orden-lineal-en (B A I) A r ∧
  (∀ C. B A I ⊆ C ∧ sub-orden-lineal-en C A r ⟶ B A I = C)
  using ⟨sub-orden-lineal-en (B A I) A r⟩ by auto
qed

```

lemma *sobreyectiva-infinito*:
assumes $\exists f:: 'a \Rightarrow \text{nat}. \forall n. \exists x \in A. n = f(x)$
shows *infinite A*
proof(*rule ccontr*)
assume $\neg \text{infinite } A$
hence *finite A* **by** *auto*
hence $\exists n. \exists g. A = g \text{ ` } \{i::\text{nat}. i < n\}$
using *finite-imp-nat-seg-image-inj-on[of A]* **by** *auto*
then obtain $n \ g$ **where** $g: A = g \text{ ` } \{i::\text{nat}. i < n\}$ **by** *auto*
obtain f **where** $(\forall n. \exists x \in A. n = (f:: 'a \Rightarrow \text{nat})(x))$
using *assms* **by** *auto*
hence $\forall m. \exists k \in \{i::\text{nat}. i < n\}. m = (f \circ g)(k)$
using g **by** *auto*
hence $(UNIV :: \text{nat set}) = (f \circ g) \text{ ` } \{i::\text{nat}. i < n\}$
by *blast*
hence *finite (UNIV :: nat set)*
using *nat-seg-image-imp-finite* **by** *blast*
thus *False* **by** *auto*
qed

lemma *familia-interseccion-infinita*:
fixes $P :: \text{nat} \Rightarrow 'a \text{ set}$
assumes $\forall n. \forall m. n \neq m \longrightarrow P \ n \cap P \ m = \{\}$
and $\forall n. (A \cap (P \ n)) \neq \{\}$
shows *infinite ($\bigcup n. (A \cap (P \ n))$)*
proof–
let $?f = \lambda x. \text{SOME } n. x \in (A \cap (P \ n))$
have $\forall n. \exists x \in (\bigcup n. (A \cap (P \ n))). n = ?f(x)$
proof
fix n
obtain a **where** $a \in (A \cap (P \ n))$ **using** *assms(2)* **by** *auto*
{fix m
have $a \in (A \cap (P \ m)) \longrightarrow m = n$
proof(*rule impI*)
assume $hip: a \in A \cap P \ m$ **show** $m = n$
proof(*rule ccontr*)
assume $m \neq n$
hence $P \ m \cap P \ n = \{\}$ **using** *assms(1)* **by** *auto*
thus *False* **using** $a \ hip$ **by** *auto*
qed
qed}
hence $\bigwedge m. a \in A \cap P \ m \Longrightarrow m = n$ **by** *auto*
hence $1: ?f(a) = n$ **using** a *some-equality* **by** *auto*

have $a \in (\bigcup n. (A \cap (P\ n)))$ **using** a **by** *auto*
thus $\exists x \in \bigcup n. A \cap P\ n. n = (SOME\ n. x \in A \cap P\ n)$ **using** 1 **by** *auto*
qed
hence $\exists f :: 'a \Rightarrow nat. \forall n. \exists x \in (\bigcup n. (A \cap (P\ n))). n = f(x)$
using *exI* **by** *auto*
thus *?thesis* **using** *sobreyectiva-infinito* **by** *auto*
qed

lemma *rama-infinita*:

assumes *arbol-infinito* $A\ r$ **and** *ramificacion-finita* $A\ r$
and $I: \forall F \in (\mathcal{F}'\ A\ r). valor\ I\ F = Verdad$
shows *infinite* $(\mathcal{B}\ A\ I)$

proof–

have $a: \forall n. \forall m. n \neq m \longrightarrow nivel\ A\ r\ n \cap nivel\ A\ r\ m = \{\}$
using *niveles-distintosa*[*of* - - $A\ r$] **by** *auto*
have $\forall n. \mathcal{B}\ A\ I \cap nivel\ A\ r\ n \neq \{\}$
using $\langle arbol-infinito\ A\ r \rangle$
 $\langle ramificacion-finita\ A\ r \rangle\ I$ *interseccion-rama-nivel1*[*of* $A\ r$]
by *blast*
hence *infinite* $(\bigcup n. (\mathcal{B}\ A\ I) \cap nivel\ A\ r\ n)$
using *familia-interseccion-infinita* a **by** *auto*
thus *infinite* $(\mathcal{B}\ A\ I)$ **by** *auto*
qed

theorem *Konig*:

assumes *arbol-infinito* $(A :: 'nodos\ set)\ r$
and *enumeracion* $(g :: nat \Rightarrow 'nodos)$
and *ramificacion-finita* $A\ r$
shows $\exists B. rama-infinita\ B\ A\ r$
proof–
have *satisfacible* $(\mathcal{T}'\ A\ r)$
proof–
have $\forall S. S \subseteq (\mathcal{T}'\ A\ r) \wedge (finite\ S) \longrightarrow satisfacible\ S$
using $\langle arbol-infinito\ A\ r \rangle\ \langle ramificacion-finita\ A\ r \rangle$ *camino-satisfacible*
by *auto*
moreover
have $\exists h. enumeracion\ (h :: nat \Rightarrow 'nodos\ formula)$
using *EnumeracionFormulasP1*[*OF* $\langle enumeracion\ (g :: nat \Rightarrow 'nodos) \rangle$]
by *auto*
ultimately
show *satisfacible* $(\mathcal{T}'\ A\ r)$
using *TeoremaCompacidad1*[*of* $(\mathcal{T}'\ A\ r)$] **by** *auto*
qed

hence $\exists I. (\forall F \in (\mathcal{T}' A r). \text{valor } I F = \text{Verdad})$
by (*unfold satisfacible-def*, *unfold modelo-def*, *auto*)
then obtain I **where** $I: \forall F \in (\mathcal{T}' A r). \text{valor } I F = \text{Verdad}$
by *auto*
hence $\forall F \in (\mathcal{F}' A r) \cup (\mathcal{G}' A r) \cup (\mathcal{H}' A r). \text{valor } I F = \text{Verdad}$
by (*unfold $\mathcal{T}'$ -def*)
hence $I1: \forall F \in (\mathcal{F}' A r). \text{valor } I F = \text{Verdad}$
and $I2: \forall F \in (\mathcal{G}' A r). \text{valor } I F = \text{Verdad}$
and $I3: \forall F \in (\mathcal{H}' A r). \text{valor } I F = \text{Verdad}$
by *auto*
let $?B = (\mathcal{B} A I)$
have *rama-infinita* $?B A r$
proof (*unfold rama-infinita-def*)
show *rama* $?B A r \wedge \text{infinite } ?B$
proof (*rule conjI*)
show *rama* $?B A r$
using $\langle \text{arbol-infinito } A r \rangle \langle \text{ramificacion-finita } A r \rangle I \text{ rama}[of A r]$
by *auto*
show *infinite* $(\mathcal{B} A I)$
using $\langle \text{arbol-infinito } A r \rangle \langle \text{ramificacion-finita } A r \rangle I1 \text{ rama-infinita}$
by *auto*
qed
qed
thus $\exists B. \text{rama-infinita } B A r$ **by** *auto*
qed
end

Referencias

- [1] M. Fitting. *First-Order Logic and Automated Theorem Proving*. Springer-Verlag, second edition, 1996.
- [2] G. Gonthier. The four colour theorem: Engineering of a formal proof. In D. Kapur, editor, *Computer Mathematics, 8th Asian Symposium, ASCM 2007, Singapore, December 15-17, 2007. Revised and Invited Papers*, volume 5081 of *Lecture Notes in Computer Science*, page 333. Springer, 2007.
- [3] G. Gonthier. Formal proof – the four-color theorem. *Notices of the AMS*, 55(11):1382–1393, 2008.

- [4] J. Harrison, J. Urban, and F. Wiedijk. History of interactive theorem proving. In J. H. Siekmann, editor, *Computational Logic*, volume 9 of *Handbook of the History of Logic*, pages 135–214. Elsevier, 2014.
- [5] D. Jiang and T. Nipkow. Hall’s marriage theorem. *Archive of Formal Proofs*, Dec. 2010. <http://isa-afp.org/entries/Marriage.html>, Formal proof development.
- [6] D. Jiang and T. Nipkow. Proof pearl: The marriage theorem. In *Certified Programs and Proofs - First International Conference, CPP 2011, Kenting, Taiwan, December 7-9, 2011. Proceedings*, pages 394–399, 2011.
- [7] T. Nipkow. Structured proofs in isar/hol. In *Types for Proofs and Programs, Second International Workshop, TYPES 2002, Berg en Dal, The Netherlands, April 24-28, 2002, Selected Papers*, pages 259–278, 2002.
- [8] T. Nipkow and L. C. Paulson. Proof pearl: Defining functions over finite sets. In J. Hurd and T. F. Melham, editors, *Theorem Proving in Higher Order Logics, 18th International Conference, TPHOLs 2005, Oxford, UK, August 22-25, 2005, Proceedings*, volume 3603 of *Lecture Notes in Computer Science*, pages 385–396. Springer, 2005.
- [9] T. Nipkow, L. C. Paulson, and M. Wenzel. *Isabelle/HOL - A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer, 2002.
- [10] T. Ridge. Ramsey’s theorem, infinitary version. *Archive of Formal Proofs*, Sept. 2004. <http://isa-afp.org/entries/Ramsey-Infinite.html>, Formal proof development.
- [11] A. K. Singh. Full Mechanized Proofs of Dilworths Theorem and Mirskys Theorem. *CoRR*, abs/1703.06133, 2017.
- [12] M. Wenzel. *Isabelle, Isar - a versatile environment for human readable formal proof documents*. PhD thesis, Technical University Munich, Germany, 2002.
- [13] F. Wiedijk. *Formalizing 100 theorems*. <http://www.cs.ru.nl/~freek/100/>.
- [14] F. Wiedijk, editor. *The Seventeen Provers of the World*, volume 3600 of *LNAI*, 2006.